

Faculty of Information Technology, Monash University

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice

FIT2004: Algorithms and Data Structures

Week 5: Greedy (Graph) Algorithms

Overview

Divide and
conquer
(W 1-3)

Greedy
algorithms
(W 4-5)

Dynamic
programming
(W 6-7)

Network flow
(W 8-9)

Data
structures
(W 10-11)

- Last Lecture
 - Introduction to Graphs
 - Graph Traversal Algorithms
- Today's Lecture
 - Dijkstra's Algorithm
 - Prim's Algorithm
 - Kruskal's Algorithm

Greedy Algorithm

- Greedy paradigm for algorithms: make a **locally optimal** choice at each stage and commit to it.
- Often it is easy to come up with ideas to solve a problem using a greedy strategy, but in **many cases the proposed greedy algorithm does not find the overall optimal solution** for the problem.
- **Correctness proofs are very important.**
- Today we will take a look at three very important greedy algorithms:
 - Dijkstra's algorithm solves the shortest path problem in graphs with non-negative weights.
 - Prim's and Kruskal's algorithms present two different ways to find a minimum spanning tree of a graph (an important problem for which there are actually multiple distinct greedy strategies that work fine).

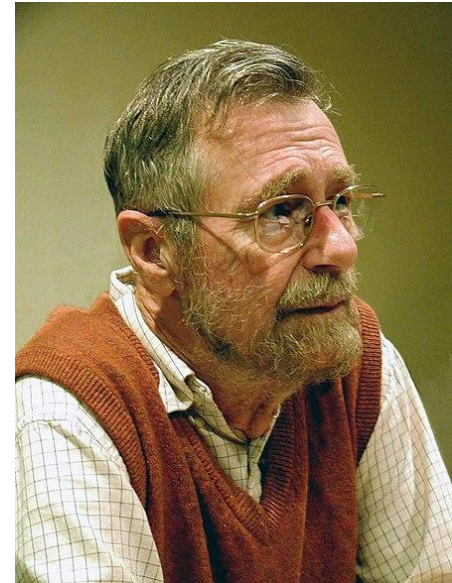
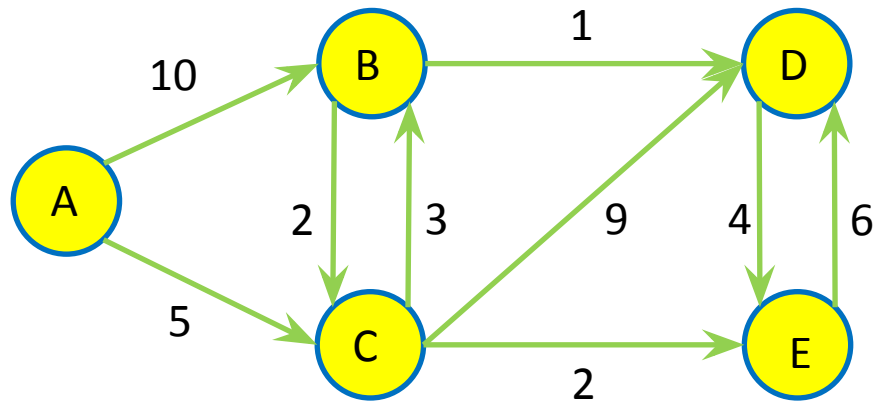
Outline

1. Dijkstra's Algorithm
2. Prim's Algorithm
3. Kruskal's Algorithm

Dijkstra's Algorithm

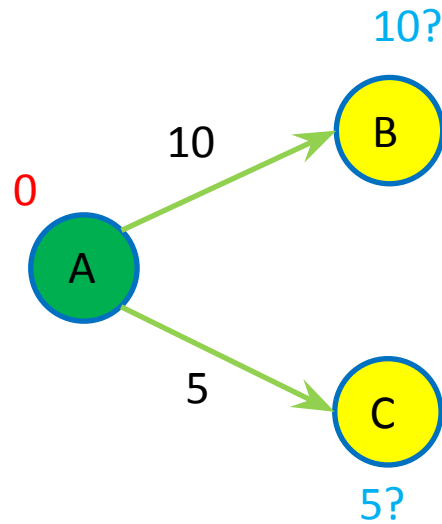
- Algorithm for solving the single source, all targets shortest path problem on graphs with **non-negative weights**. Closely related to BFS.
- It keeps track of a set S of nodes whose distance to the source has already been determined.
- Initially S only contains the source node, and it **grows by one element** at a time until containing all nodes that are reachable from the source node.
- At each iteration, from all nodes that are one edge away from S , **add to S the node u that has the smallest distance to the source**.
- By doing so, the overall effect is that the nodes will be added to S in increasing order of distance to the source (as we will see soon).

Dijkstra's Algorithm

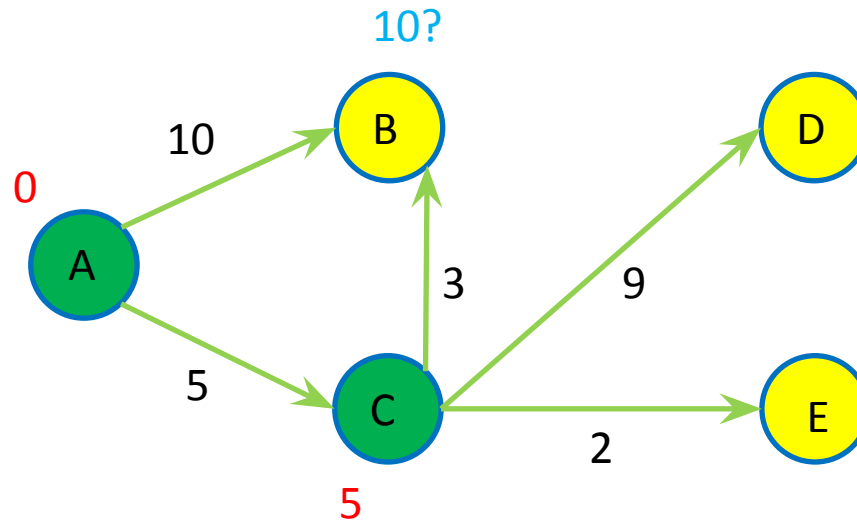


Edsger W. Dijkstra
Turing Award 1972

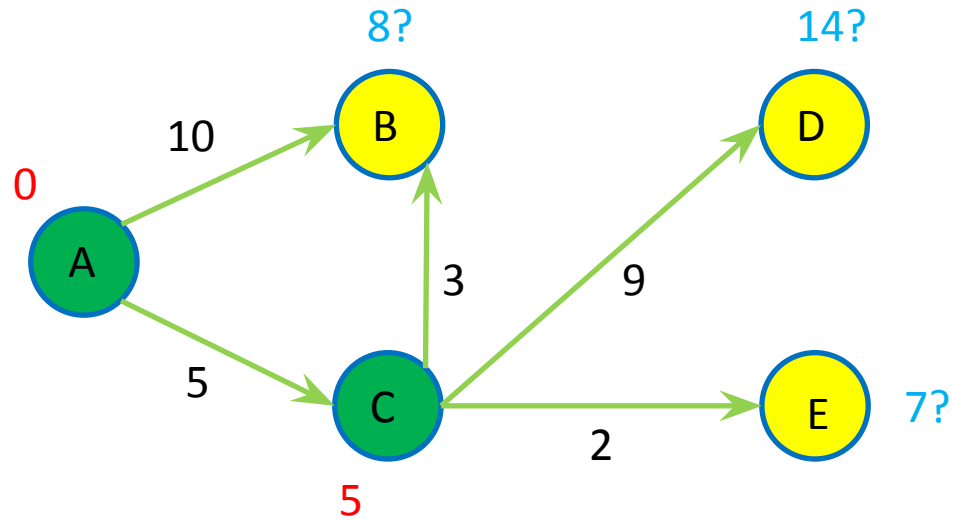
Dijkstra's Algorithm



Dijkstra's Algorithm

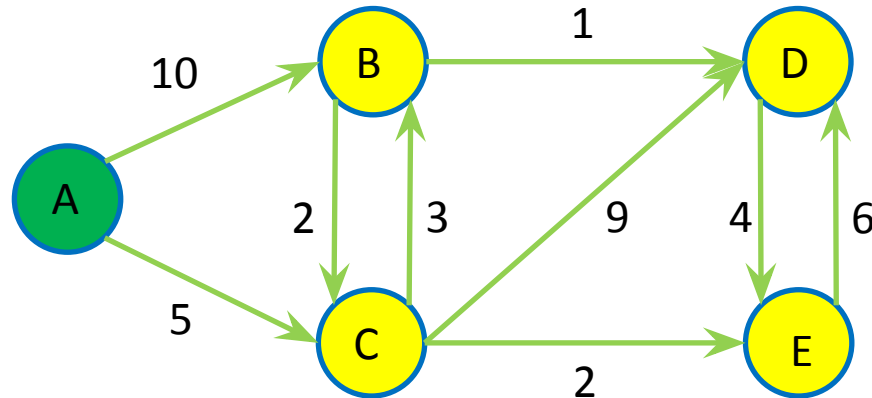


Dijkstra's Algorithm



Dijkstra's Algorithm

u:



Q:

A	B	C	D	E
0	Inf	Inf	Inf	Inf

Q is a priority queue, where priority is based on distance

Pred:

A	B	C	D	E
-	-	-	-	-

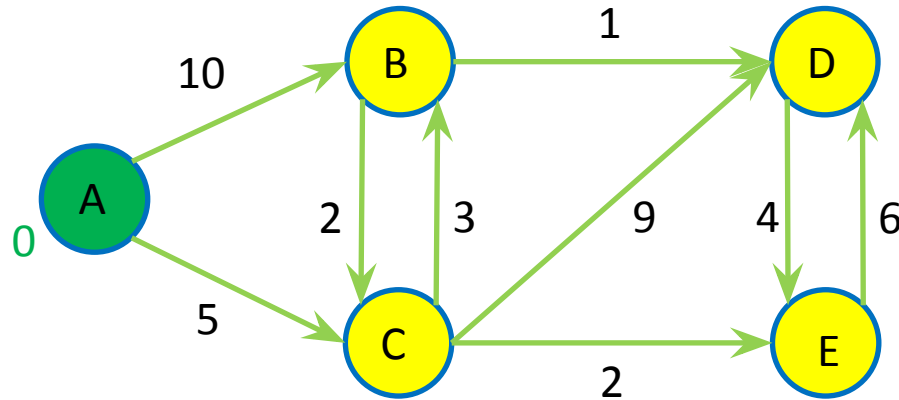
Pred and Dist are the usual ID-indexed arrays

Dist:

A	B	C	D	E
0	Inf	Inf	Inf	Inf

Dijkstra's Algorithm

u: **A**



Q:

B	C	D	E
Inf	Inf	Inf	Inf

Q is a priority queue, where priority is based on distance

Pred:

A	B	C	D	E
-	-	-	-	-

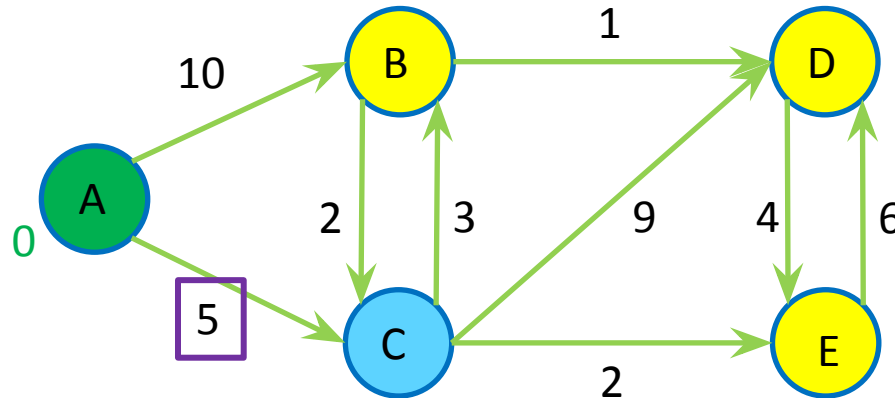
Pred and Dist are the usual ID-indexed arrays

Dist:

A	B	C	D	E
0	Inf	Inf	Inf	Inf

Dijkstra's Algorithm

u: **A**



Q:

B	C	D	E
Inf	Inf	Inf	Inf

Pred:

A	B	C	D	E
-	-	-	-	-

Dist:

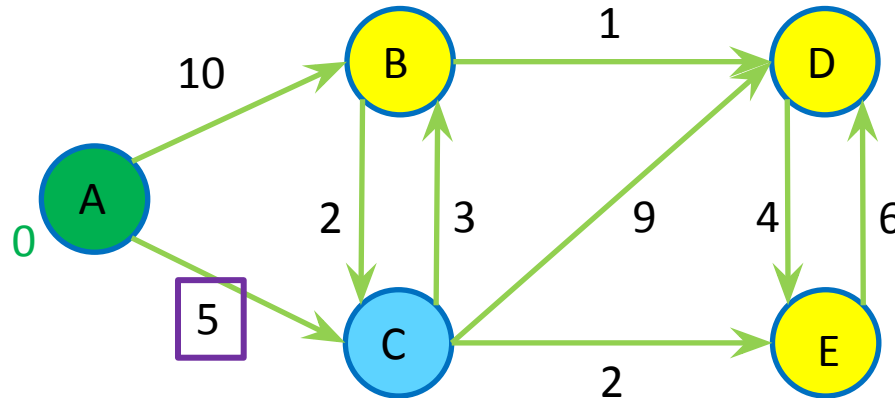
A	B	C	D	E
0	Inf	Inf	Inf	Inf

For each neighbour v of u , relax along that edge

- If $\text{dist}[u] + w(u,v) < \text{dist}[v]$
- Update $\text{dist}[v]$
- Set $\text{pred}[v] = u$

Dijkstra's Algorithm

u: **A**



Q:

B	C	D	E
Inf	Inf	Inf	Inf

Pred:

A	B	C	D	E
-	-	-	-	-

Dist:

A	B	C	D	E
0	Inf	Inf	Inf	Inf

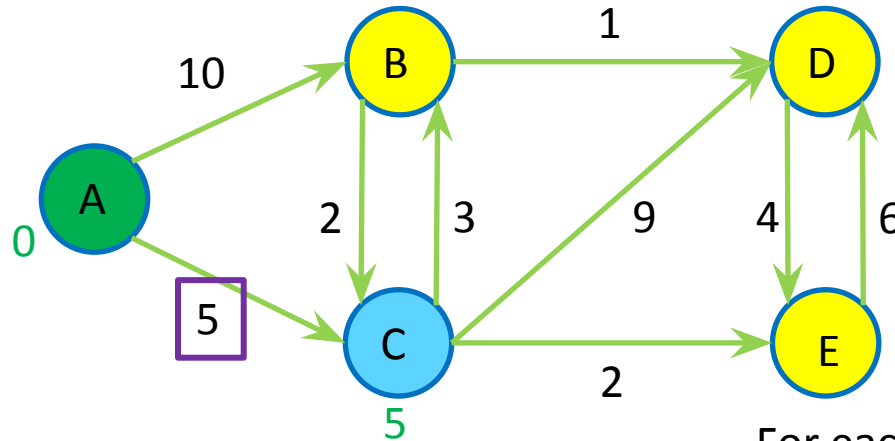
For each neighbour v of u , relax along that edge

- If $\text{dist}[u] + w(u,v) < \text{dist}[v]$
- Update $\text{dist}[v]$
- Set $\text{pred}[v] = u$

- $0 + 5 < \text{Inf}$

Dijkstra's Algorithm

u: **A**



Q:

C	B	D	E
5	Inf	Inf	Inf

Pred:

A	B	C	D	E
-	-	A	-	-

Dist:

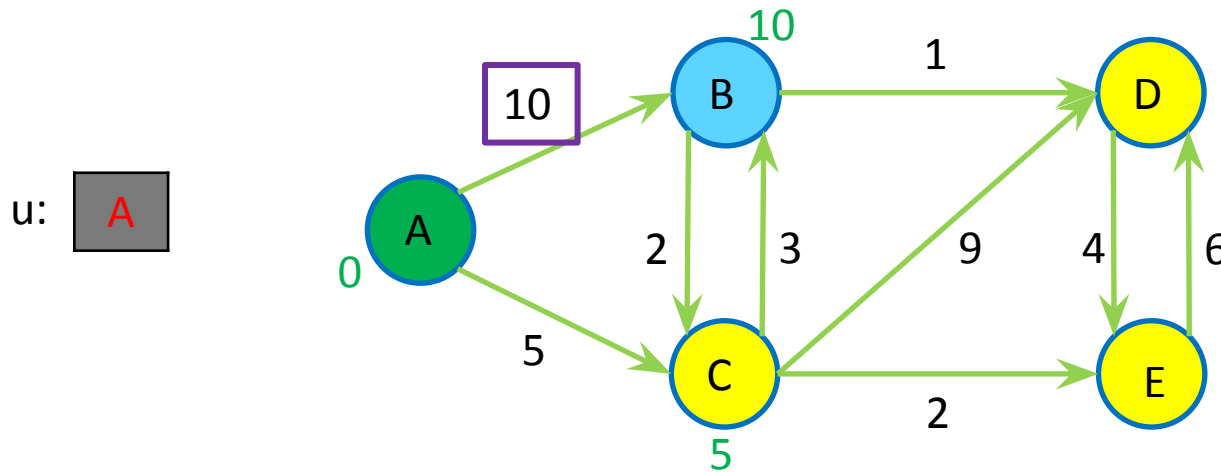
A	B	C	D	E
0	Inf	5	Inf	Inf

For each neighbour v of u , relax along that edge

- If $\text{dist}[u] + w(u,v) < \text{dist}[v]$
- Update $\text{dist}[v]$
- Set $\text{pred}[v] = u$

- $0 + 5 < \text{Inf}$
- Set $\text{dist}[C] = 5$
- Set $\text{pred}[C] = A$
- Note that this changes the order of Q

Dijkstra's Algorithm



Q:

C	B	D	E
5	10	Inf	Inf

Doing the same for B

- $0 + 10 < \text{Inf}$
- $\text{Dist}[B] = 10$

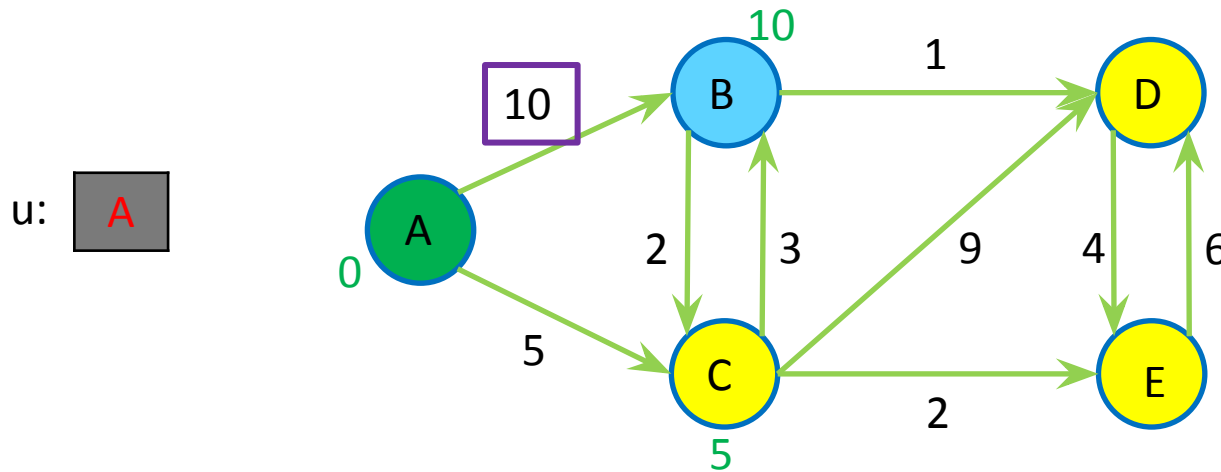
Pred:

A	B	C	D	E
-	-	A	-	-

Dist:

A	B	C	D	E
0	10	5	Inf	Inf

Dijkstra's Algorithm



Q:

C	B	D	E
5	10	Inf	Inf

Pred:

A	B	C	D	E
-	A	A	-	-

Dist:

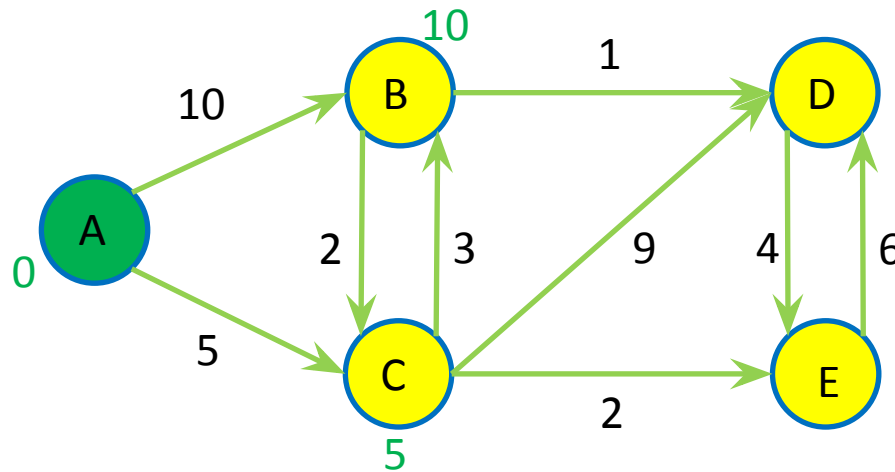
A	B	C	D	E
0	10	5	Inf	Inf

Doing the same for B

- $0 + 10 < \text{Inf}$
- $\text{Dist}[B] = 10$
- $\text{Pred}[B] = A$

Dijkstra's Algorithm

u: **A**



Q:

C	B	D	E
5	10	Inf	Inf

Pred:

A	B	C	D	E
-	A	A	-	-

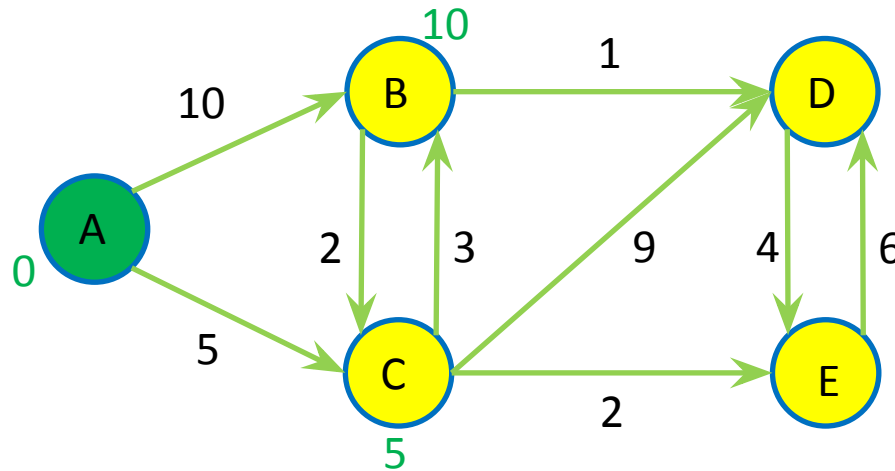
Dist:

A	B	C	D	E
0	10	5	Inf	Inf

- Finished with A, so pop from Q
- Notice that this will always be the vertex with the smallest distance estimate.

Dijkstra's Algorithm

u: **C**



Q:

B	D	E
10	Inf	Inf

Pred:

A	B	C	D	E
-	A	A	-	-

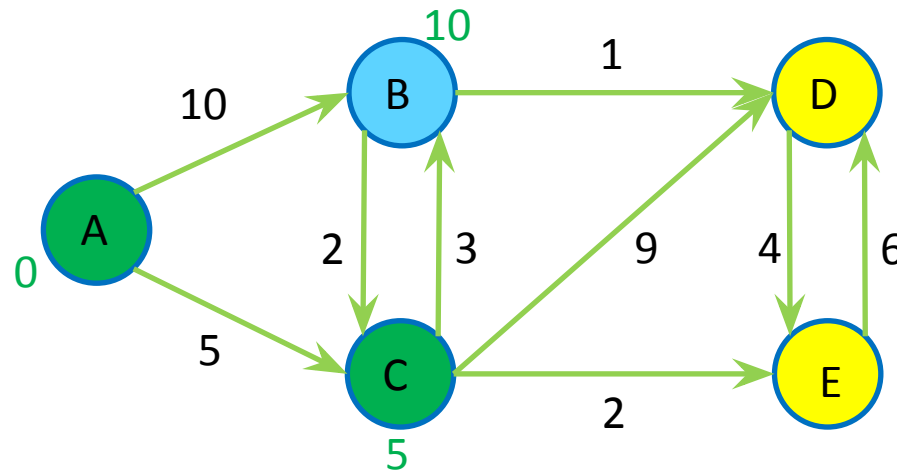
Dist:

A	B	C	D	E
0	10	5	Inf	Inf

- Finished with A, so pop from Q
- Notice that this will always be the vertex with the smallest distance estimate.
- The distance of this vertex is now finalised

Dijkstra's Algorithm

u: **C**



Q:

B	D	E
10	Inf	Inf

- Relax B from C
- $\text{Dist}[C] + w(C, B) = 5 + 3 < 10$

Pred:

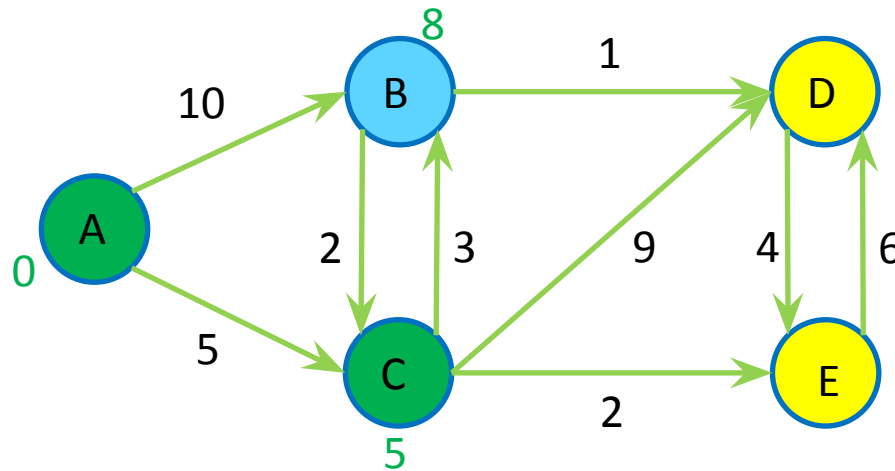
A	B	C	D	E
-	A	A	-	-

Dist:

A	B	C	D	E
0	10	5	Inf	Inf

Dijkstra's Algorithm

u: **C**



Q:

B	D	E
8	Inf	Inf

Pred:

A	B	C	D	E
-	C	A	-	-

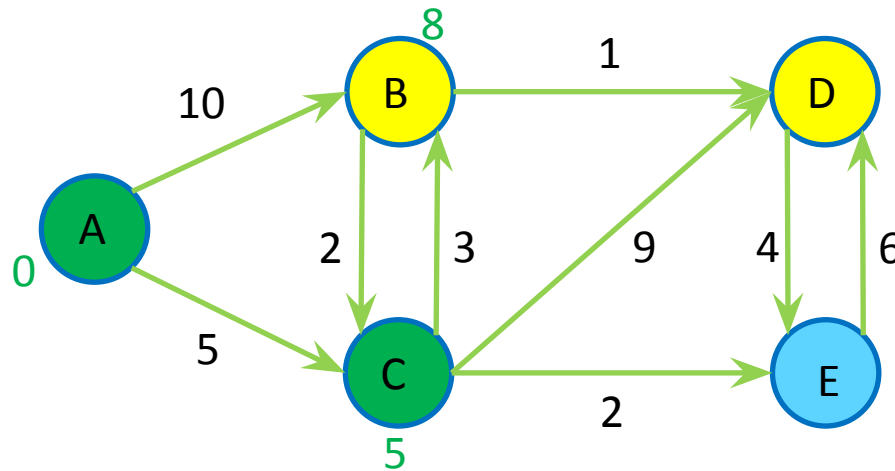
Dist:

A	B	C	D	E
0	8	5	Inf	Inf

- Relax B from C
- $\text{Dist}[C] + w(C, B) = 5 + 3 < 10$
- $\text{Dist}[B] = 8$
- $\text{Pred}[B] = C$

Dijkstra's Algorithm

u: **C**



Q:

B	D	E
8	Inf	Inf

- Relax E from C
- $\text{Dist}[C] + w(C, E) = 5 + 2 < \text{Inf}$

Pred:

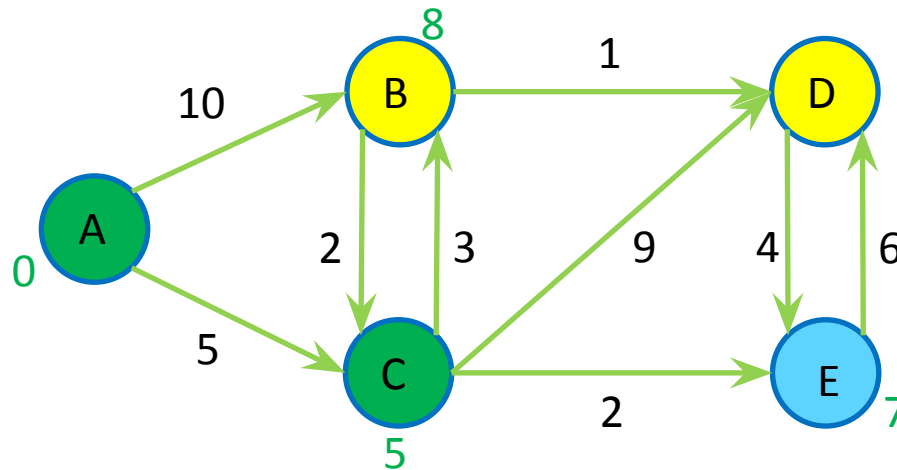
A	B	C	D	E
-	C	A	-	-

Dist:

A	B	C	D	E
0	8	5	Inf	Inf

Dijkstra's Algorithm

u: **C**



Q:

E	B	D
7	8	Inf

Pred:

A	B	C	D	E
-	C	A	-	C

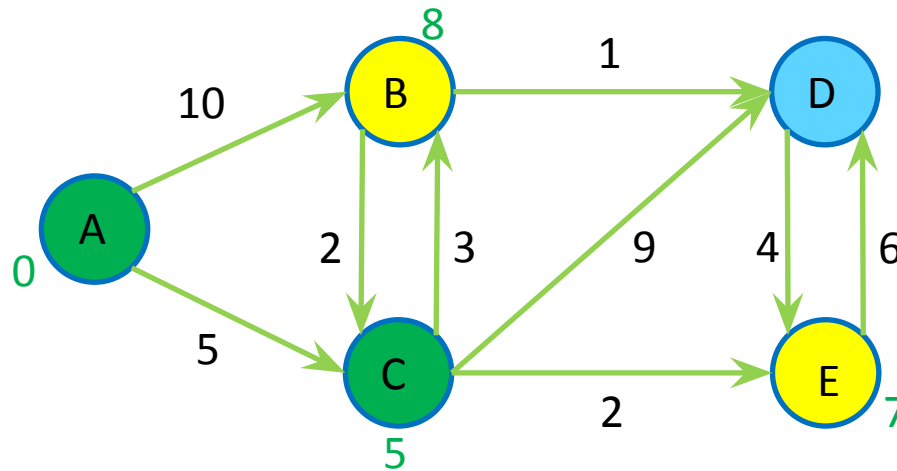
Dist:

A	B	C	D	E
0	8	5	Inf	7

- Relax E from C
- $\text{Dist}[C] + w(C, E) = 5 + 2 < \text{Inf}$
- $\text{Dist}[E] = 7$
- $\text{Pred}[E] = C$

Dijkstra's Algorithm

u: **C**



Q:

E	B	D
7	8	Inf

- Relax D from C
- $\text{Dist}[D] + w(C, D) = 5 + 9 < \text{Inf}$

Pred:

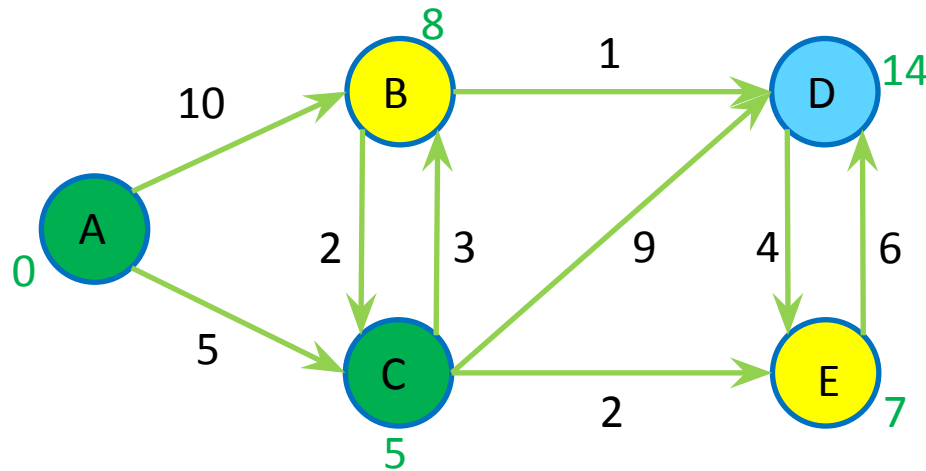
A	B	C	D	E
-	C	A	-	C

Dist:

A	B	C	D	E
0	8	5	Inf	7

Dijkstra's Algorithm

u: **C**



Q:

E	B	D
7	8	14

Pred:

A	B	C	D	E
-	C	A	C	C

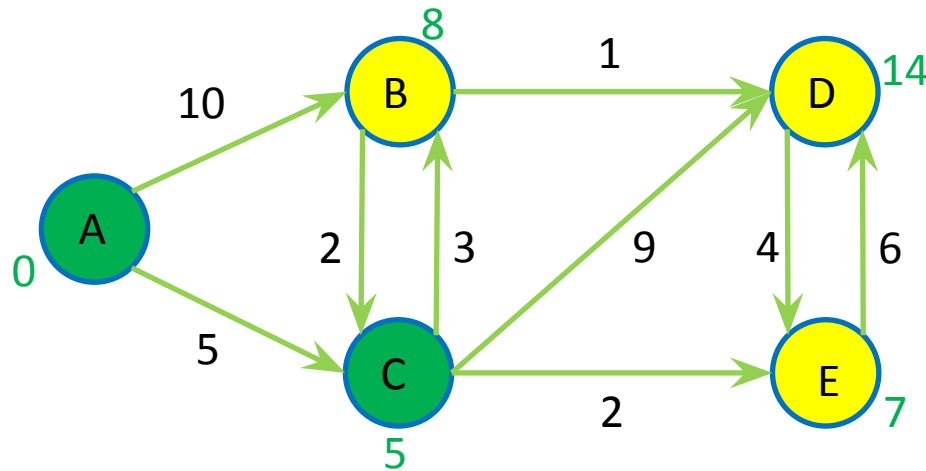
Dist:

A	B	C	D	E
0	8	5	14	7

- Relax D from C
- $\text{Dist}[D] + w(C, D) = 5 + 9 < \text{Inf}$
- $\text{Dist}[D] = 14$
- $\text{Pred}[D] = C$

Dijkstra's Algorithm

u: **C**



Q:

E	B	D
7	8	14

Pred:

A	B	C	D	E
-	C	A	C	C

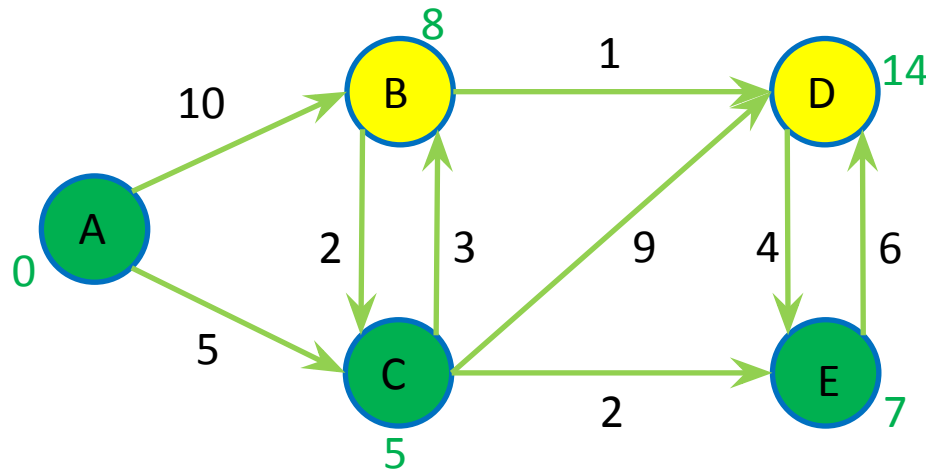
Dist:

A	B	C	D	E
0	8	5	14	7

- Done with C
- Pop another vertex from Q and finalise it

Dijkstra's Algorithm

u: **E**



Q:

B	D
8	14

Pred:

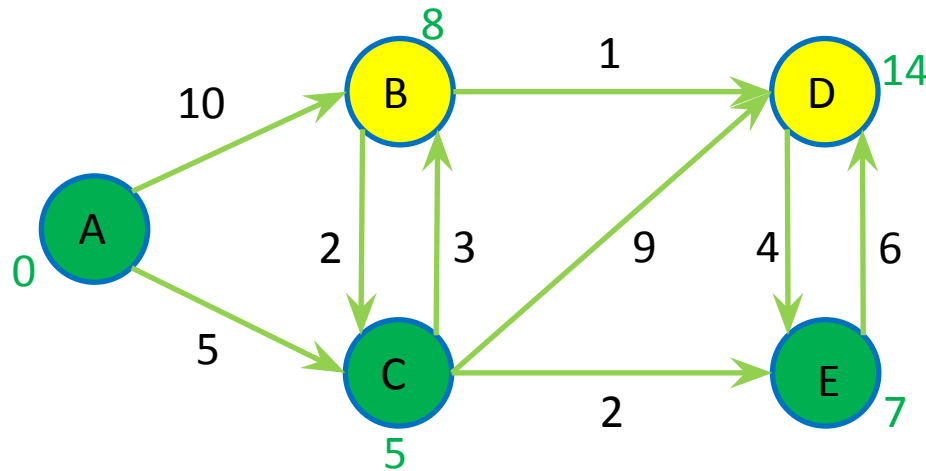
A	B	C	D	E
-	C	A	C	C

Dist:

A	B	C	D	E
0	8	5	14	7

Dijkstra's Algorithm

u: **E**



Q:

B	D
8	14

- Relax D from E
- $\text{Dist}[E] + w(E, D) = 7 + 6 < 14 = \text{Dist}[D]$

Pred:

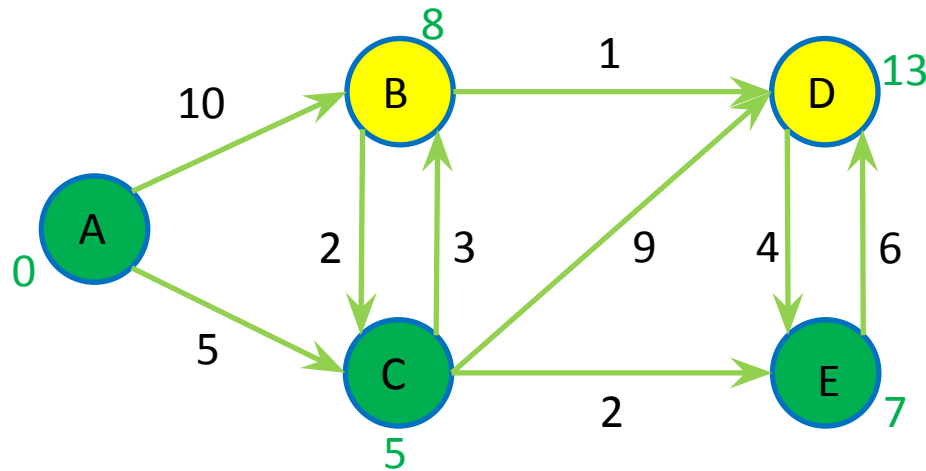
A	B	C	D	E
-	C	A	C	C

Dist:

A	B	C	D	E
0	8	5	14	7

Dijkstra's Algorithm

u: **E**



Q:

B	D
8	13

Pred:

A	B	C	D	E
-	C	A	E	C

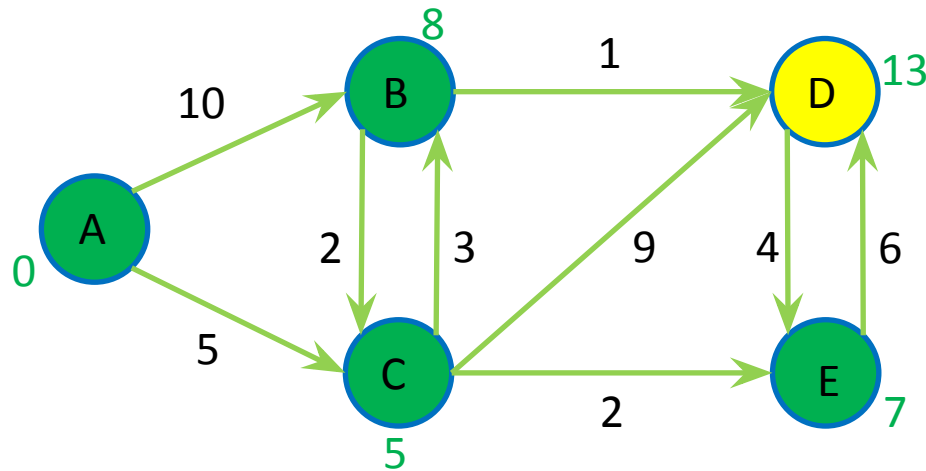
Dist:

A	B	C	D	E
0	8	5	13	7

- Relax D from E
- $\text{Dist}[E] + w(E, D) = 7 + 6 < 14 = \text{Dist}[D]$
- $\text{Dist}[D] = 13$
- $\text{Pred}[D] = D$
- Done with E
- Pop B

Dijkstra's Algorithm

u: **B**



Q:

D
13

- Relax D from B
- $\text{Dist}[B] + w(B, D) = 8 + 1 = 9 < 13 = \text{Dist}[D]$

Pred:

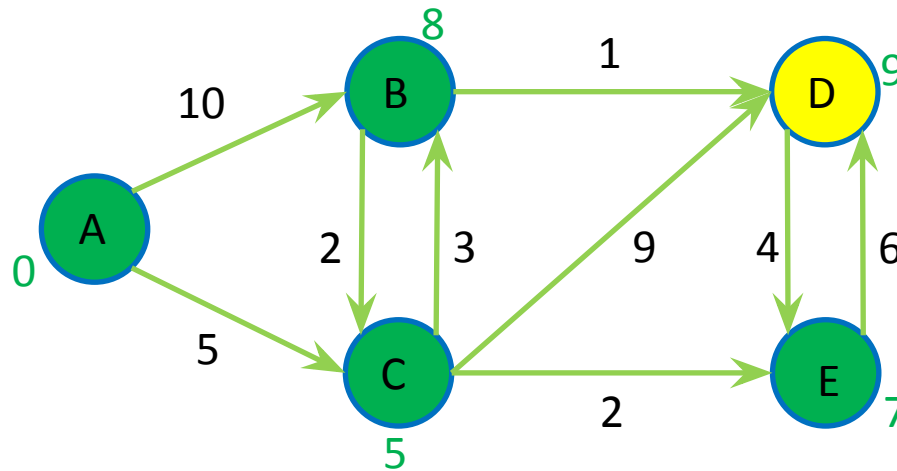
A	B	C	D	E
-	C	A	E	C

Dist:

A	B	C	D	E
0	8	5	13	7

Dijkstra's Algorithm

u: **B**



Q:

D
9

Pred:

A	B	C	D	E
-	C	A	B	C

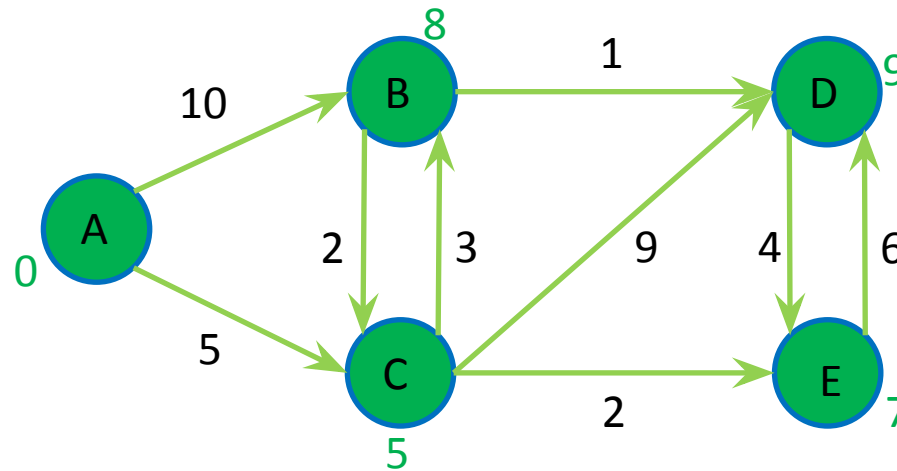
Dist:

A	B	C	D	E
0	8	5	9	7

- Relax D from B
- $\text{Dist}[B] + w(B, D) = 8 + 1 = 9 < 13 = \text{Dist}[D]$
- $\text{Dist}[D] = 9$
- $\text{Pred}[D] = B$
- Done with B
- Pop D

Dijkstra's Algorithm

u: **D**



Q:

- Relax E from D
- $\text{Dist}[D] + w(D, E) = 9 + 4 = 13 > 7$
 - No update

Pred:

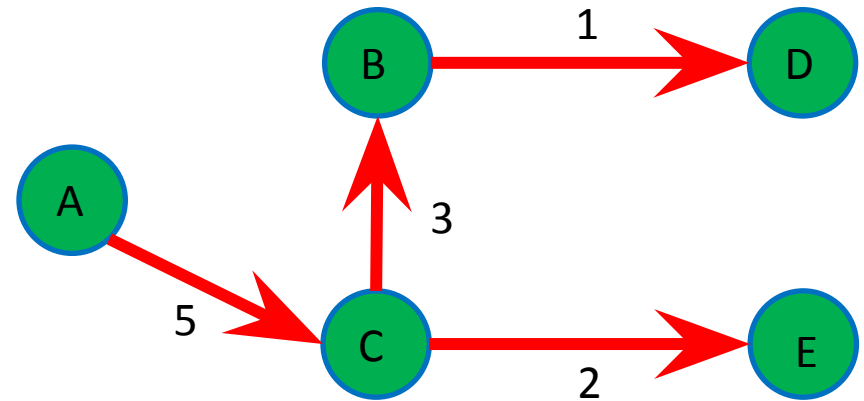
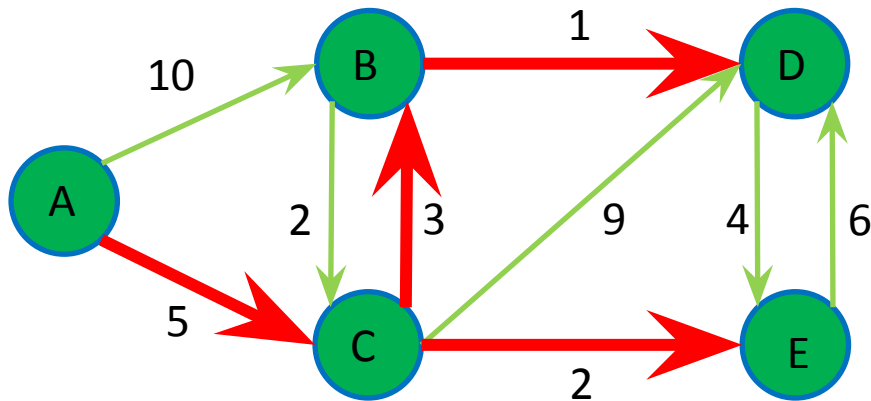
A	B	C	D	E
-	C	A	B	C

Dist:

A	B	C	D	E
0	8	5	9	7

- Done!

Dijkstra's Algorithm



Q:

Pred:

A	B	C	D	E
-	C	A	B	C

Dist:

A	B	C	D	E
0	8	5	9	7

Dijkstra's Algorithm

Algorithm 38 Dijkstra's algorithm

```
1: function DIJKSTRA( $G = (V, E)$ ,  $s$ )
2:    $dist[1..n] = \infty$ 
3:    $pred[1..n] = 0$ 
4:    $dist[s] = 0$ 
5:    $Q = \text{priority\_queue}(V[1..n], \text{key}(v) = dist[v])$ 
6:   while  $Q$  is not empty do
7:      $u = Q.\text{pop\_min}()$ 
8:     for each edge  $e$  that is adjacent to  $u$  do
9:       // Priority queue keys must be updated if relax improves a distance estimate!
10:      RELAX( $e$ )
11:   return  $dist[1..n], pred[1..n]$ 
```

Algorithm 37 Edge relaxation

```
1: function RELAX( $e = (u, v)$ )
2:   if  $dist[v] > dist[u] + w(u, v)$  then
3:      $dist[v] = dist[u] + w(u, v)$ 
4:      $pred[v] = u$ 
```



Dijkstra's Algorithm

Algorithm 38 Dijkstra's algorithm

```
1: function DIJKSTRA( $G = (V, E)$ ,  $s$ )
2:    $dist[1..n] = \infty$ 
3:    $pred[1..n] = 0$ 
4:    $dist[s] = 0$ 
5:    $Q = \text{priority\_queue}(V[1..n], \text{key}(v) = dist[v])$ 
6:   while  $Q$  is not empty do
7:      $u = Q.\text{pop\_min}()$ 
8:     for each edge  $e$  that is adjacent to  $u$  do
9:       // Priority queue keys must be updated if relax improves a distance estimate!
10:      RELAX( $e$ )
11:   return  $dist[1..n], pred[1..n]$ 
```

Time Complexity:

- While loop executes $\Theta(V)$ times
 - Find the vertex with smallest distance and delete it: depends on priority queue implementation ($Q.\text{extract_min}$)
- Each edge visited once $\rightarrow \Theta(E)$
 - Updating the priority queue: depends on implementation ($Q.\text{update}$)
 - Relaxation is $\Theta(1)$ since we can find distances and compare them in $\Theta(1)$
- Total cost: $\Theta(E * Q.\text{update} + V * Q.\text{extract_min})$

Dijkstra's Algorithm using min-heap

Required additional data structure:

- Create an array called **Vertices**.
- **Vertices**[i] will record the **location** of i-th vertex in the min-heap

Updating the distance of a vertex v in min-heap in $\Theta(\log V)$

- Find the location in the queue (heap) in $\Theta(1)$ using **Vertices**
- Now do the normal heap-up operation in $\Theta(\log V)$
 - For each swap performed between two vertices x and y during the upHeap
 - Update **Vertices**[x] and **Vertices**[y] to record their updated **locations** in the min-heap

Time Complexity of Dijkstra's Algorithm

- While loop executes $\Theta(V)$ times
 - Extract the vertex with smallest distance using min_heap: $\Theta(\log V)$
- Each edge visited once $\rightarrow \Theta(E)$
 - Updating the priority queue using min_heap: $\Theta(\log V)$
 - Relaxation is $\Theta(1)$ since we can find distances and compare them in $\Theta(1)$
- Total cost: $\Theta(E * Q.\text{update} + V * Q.\text{extract_min})$
- Total cost: $\Theta(E \log V + V \log V)$
 - Which term dominates?
- If the graph is connected, then $E \geq V-1$
 - E dominates V
- Total cost: $\Theta(E \log V)$

Proof of Correctness

Claim: For every vertex v which has been removed from the queue, $\text{dist}[v]$ is correct.

- Notation:
 - V is the set of vertices
 - Q is the set of vertices in the queue
 - $S = V / Q$ = the set of vertices who have been removed from the queue

Base Case

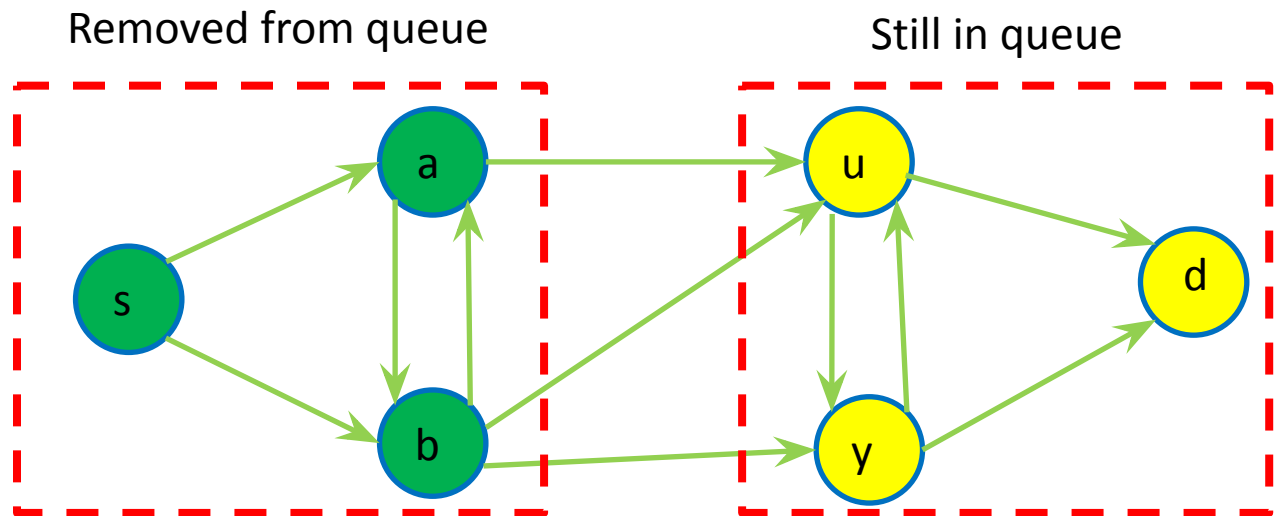
- $\text{dist}[s]$ is initialised to 0, which is the shortest distance from s to s (since there are no negative weights).

Proof of Correctness

Claim: For every vertex v which has been removed from the queue, $\text{dist}[v]$ is correct.

Inductive Step:

- Assume that the claim holds for all vertices which have been removed from the queue, in other words it holds for all vertices in the set S .
- Let u be the next vertex which is removed from the queue.
- We will show that $\text{dist}[u]$ is correct.



Proof of Correctness

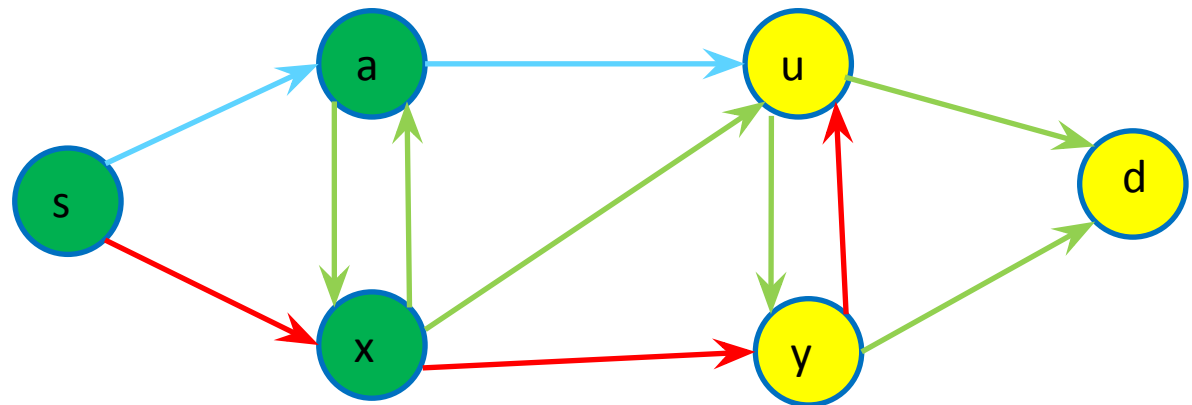
Claim: For every vertex v which has been removed from the queue, $\text{dist}[v]$ is correct.

Inductive Step:

- Suppose (for contradiction) there is a shortest path P , $s \rightsquigarrow u$ with $\text{len}(P) < \text{dist}[u]$.
- Let x be the furthest vertex on P which is in S (i.e. has been finalised).
- By the inductive hypothesis, $\text{dist}[x]$ is correct (since it is in S).

Current path

Assumed shorter path (P)

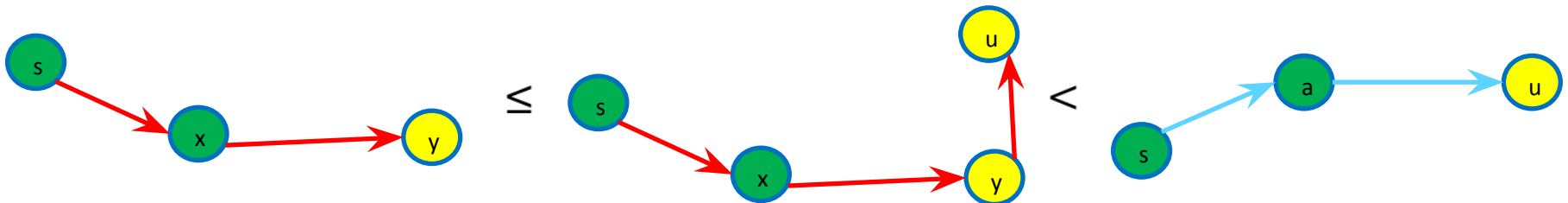
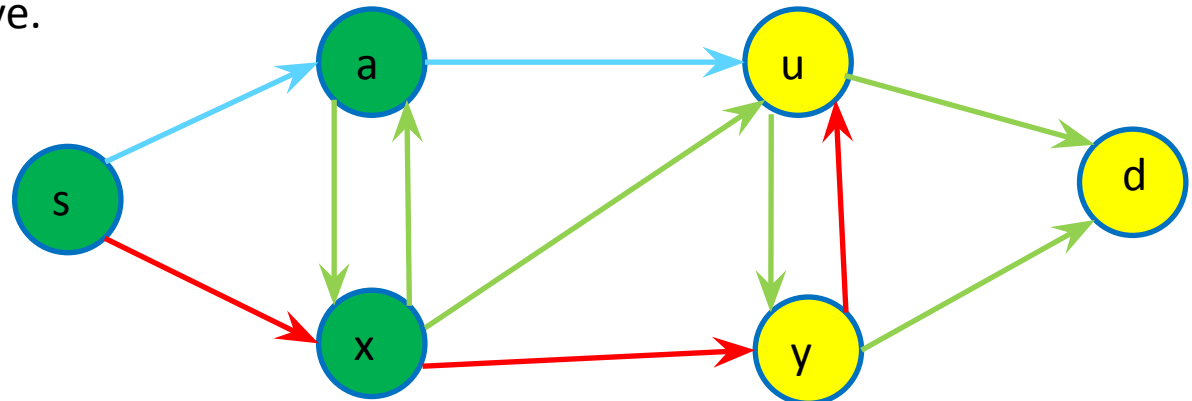


Proof of Correctness

Claim: For every vertex v which has been removed from the queue, $\text{dist}[v]$ is correct.

Inductive Step:

- By the inductive hypothesis, $\text{dist}[x]$ is correct (since it is in S).
- Let y be the next vertex on P after x .
- By assumption $\text{len}(P) < \text{dist}[u]$.
- Edge weights are non-negative.
- $\text{len}(s \rightsquigarrow y) \leq \text{len}(P) < \text{dist}[u]$.

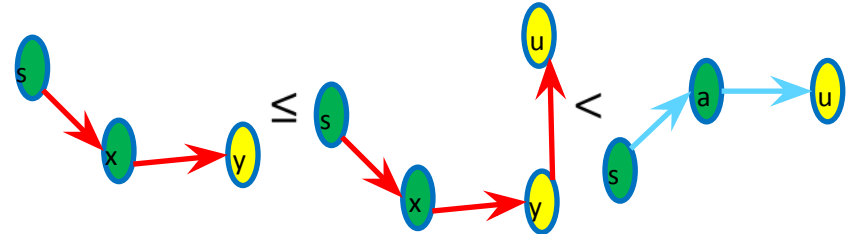


Proof of Correctness

Claim: For every vertex v which has been removed from the queue, $\text{dist}[v]$ is correct.

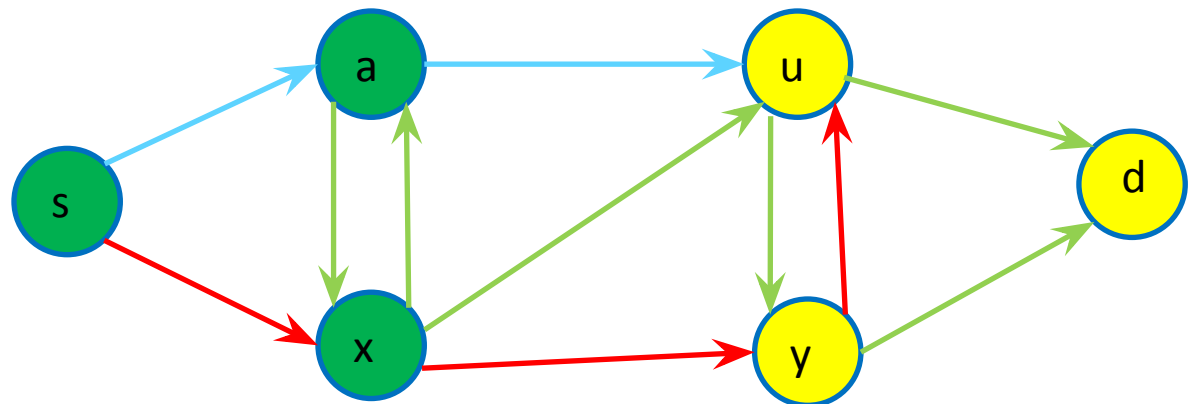
Inductive Step:

- $\text{len}(s \rightsquigarrow y) \leq \text{len}(P) < \text{dist}[u]$
- Since we said that P (via x and y) is a **shortest path**...
- $\text{dist}[y] = \text{len}(s \rightsquigarrow y) < \text{dist}[u]$.
- So $\text{dist}[y] < \text{dist}[u]$...
- If $y \neq u$, why didn't y get removed before u ???
- If $y = u$, how can $\text{dist}[y] < \text{dist}[u]$???



Current path

Assumed shorter path (P)



Proof of Correctness

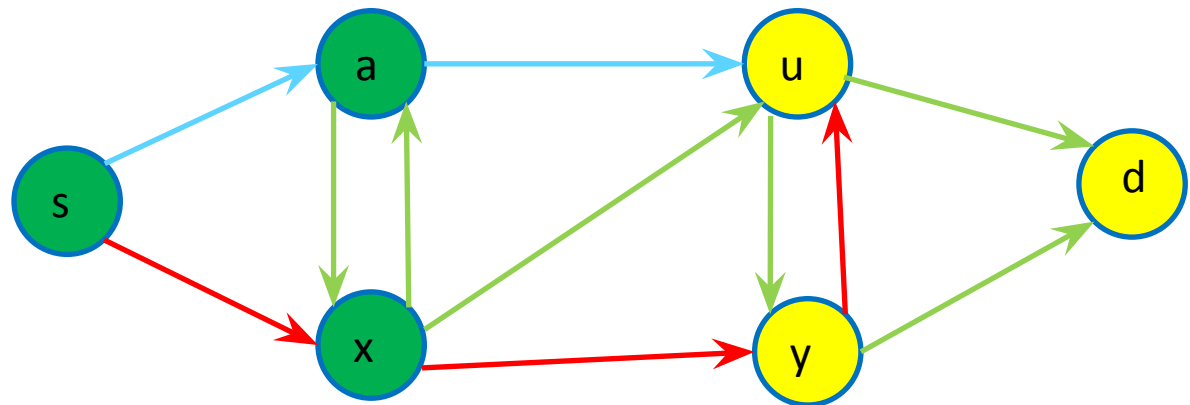
Claim: For every vertex v which has been removed from the queue, $\text{dist}[v]$ is correct.

Inductive Step:

- Having obtained a contradiction, we can negate our assumption, namely:
- “Suppose (for contradiction) there is a shorter path P , $s \rightsquigarrow u$ with $\text{len}(P) < \text{dist}[u]$.”
- So there is no such path and $\text{dist}[u]$ is correct.
- So by induction, the distance of every vertex is correct when Dijkstra’s algorithm terminates.

Current path

Assumed shorter path (P)



Outline

1. Dijkstra's Algorithm
2. Prim's Algorithm
3. Kruskal's Algorithm

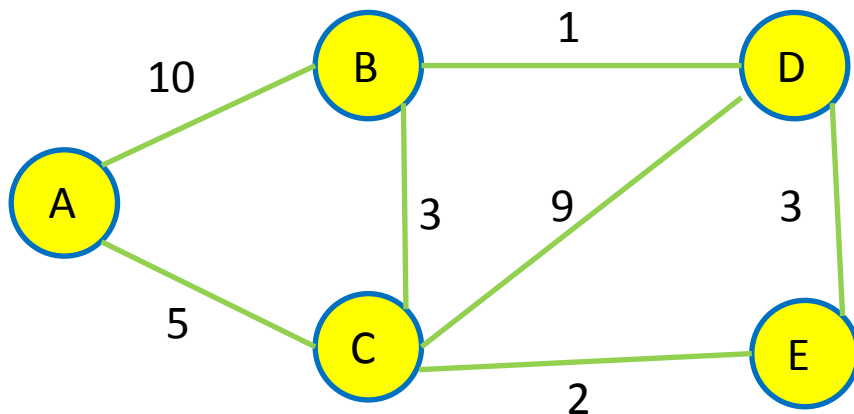
What is a Spanning Tree

Tree:

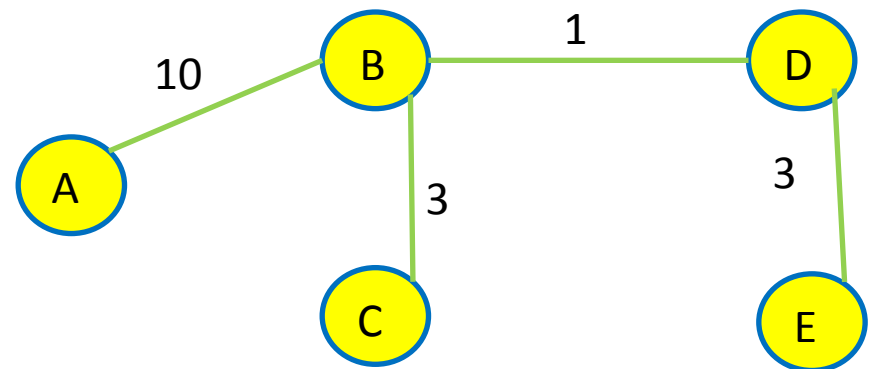
A tree is a connected undirected graph with no cycles in it.

Spanning Tree:

- A **spanning tree** of a general undirected weighted graph G is a tree that **spans** G (i.e., a tree that includes every vertex of G) and is a **subgraph** of G (i.e., every edge in the spanning tree belongs to G).

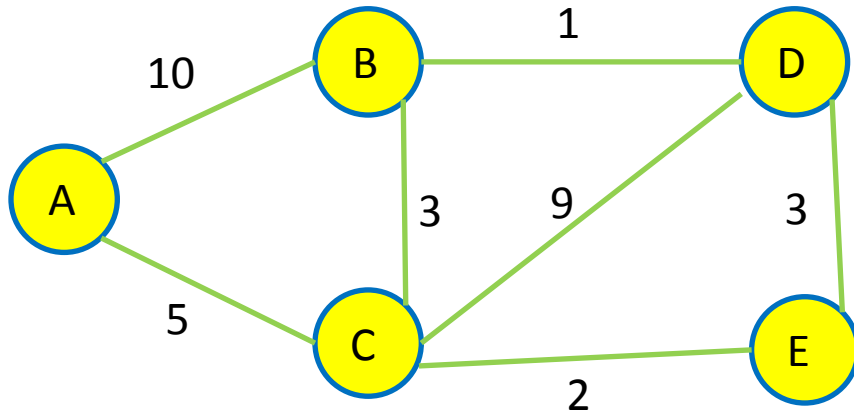


An undirected graph

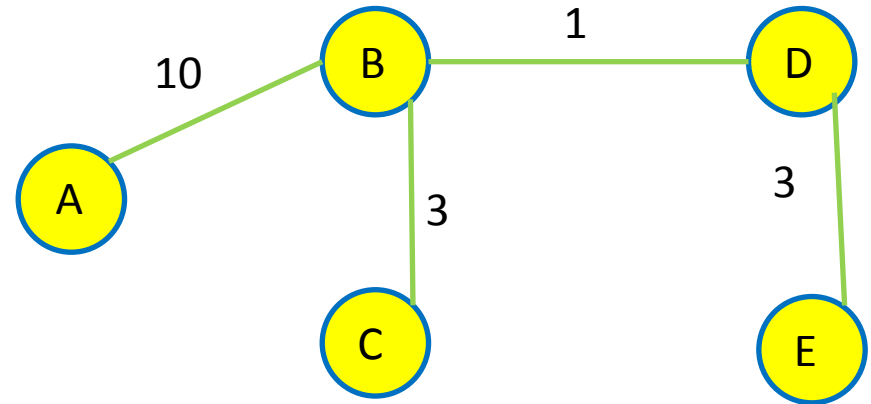


Spanning Tree

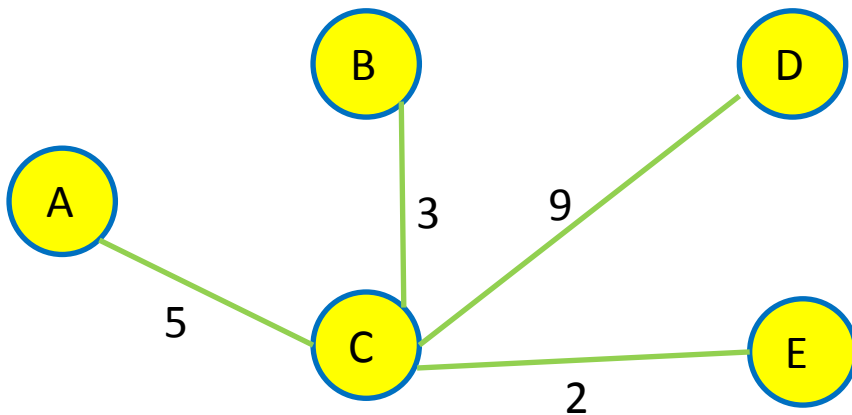
Spanning Tree Examples



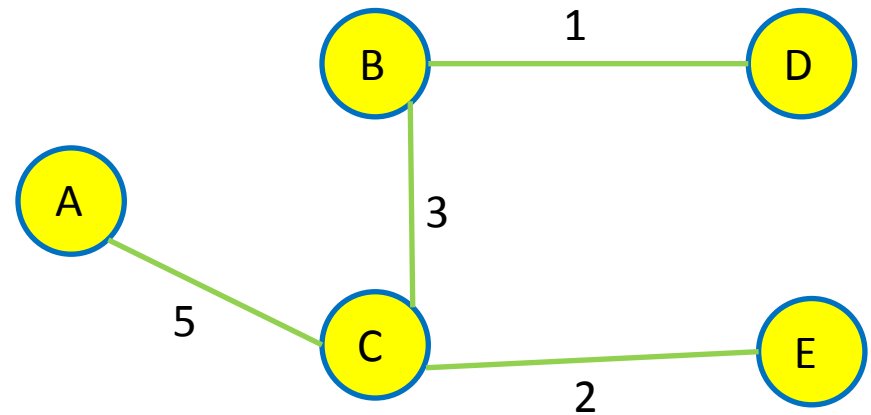
Graph



Spanning Tree 1



Spanning Tree 2



Spanning Tree 3

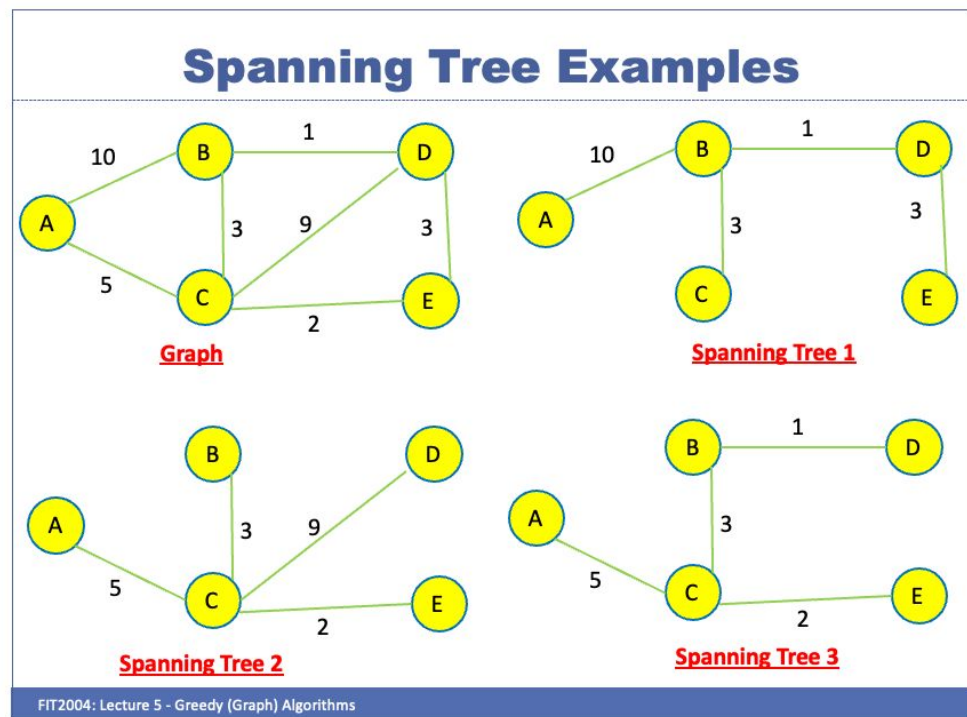
What is a Spanning Tree

Is it true that a spanning tree of a connected graph G is a **maximal set of edges** of G that contains **no cycles**?

- Yes

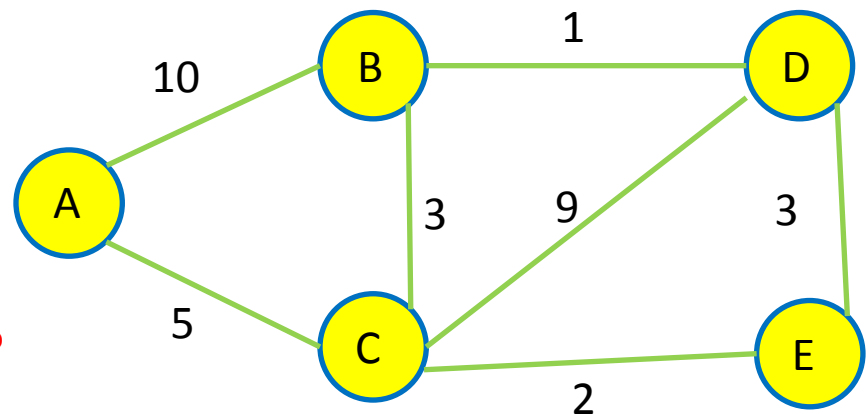
Is it true that a spanning tree of a connected graph G is a **minimal set of edges** that **connect all vertices**?

- Yes



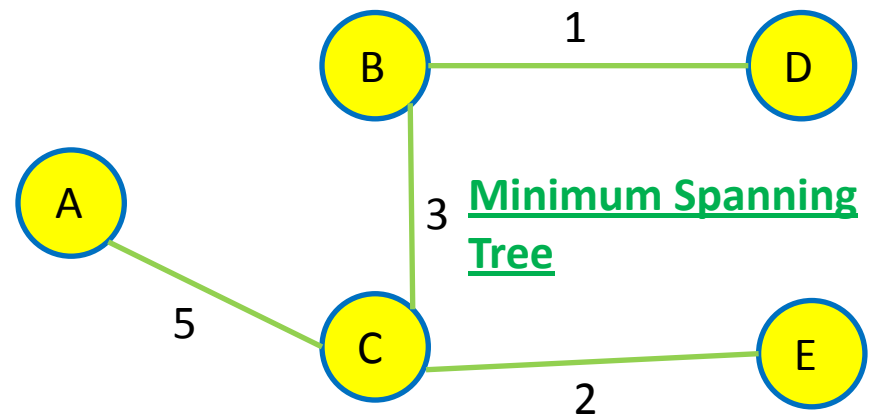
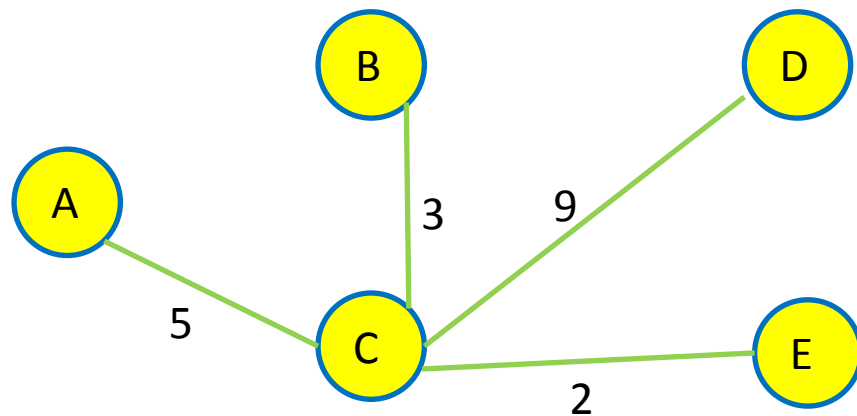
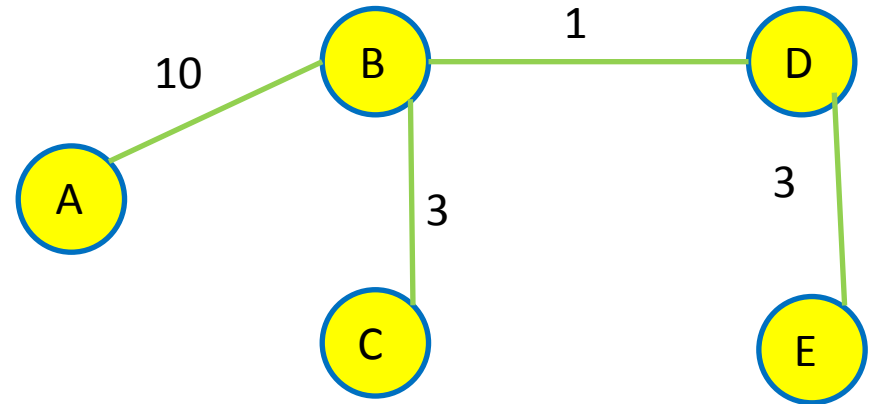
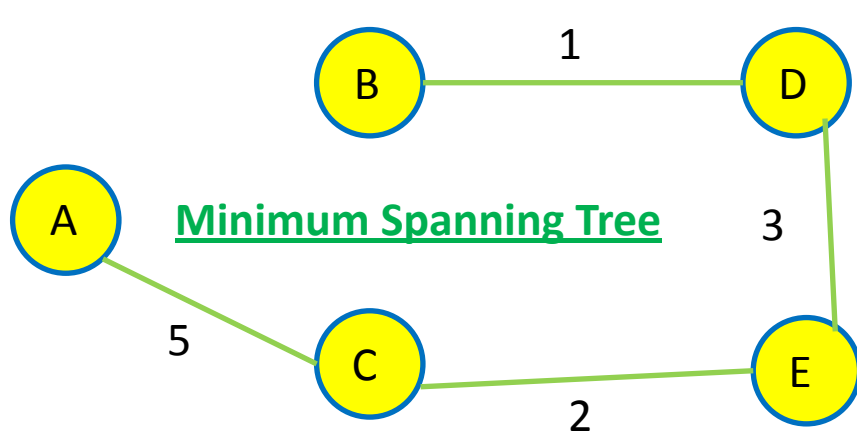
Minimum Spanning Tree (MST)

- Weight of a spanning tree is the sum of the weights of the edges in the tree.
- A **minimum spanning tree** of a **weighted** general graph G is a tree that **spans** G , whose weight is minimum over all possible spanning trees for this graph.
- There may be more than one minimum spanning tree for a graph G (e.g., two or more spanning trees with the same minimum weight).



What is the weight of the MST in this graph?
How many MSTs are in this graph?

Spanning Trees and MSTs



MST Algorithms

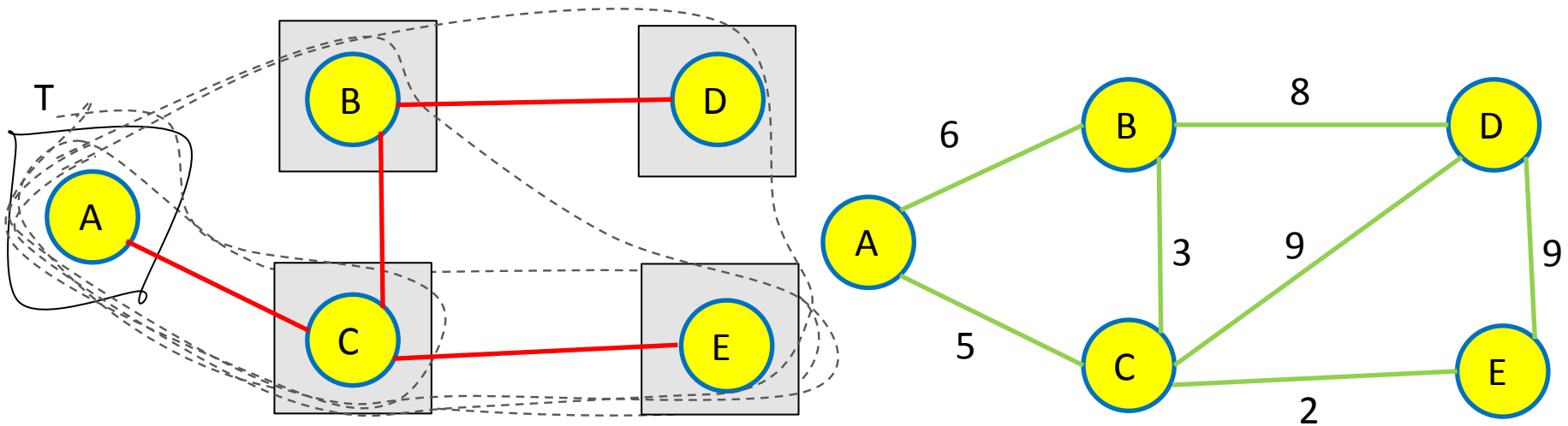
- Let T denote the MST we are constructing, initialised to be empty.
- An edge e is said to be safe if $\{T \cup e\}$ is a subset of **some** MST

General Strategy:

- $T = \text{null}$
- **while** T can be grown safely:
 - find an edge $e = \langle x, y \rangle$ along which T **is** safe to grow
 - $T = \{T\} \text{ union } \{\langle x, y \rangle\}$
- **return** T
- We will study two **greedy** algorithms that follow this strategy.

Prim's Algorithm: Overview

- Start by picking any vertex v to be the source of T .
- While T does not contain **all** vertices in the graph:
 - Find **shortest edge** e that goes from T to a different connected component.
 - Add e to the tree T .



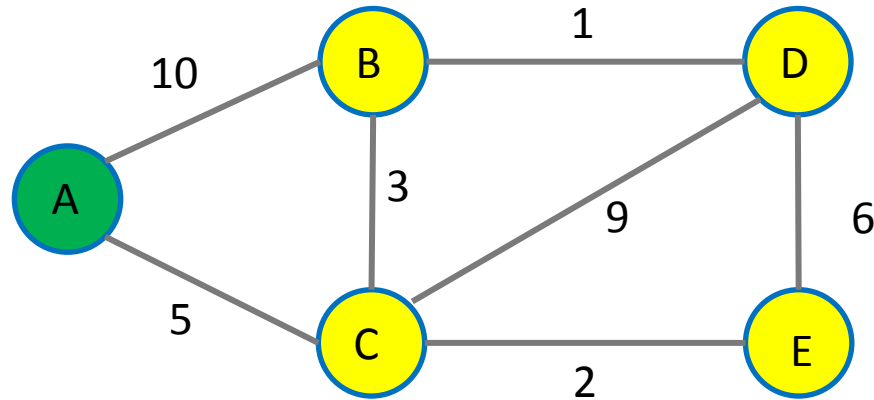
Connected components

MST Algorithms: Prim's Algorithm

Prim's Algorithm (small modification of Dijkstra's Algorithm)

- We start with no edges in T , so there are V connected components in T (each with a single node) and select one node as the source.
- At each iteration, we add to T the shortest edge that links the connected component containing the source to a different connected component.
- In each iteration, the number of connected components decreases by 1 and the number of nodes in the connected component containing the source increases by 1.
- Cycles are avoided as no edges linking two nodes that are already in the same connected component are ever added to T .
- Time complexity: $\Theta(E \log V)$

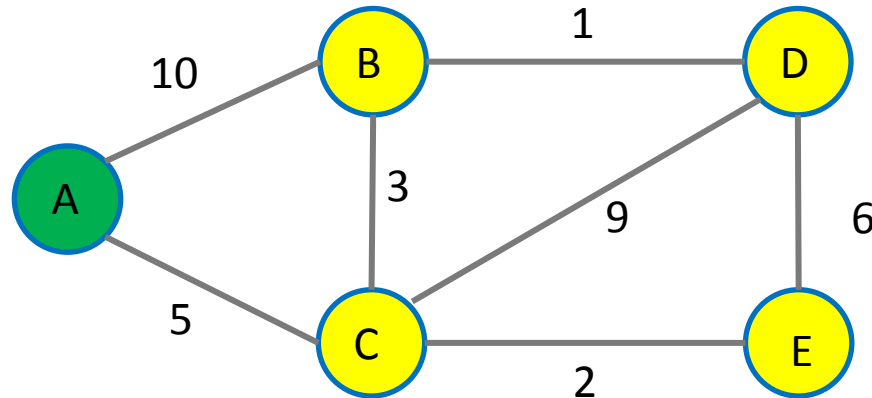
Prim's Algorithm



Robert C. Prim

Prim's Algorithm

u:



Q:

A	B	C	D	E
0	Inf	Inf	Inf	Inf

Prim's deals with distance estimates to reach T instead of distance estimates to the source

parent:

A	B	C	D	E
-	-	-	-	-

Q is a priority queue, where priority is based on distance

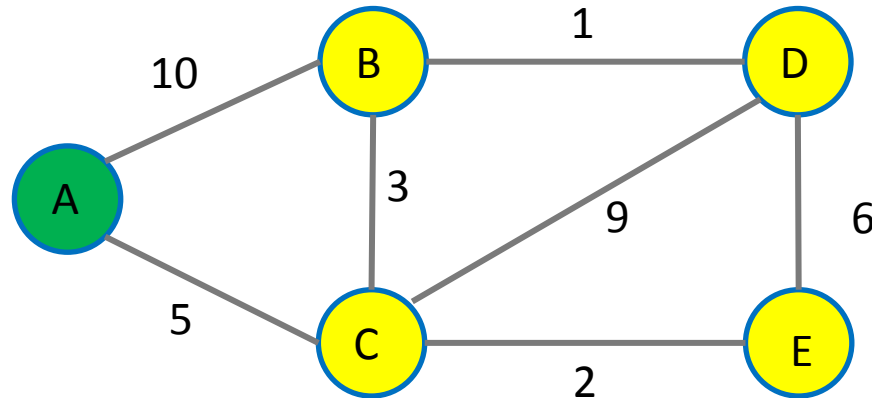
Dist:

A	B	C	D	E
0	Inf	Inf	Inf	inf

parent and Dist are the usual ID-indexed arrays

Prim's Algorithm

u: A



Q:

B	C	D	E
Inf	Inf	Inf	Inf

Prim's deals with distance estimates to reach T instead of distance estimates to the source

parent:

A	B	C	D	E
-	-	-	-	-

Q is a priority queue, where priority is based on distance

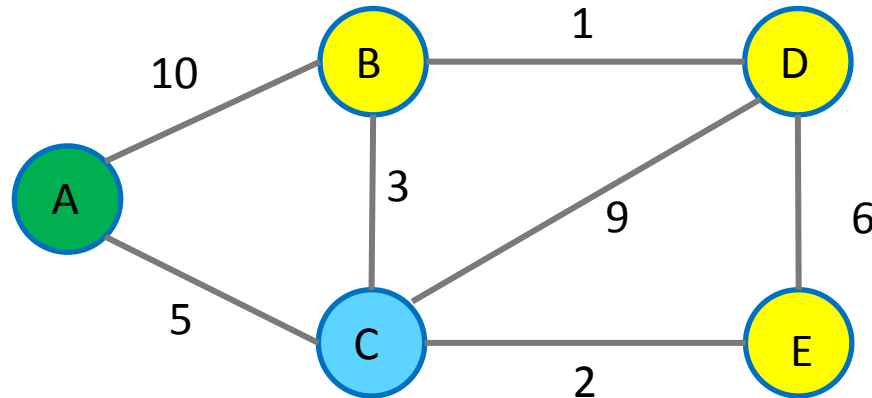
Dist:

A	B	C	D	E
0	Inf	Inf	Inf	inf

parent and Dist are the usual ID-indexed arrays

Prim's Algorithm

u: A



Q:

B	C	D	E
Inf	Inf	Inf	Inf

parent:

A	B	C	D	E
-	-	-	-	-

Dist:

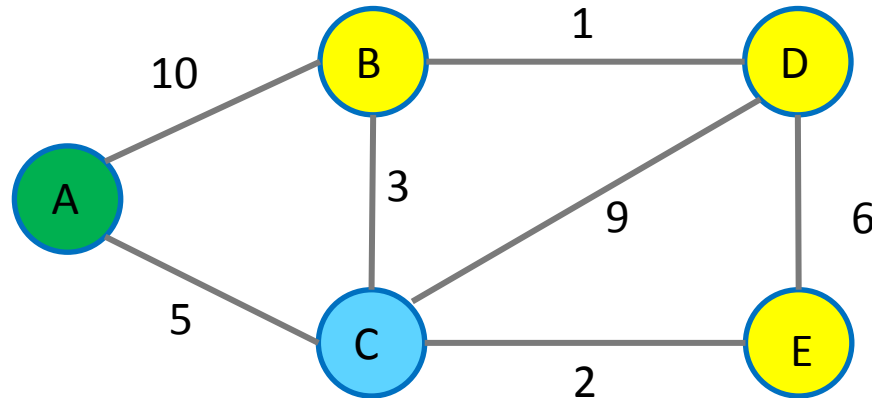
A	B	C	D	E
0	Inf	Inf	Inf	inf

For each neighbour v of u , try to update distance estimate

$5 < \text{inf}$
 $\text{Dist}[C] = 5$
 $\text{parent}[C] = A$

Prim's Algorithm

u: A



Q:

B	C	D	E
Inf	Inf	Inf	Inf

For each neighbour v of u , try to update distance estimate

parent:

A	B	C	D	E
-	-	-	-	-

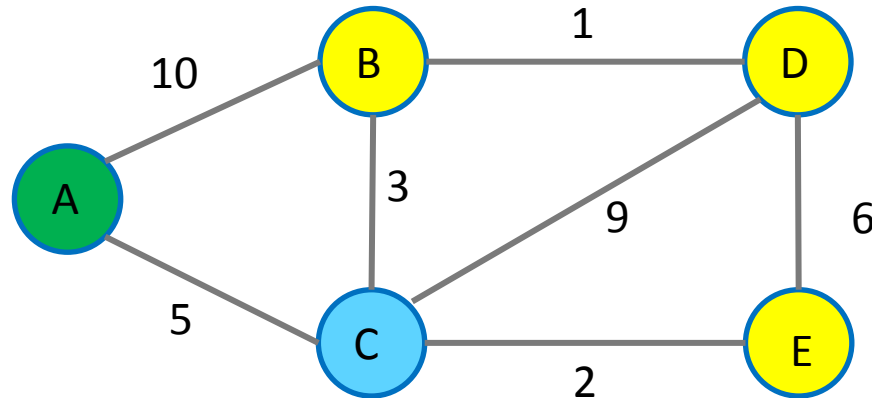
This time we just care about the edge, not the distance to u + the edge

Dist:

A	B	C	D	E
0	Inf	Inf	Inf	inf

Prim's Algorithm

u: A



Q:

B	C	D	E
Inf	Inf	Inf	Inf

parent:

A	B	C	D	E
-	-	-	-	-

Dist:

A	B	C	D	E
0	Inf	Inf	Inf	inf

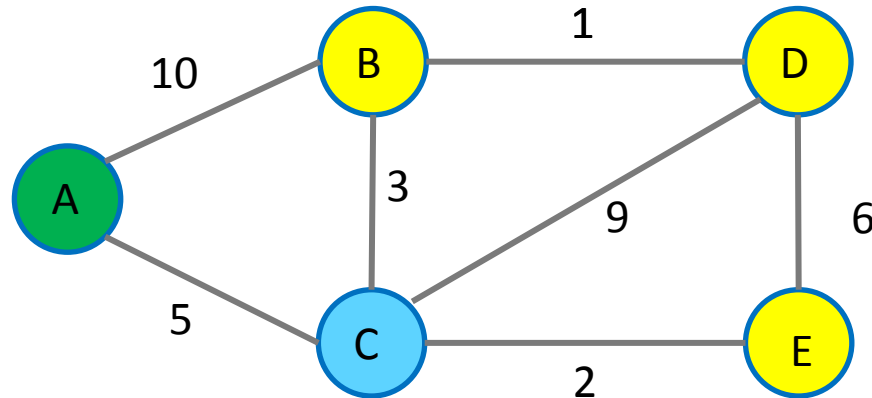
For each neighbour v of u , try to update distance

$W(A,C) = 5$
 $\text{Dist}[C] = \text{inf}$

$5 < \text{inf}$, so update
 $\text{Dist}[C] = 5$
 $\text{parent}[C] = A$

Prim's Algorithm

u: A



Q:

C	B	D	E
5	Inf	Inf	Inf

Note that this changes the order of Q

parent:

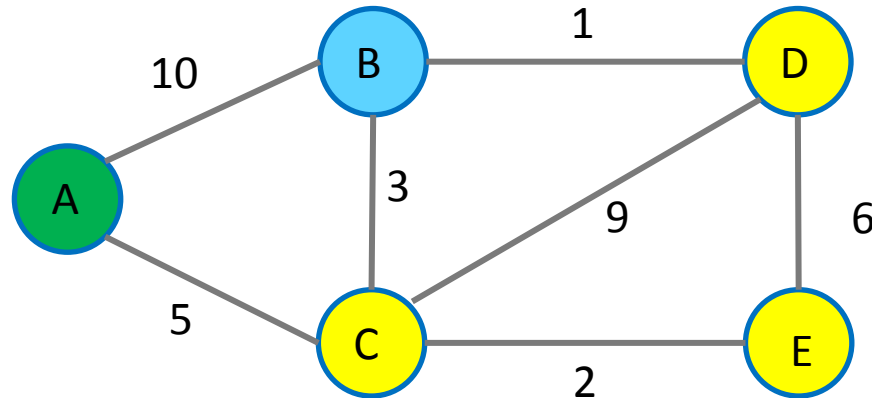
A	B	C	D	E
-	-	A	-	-

Dist:

A	B	C	D	E
0	Inf	5	Inf	inf

Prim's Algorithm

u: A



Q:

C	B	D	E
5	10	Inf	Inf

Doing the same for B
 $W(A,B) = 10$
 $\text{Dist}[B] = \text{inf}$

parent:

A	B	C	D	E
-	-	A	-	-

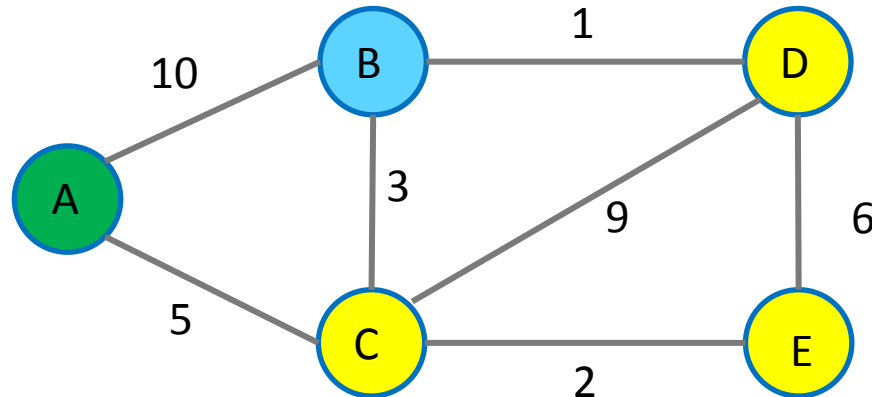
$10 < \text{inf}$

Dist:

A	B	C	D	E
0	Inf	5	Inf	inf

Prim's Algorithm

u: A



Q:

C	B	D	E
5	10	Inf	Inf

Doing the same for B
 $W(A,B) = 10$
 $\text{Dist}[B] = \text{inf}$

parent:

A	B	C	D	E
-	A	A	-	-

$10 < \text{inf}$, so update

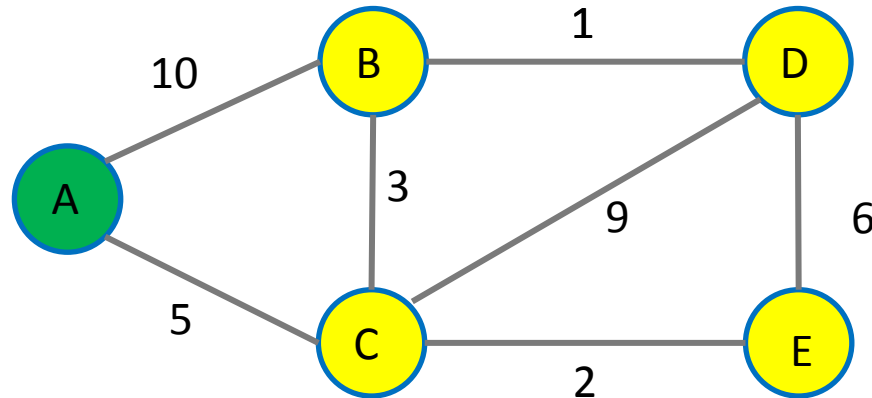
- $\text{Dist}[B] = 10$
- $\text{parent}[B] = A$

Dist:

A	B	C	D	E
0	10	5	Inf	inf

Prim's Algorithm

u: A



Q:

C	B	D	E
5	10	Inf	Inf

parent:

A	B	C	D	E
-	A	A	-	-

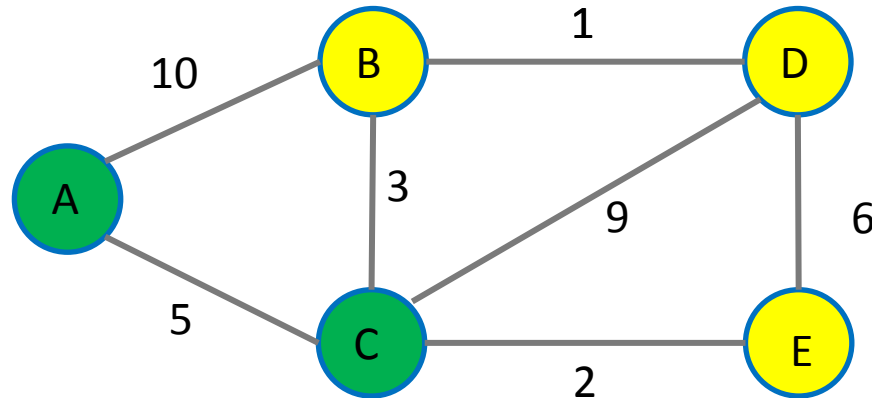
Dist:

A	B	C	D	E
0	10	5	Inf	inf

- Finished with A, so pop from Q
- Notice that this will always be the vertex with the smallest distance estimate on Q

Prim's Algorithm

u: C



Q:

B	D	E
10	Inf	Inf

parent:

A	B	C	D	E
-	A	A	-	-

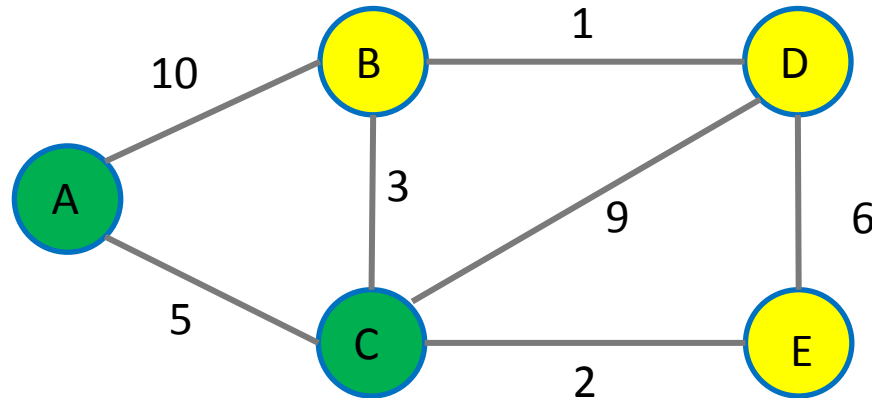
Dist:

A	B	C	D	E
0	10	5	Inf	inf

- Finished with A, so pop from Q
- Notice that this will always be the vertex with the smallest distance estimate on Q
- This vertex is now in the MST

Prim's Algorithm

u: C



Q:

B	D	E
10	Inf	Inf

parent:

A	B	C	D	E
-	A	A	-	-

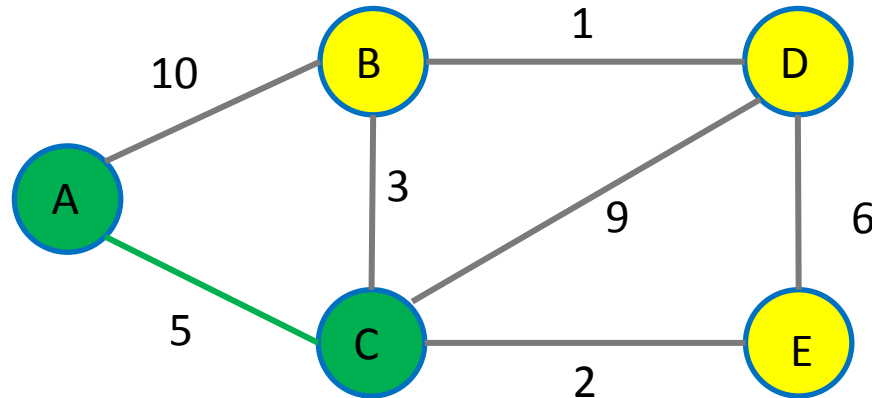
Dist:

A	B	C	D	E
0	10	5	Inf	inf

- The edge we add is the edge between u and parent[u]
- So in this case, A->C

Prim's Algorithm

u: C



Q:

B	D	E
10	Inf	Inf

parent:

A	B	C	D	E
-	A	A	-	-

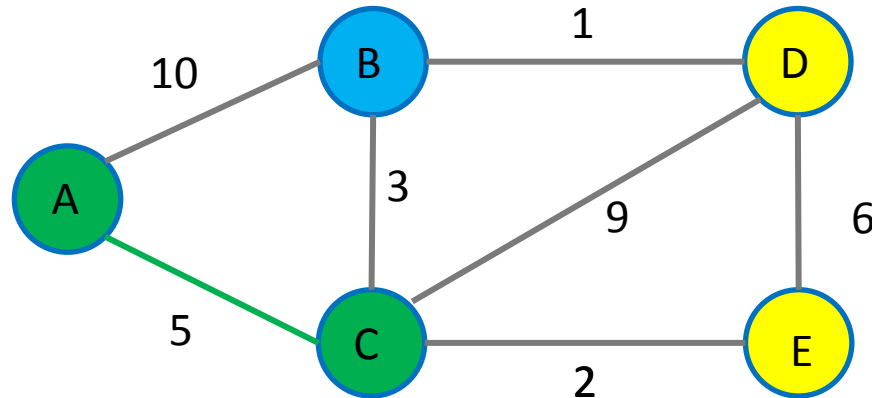
Dist:

A	B	C	D	E
0	10	5	Inf	inf

- The edge we add is the edge between u and parent[u]
- So in this case, A->C

Prim's Algorithm

u: **C**



Q:

B	D	E
10	Inf	Inf

- Update B from C
- $w(C, B) = 3$
- $\text{Dist}[B] = 10$

parent:

A	B	C	D	E
-	A	A	-	-

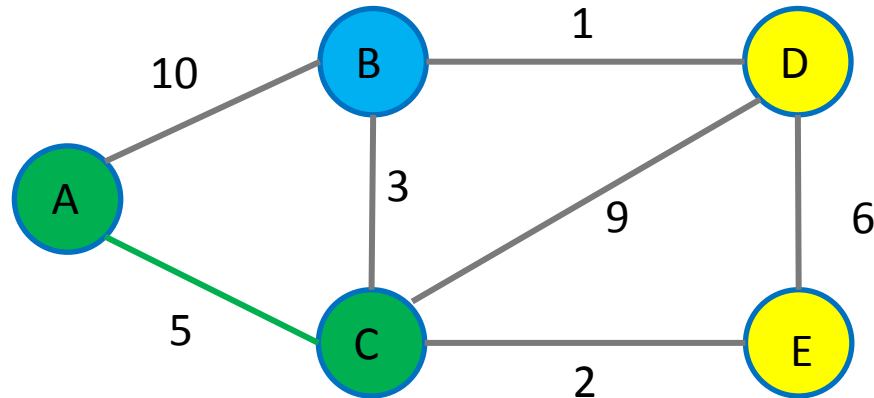
- $3 < 10$ so update $\text{dist}[B]$ and $\text{parent}[B]$

Dist:

A	B	C	D	E
0	10	5	Inf	inf

Prim's Algorithm

u: C



Q:

B	D	E
3	Inf	Inf

parent:

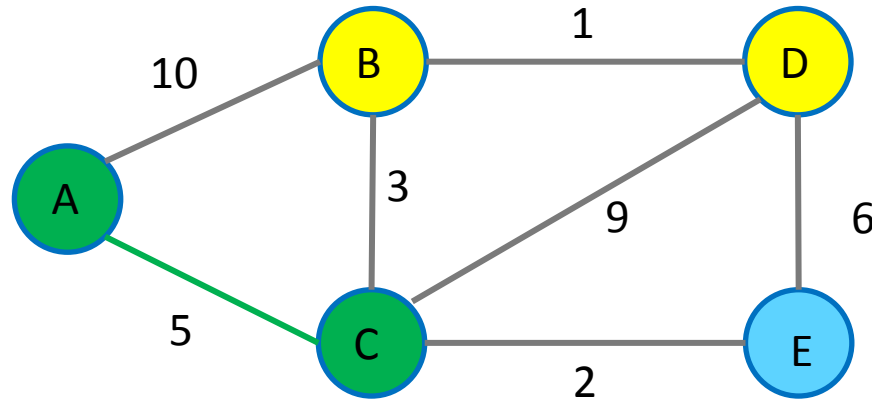
A	B	C	D	E
-	C	A	-	-

Dist:

A	B	C	D	E
0	3	5	Inf	inf

Prim's Algorithm

u: **C**



Q:

B	D	E
3	Inf	Inf

- Update E from C

parent:

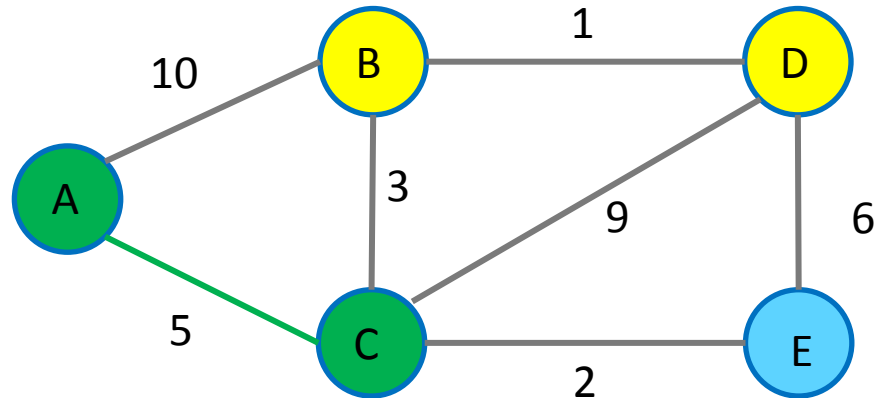
A	B	C	D	E
-	C	A	-	-

Dist:

A	B	C	D	E
0	3	5	Inf	inf

Prim's Algorithm

u: C



Q:

E	B	D
2	3	inf

parent:

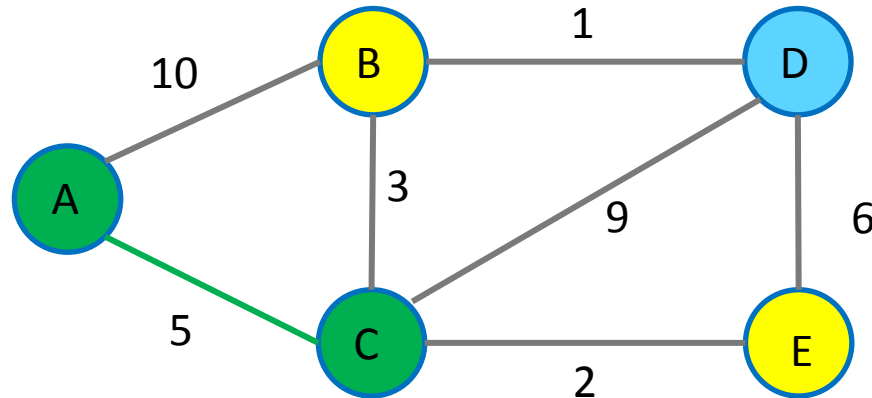
A	B	C	D	E
-	C	A	-	C

Dist:

A	B	C	D	E
0	3	5	Inf	2

Prim's Algorithm

u: **C**



Q:

E	B	D
2	3	inf

- Update D from C

parent:

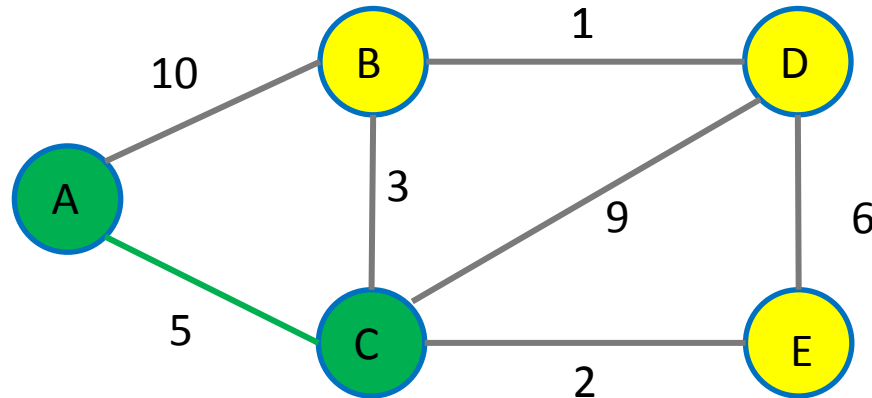
A	B	C	D	E
-	C	A	-	C

Dist:

A	B	C	D	E
0	3	5	Inf	2

Prim's Algorithm

u: **C**



Q:

E	B	D
2	3	9

parent:

A	B	C	D	E
-	C	A	C	C

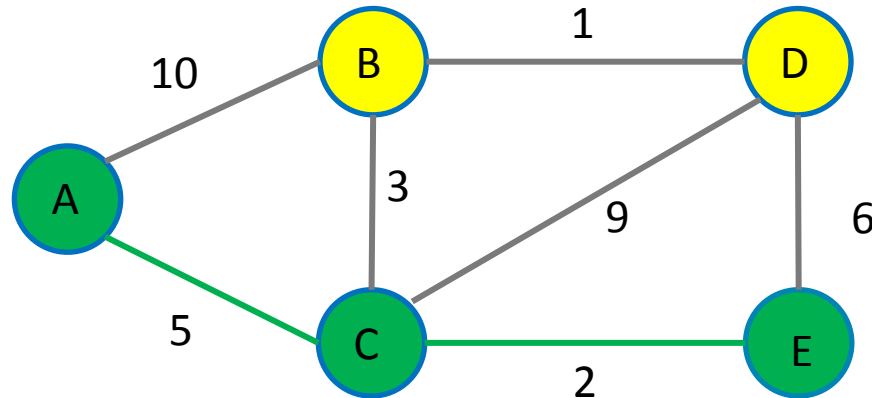
Dist:

A	B	C	D	E
0	3	5	9	2

- Done with C
- Pop another vertex from Q and add it to the MST

Prim's Algorithm

u: **E**



Q:

B	D
3	9

parent:

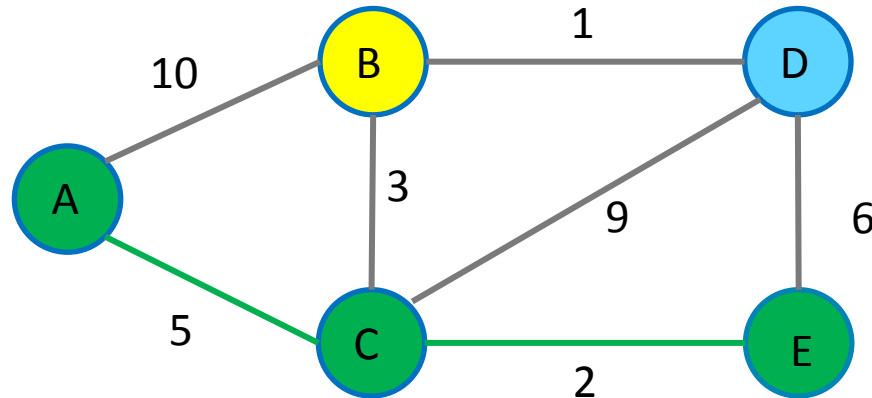
A	B	C	D	E
-	C	A	C	C

Dist:

A	B	C	D	E
0	3	5	9	2

Prim's Algorithm

u: **E**



Q:

B	D
3	6

parent:

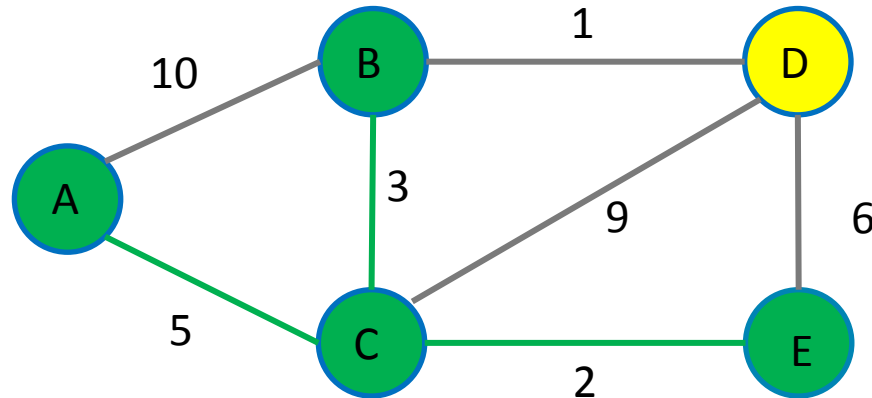
A	B	C	D	E
-	C	A	E	C

Dist:

A	B	C	D	E
0	3	5	6	2

Prim's Algorithm

u: **B**



Q: **D**
6

parent:

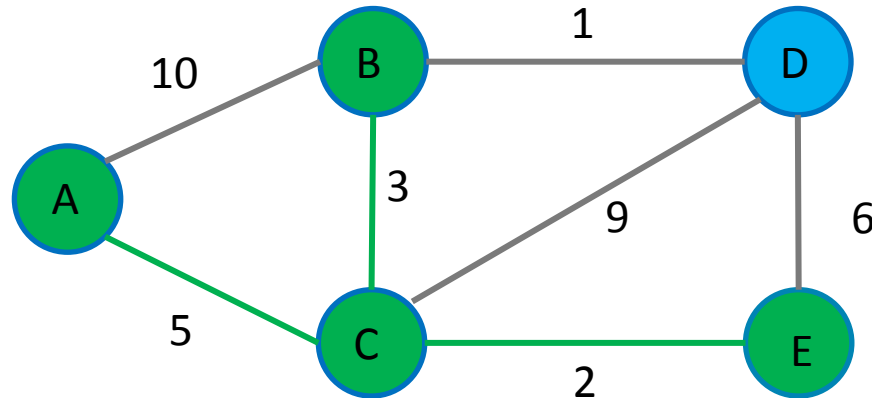
A	B	C	D	E
-	C	A	E	C

Dist:

A	B	C	D	E
0	3	5	6	2

Prim's Algorithm

u: **B**



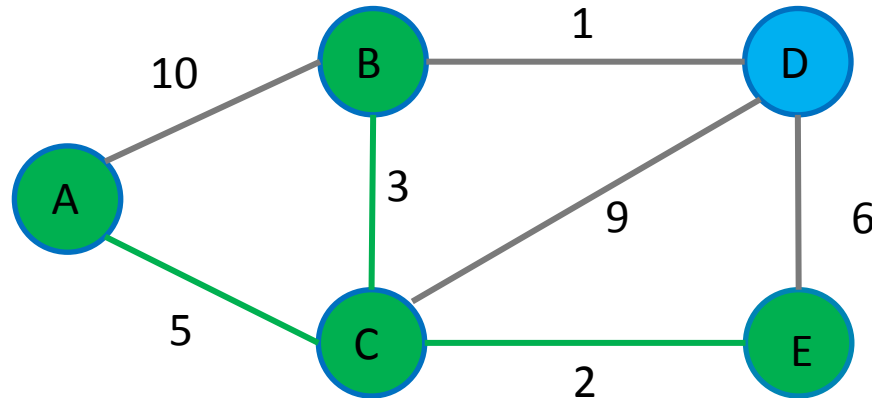
Q: **D**
6

parent	A	B	C	D	E
	-	C	A	E	C

Dist:	A	B	C	D	E
	0	3	5	6	2

Prim's Algorithm

u: **B**



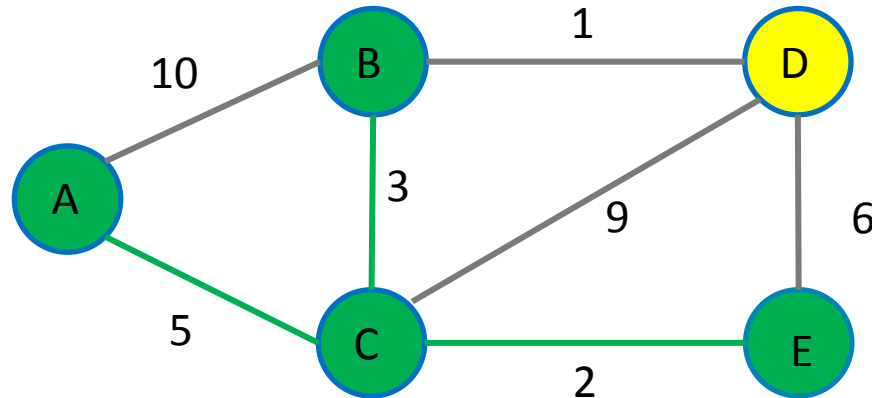
Q: **D**
1

parent	A	B	C	D	E
	-	C	A	B	C

Dist:	A	B	C	D	E
	0	3	5	1	2

Prim's Algorithm

u: **B**



Q: **D**
1

parent:

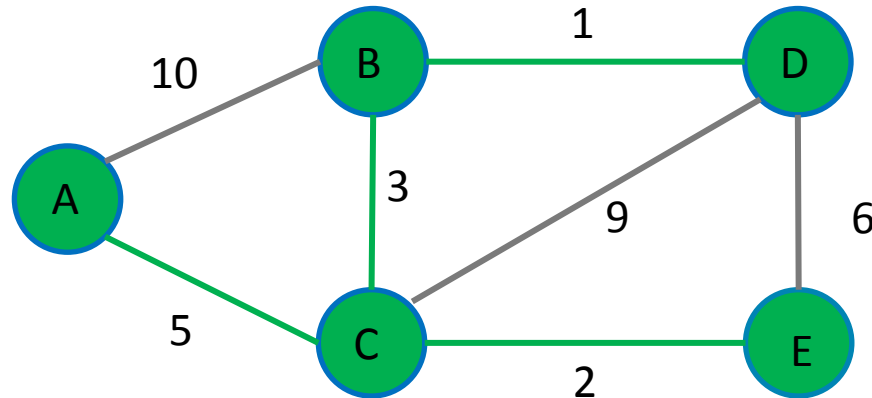
A	B	C	D	E
-	C	A	B	C

Dist:

A	B	C	D	E
0	3	5	1	2

Prim's Algorithm

u: **D**



Q:

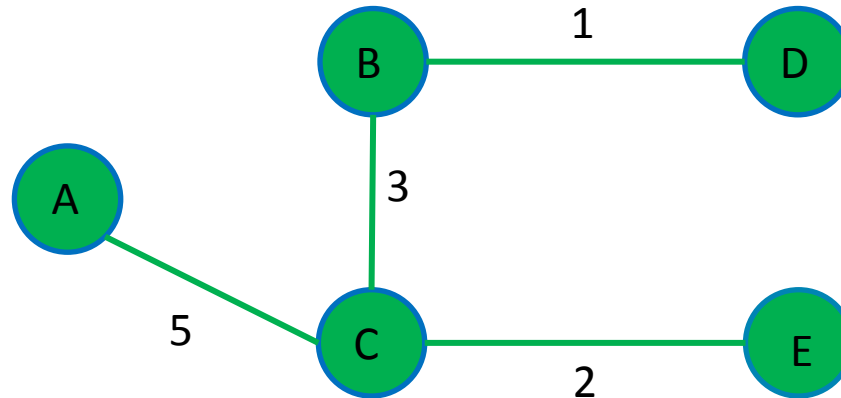
parent	A	B	C	D	E
	-	C	A	B	C

Dist:

A	B	C	D	E
0	3	5	1	2

Prim's Algorithm

u: D



Q:

parent:	A	B	C	D	E
	-	C	A	B	C

Dist:

A	B	C	D	E
0	3	5	1	2

Prim's Algorithm

Algorithm 69 Prim's algorithm

```
1: function PRIM( $G = (V, E)$ ,  $r$ )
2:    $dist[1..n] = \infty$ 
3:    $parent[1..n] = \mathbf{null}$ 
4:    $T = (\{r\}, \emptyset)$ 
5:    $dist[r] = 0$ 
6:    $Q = \text{priority\_queue}(V[1..n], \text{key}(v) = dist[v])$ 
7:   while  $Q$  is not empty do
8:      $u = Q.\text{pop\_min}()$ 
9:      $T.\text{add\_vertex}(u)$ 
10:     $T.\text{add\_edge}(parent[u], u)$ 
11:    for each edge  $e = (u, v)$  adjacent to  $u$  do
12:      if not  $v \in T$  and  $dist[v] > w(u, v)$  then
13:        // Remember to update the key of  $v$  in the priority queue!
14:         $dist[v] = w(u, v)$ 
15:         $parent[v] = u$ 
16:  return  $T$ 
```

Prim's Algorithm: Complexity

It is very similar to Dijkstra's Algorithm and its complexity is the same as Dijkstra's Algorithm

- $\theta(V \log V + E \log V)$ if min-heap is used

Since the input graph G is connected, $E \geq V-1$. Hence, the complexity can be simplified to $\theta(E \log V)$.

Prim's Algorithm: Correctness

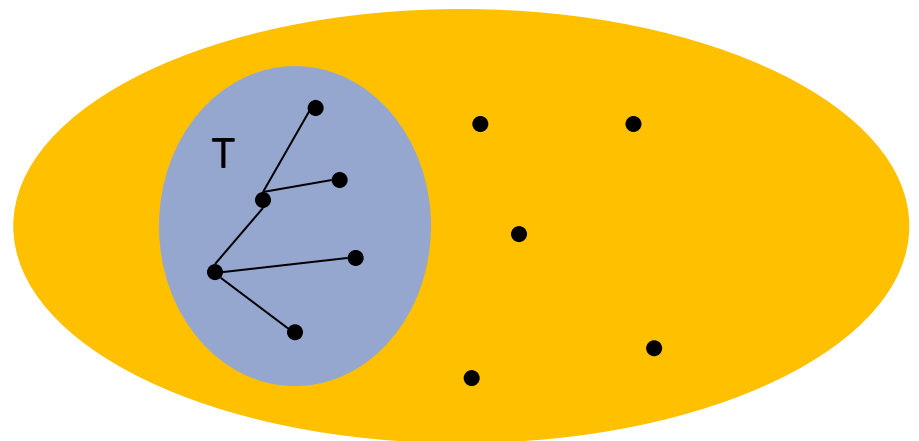
#INV: Every iteration of Prim's algorithm, the current set of selected edges in T is a subset of **some** minimum spanning tree of G

Base Case:

- The invariant is true initially when T is empty

Inductive step:

- **We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration**
- Assume T is a subset of some MST M



Prim's Algorithm: Correctness

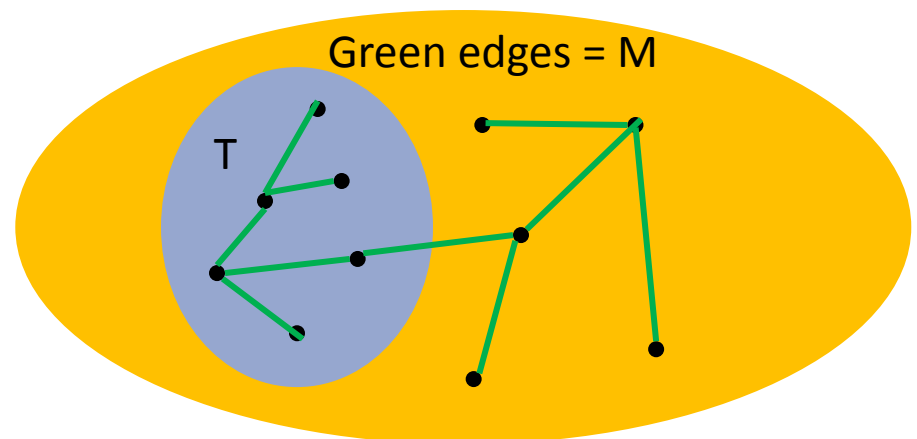
#INV: Every iteration of Prim's algorithm, the current set of selected edges in T is a subset of **some** minimum spanning tree of G

Base Case:

- The invariant is true initially when T is empty

Inductive step:

- **We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration**
- Assume T is a subset of some MST M



Prim's Algorithm: Correctness

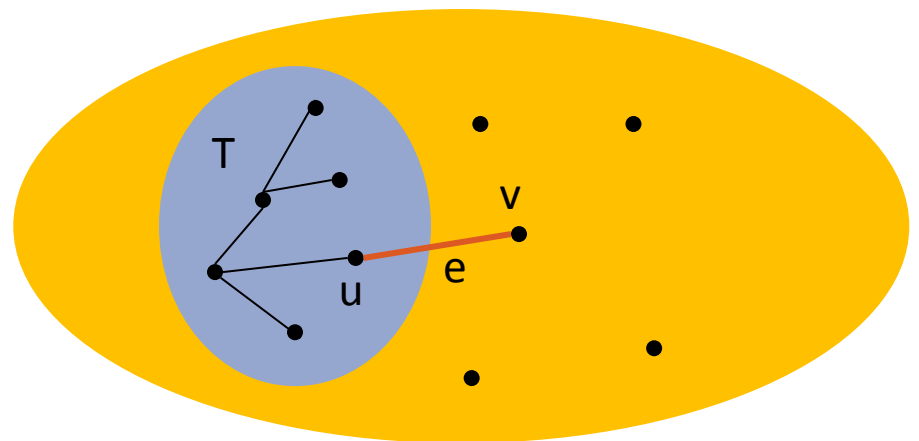
#INV: Every iteration of Prim's algorithm, the current set of selected edges in T is a subset of **some** minimum spanning tree of G

Base Case:

- The invariant is true initially when T is empty

Inductive step:

- **We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration**
- Assume T is a subset of some MST M
- Let $e = (u, v)$ be the lightest edge that connects some v in T to some u not in T (i.e. this is the edge Prim's algorithm will choose in this iteration)
- If e is in M , then $T \cup \{e\}$ is a subset of M , which is an MST, so the invariant holds



Prim's Algorithm: Correctness

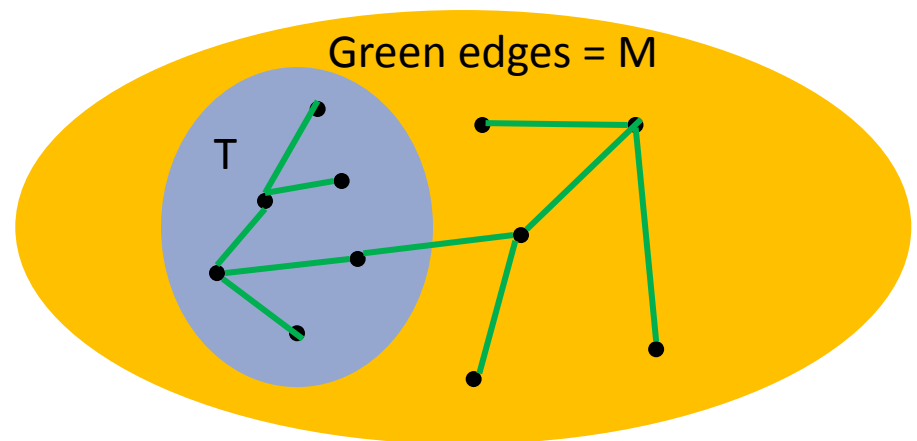
#INV: Every iteration of Prim's algorithm, the current set of selected edges in T is a subset of **some** minimum spanning tree of G

Base Case:

- The invariant is true initially when T is empty

Inductive step:

- **We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration**
- Assume T is a subset of some MST M
- Let $e = (u,v)$ be the lightest edge that connects some v in T to some u not in T (i.e. this is the edge Prim's algorithm will choose in this iteration)
- If e is in M , then $T \cup \{e\}$ is a subset of M , which is an MST, so the invariant holds



Prim's Algorithm: Correctness

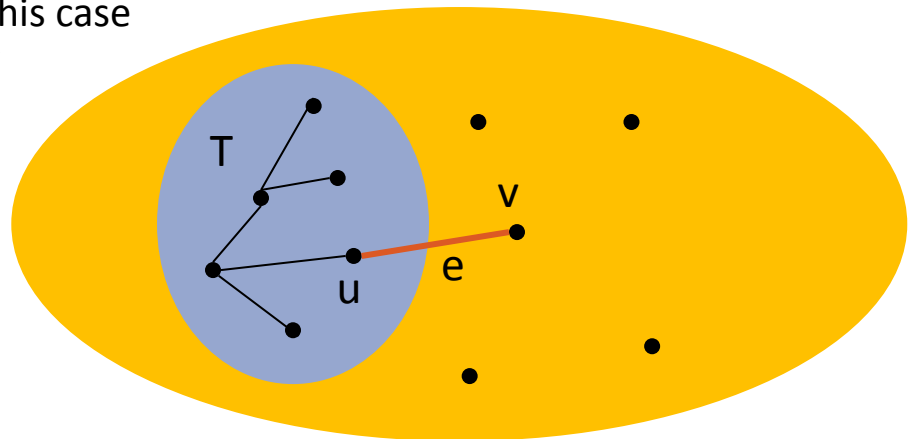
#INV: Every iteration of Prim's algorithm, the current set of selected edges in T is a subset of **some** minimum spanning tree of G

Base Case:

- The invariant is true initially when T is empty

Inductive step:

- **We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration**
- Assume T is a subset of some MST M
- Let $e = (u, v)$ be the lightest edge that connects some v in T to some u not in T (i.e. this is the edge Prim's algorithm will choose in this iteration)
- If e is in M , then $T \cup \{e\}$ is a subset of M , which is an MST, so the invariant holds
- The interesting case is where e is not in M . In this case we have to show that there is some other MST which contains $T \cup \{e\}$



Prim's Algorithm: Correctness

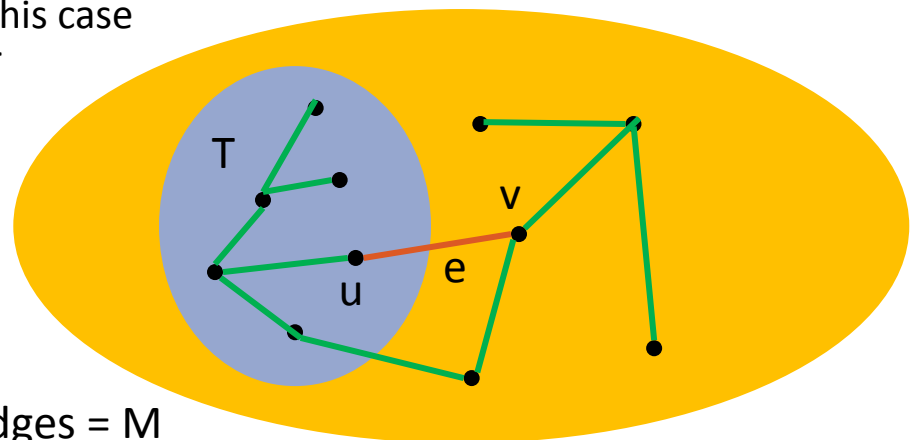
#INV: Every iteration of Prim's algorithm, the current set of selected edges in T is a subset of **some** minimum spanning tree of G

Base Case:

- The invariant is true initially when T is empty

Inductive step:

- **We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration**
- Assume T is a subset of some MST M
- Let $e = (u, v)$ be the lightest edge that connects some v in T to some u not in T (i.e. this is the edge Prim's algorithm will choose in this iteration)
- If e is in M , then $T \cup \{e\}$ is a subset of M , which is an MST, so the invariant holds
- The interesting case is where e is not in M . In this case we have to show that there is some other MST which contains $T \cup \{e\}$

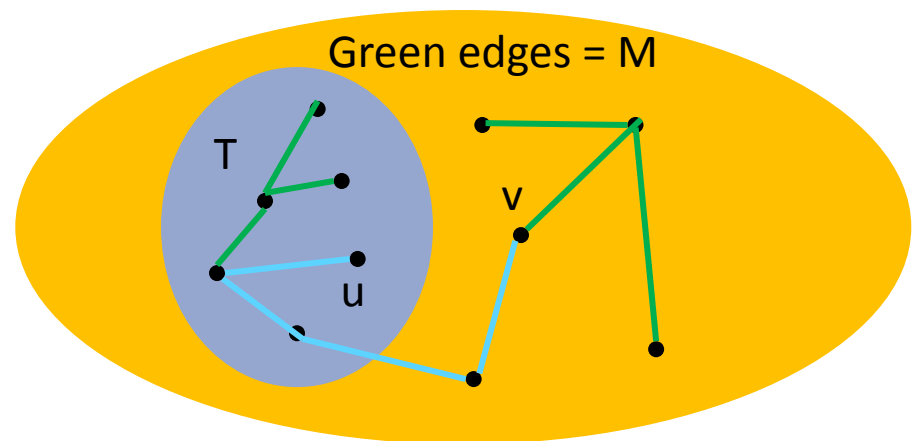


Green edges = M

Prim's Algorithm: Correctness

Inductive step:

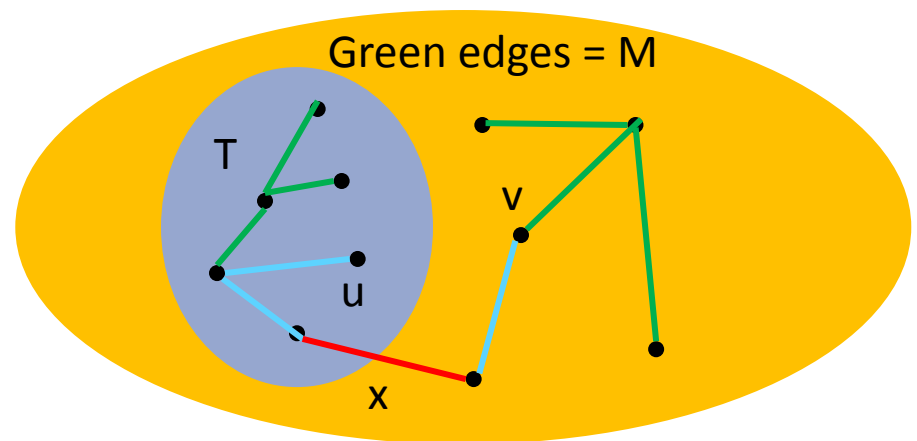
- We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration
- Since M is a tree, there is exactly one path from u to v in M (shown in blue)
- u and v are not connected in T (since v is not in T). Consider the first edge on the blue path which is **not** contained in T (call this edge x).



Prim's Algorithm: Correctness

Inductive step:

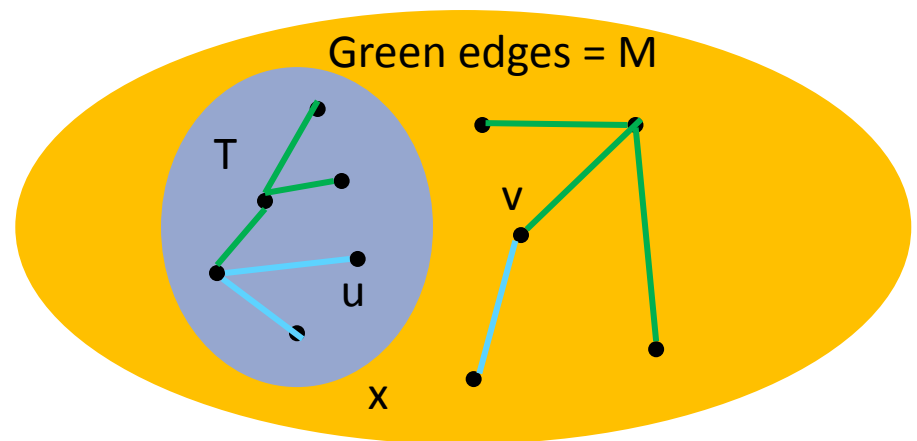
- We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration
- Since M is a tree, there is exactly one path from u to v in M (shown in blue)
- u and v are not connected in T (since v is not in T). Consider the first edge on the blue path which is **not** contained in T (call this edge x).
- One vertex of this edge is in T , the other is not.
- Removing this edge would disconnect M



Prim's Algorithm: Correctness

Inductive step:

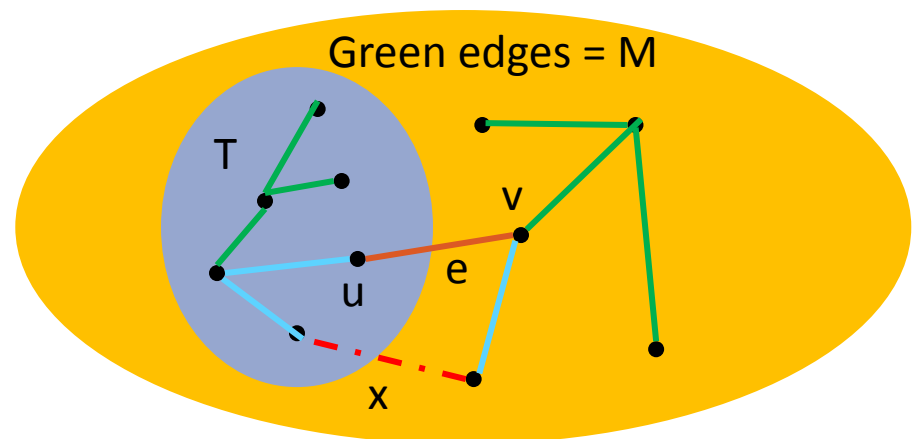
- We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration
- Since M is a tree, there is **exactly one path from u to v in M** (shown in blue)
- u and v are not connected in T (since v is not in T). Consider the first edge on the blue path which is **not** contained in T (call this edge x).
- One vertex of this edge is in T , the other is not.
- Removing this edge would disconnect M



Prim's Algorithm: Correctness

Inductive step:

- We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration
- Since M is a tree, there is exactly one path from u to v in M (shown in blue)
- u and v are not connected in T (since v is not in T). Consider the first edge on the blue path which is **not** contained in T (call this edge x).
- One vertex of this edge is in T , the other is not.
- Removing this edge would disconnect M
- Adding the edge $e=(u,v)$ would form a new spanning tree, M'
- Since Prim's algorithm **always selects the lightest edge incident to T** , we know that **$w(e) \leq w(x)$**
- So the weight of M' is no greater than the weight of M , therefore choosing e is correct



Outline

1. Dijkstra's Algorithm
2. Prim's Algorithm
3. **Kruskal's Algorithm**

MST Algorithms: Kruskal's Algorithm

Kruskal's Algorithm

- We start with no edges in T , so there are V connected components in T (each with a single node).
- We process edges in ascending order of edge weights, and add an edge e to T if this addition does not create a cycle.
- Each time that one edge e is added to T , e is the smallest edge that links two distinct connected components of T .
- Cycles are avoided as no edges linking two nodes that are already in the same connected component are ever added to T . At any point of the execution T is a forest (i.e., a set of trees)
- Time complexity: $\Theta(E \log V)$.

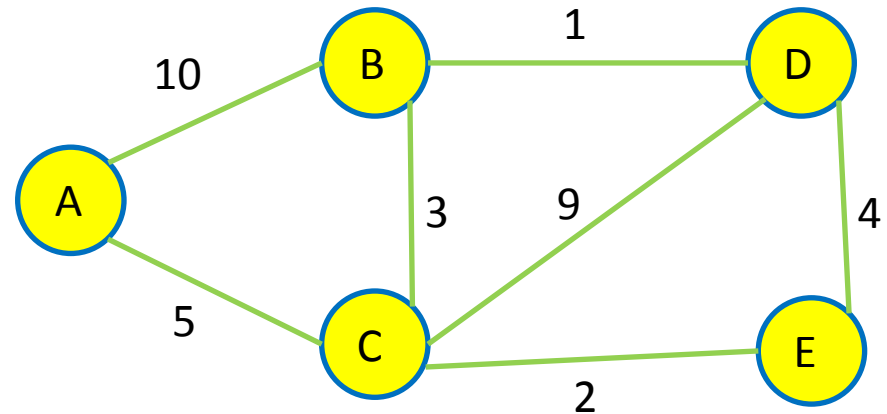
Kruskal's Algorithm

Kruskals($G(V, E)$)

- Sort the edges in ascending order of weights
- Let T be a graph with V as its vertices, and no edges
- For each edge (u, v) in ascending order
 - If adding (u, v) does not create a cycle in T
 - Add (u, v) to T
- Return T



Joseph Kruskal



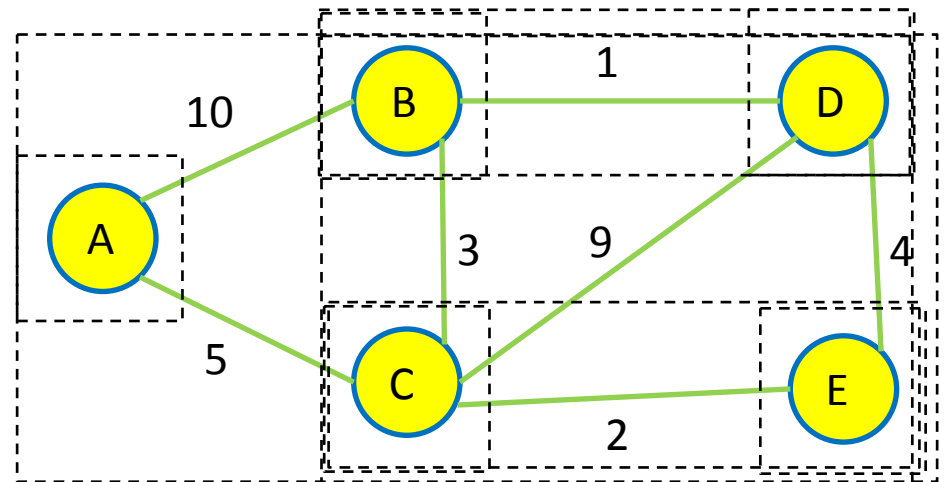
Kruskal's Algorithm

Kruskals($G(V, E)$)

- Sort the edges in ascending order of weights
- Let T be a graph with V as its vertices, and no edges
- For each edge (u, v) in ascending order
 - If adding (u, v) does not create a cycle in T
 - Add (u, v) to T
- Return T

Connected Components

How to determine if the edge will create a cycle???



Sorted Edges:

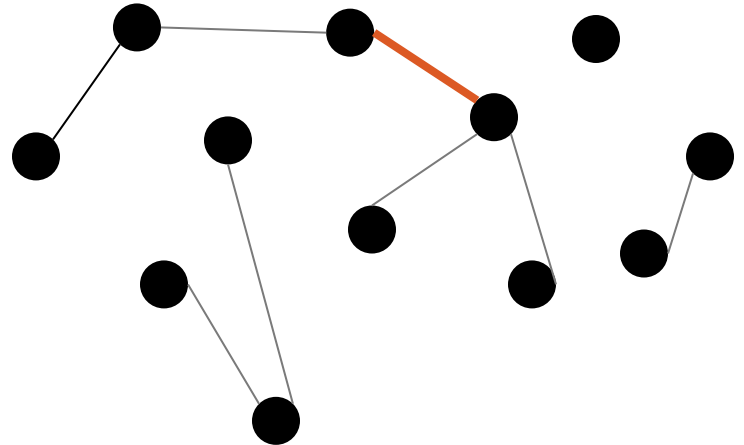
B↔D, 1	C↔E, 2	C↔B, 3	E↔D, 4	A↔C, 5	C↔D, 9	A↔B, 10
--------	--------	--------	--------	--------	--------	---------

Finalized (in MST):

B↔D	C↔E	C↔B	A↔C
-----	-----	-----	-----

Kruskal's Algorithm

At some point in the algorithm, we want to check if an edge can be added.
Can we add the red edge?



Kruskal's Algorithm

At some point in the algorithm, we want to check if an edge can be added.

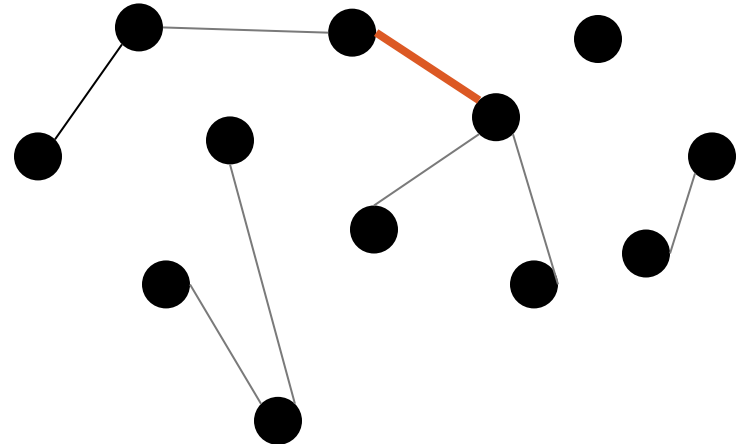
Can we add the red edge?

An edge can be added if it does not create a cycle.

OR

An edge can be added if it is not between two nodes which belong to the same connected component.

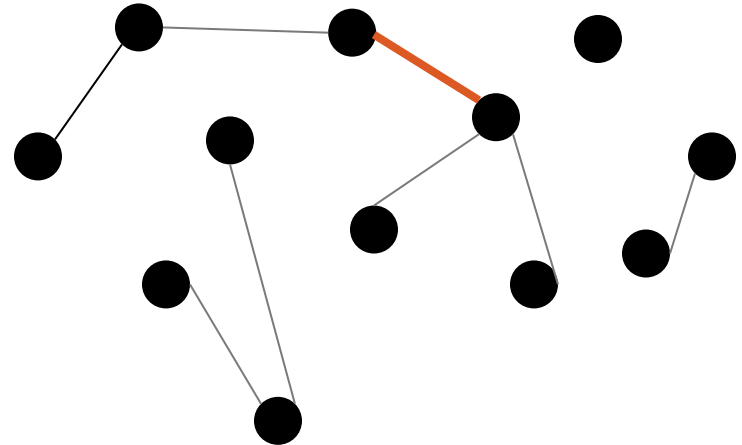
Importantly, adding an edge between two distinct connected components “merges” those connected components into one.



Kruskal's Algorithm

An edge can be added if it is not between two nodes which belong to the same connected component.

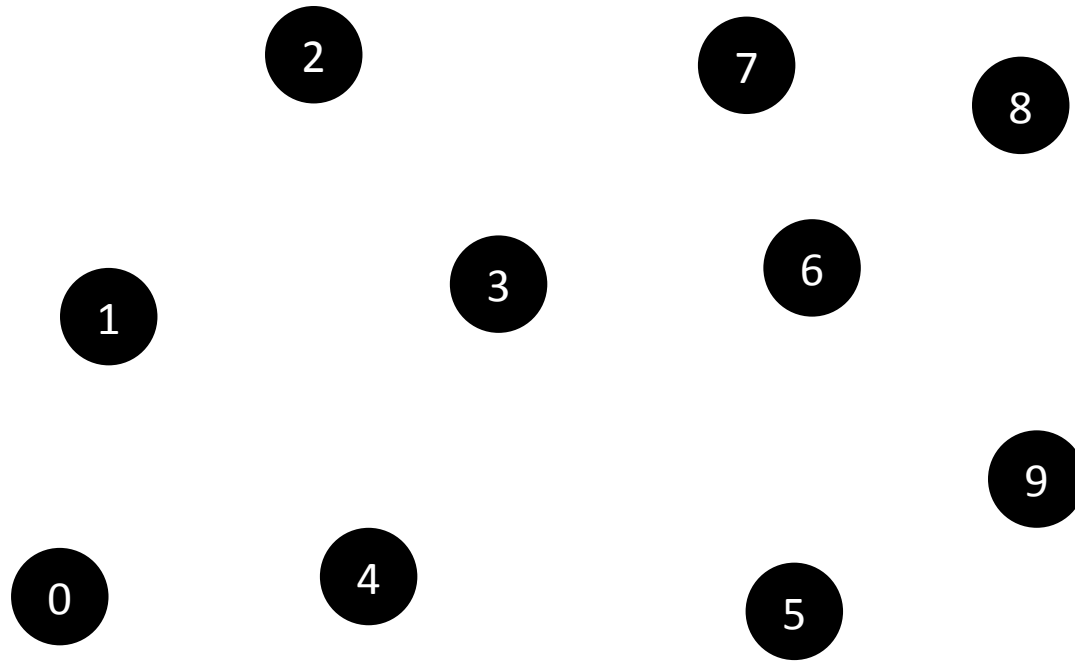
Importantly, adding an edge between two distinct connected components “merges” those connected components into one.



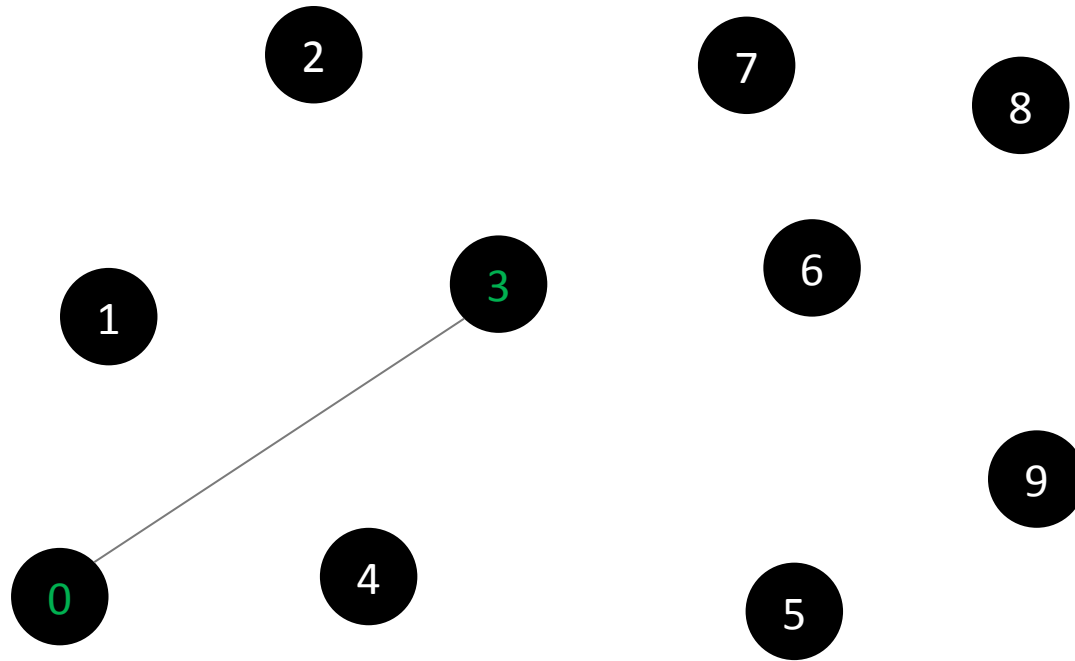
Remember that in Kruskal's algorithm, each time that one edge e is added to T , e is the smallest edge that links two distinct connected components of T .

Intuitively, Kruskal's algorithm runs for $V-1$ iterations, and in each iteration it merges the two distinct connected components that have the smallest distance between themselves.

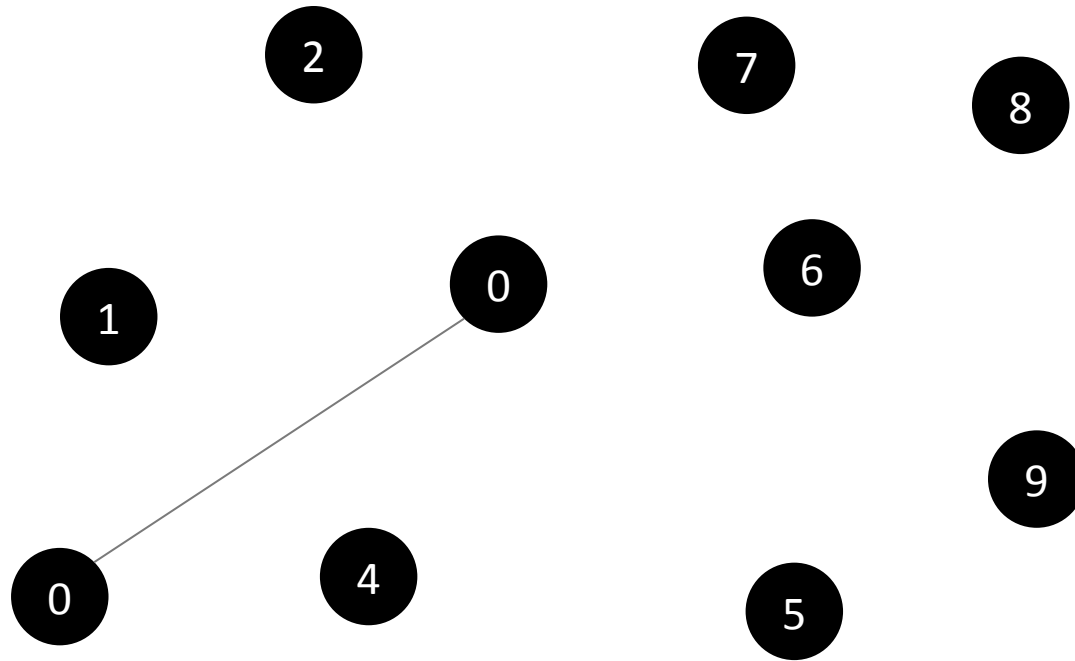
Kruskal's Algorithm



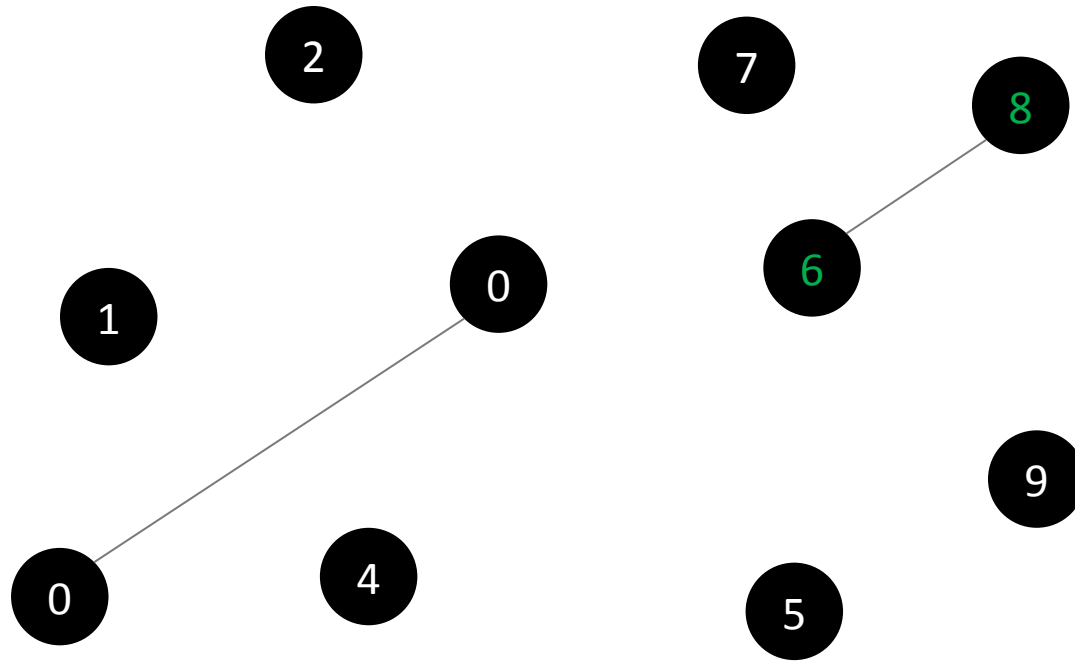
Kruskal's Algorithm



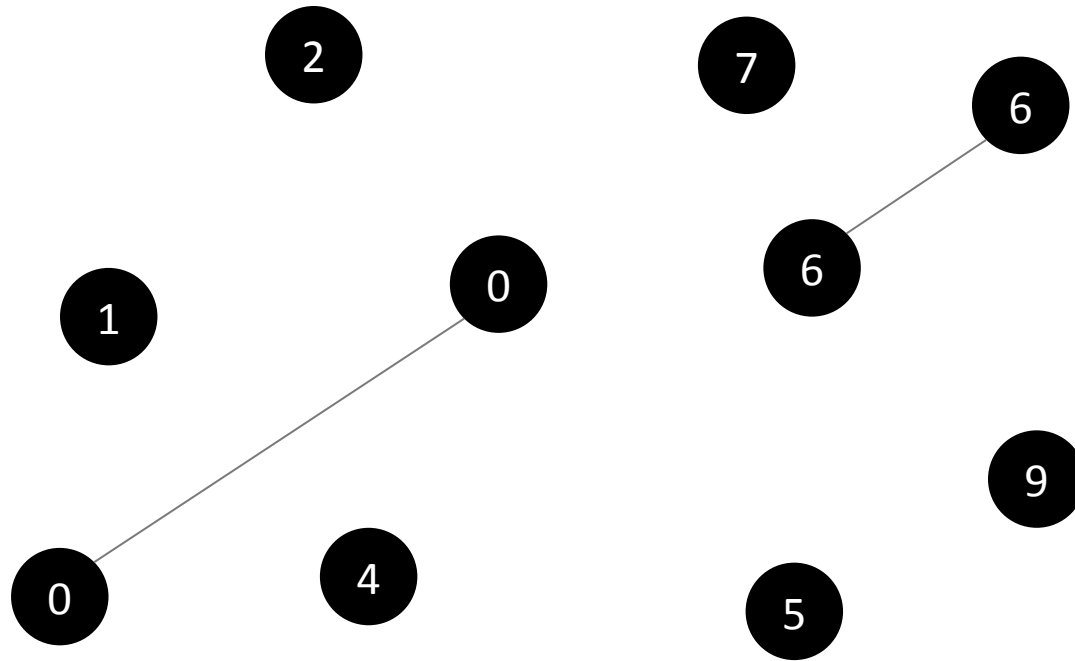
Kruskal's Algorithm



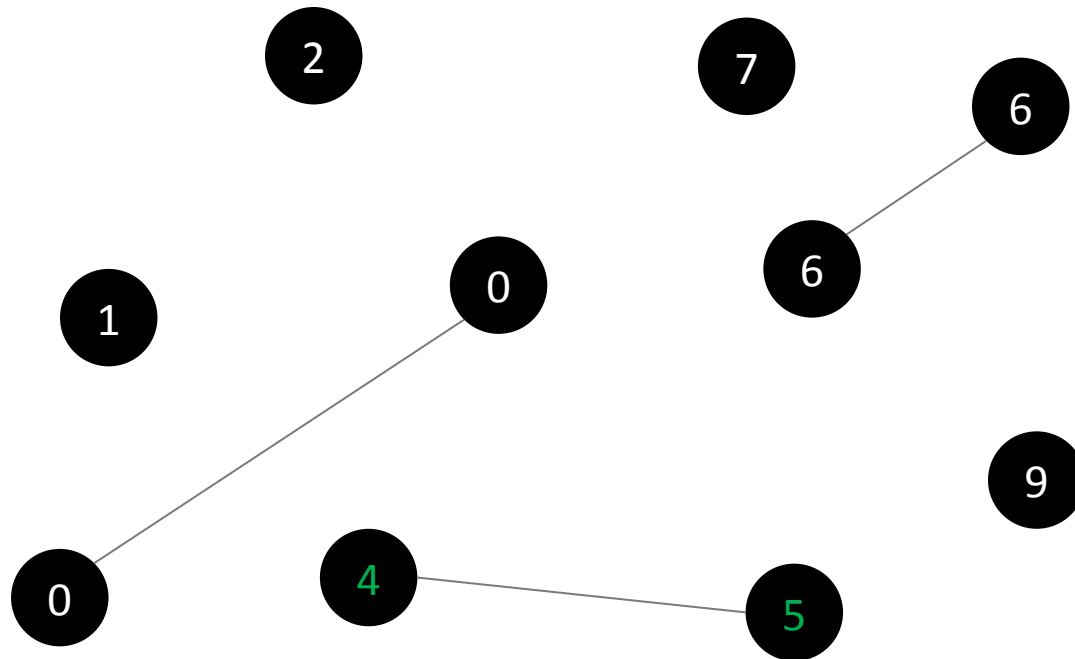
Kruskal's Algorithm



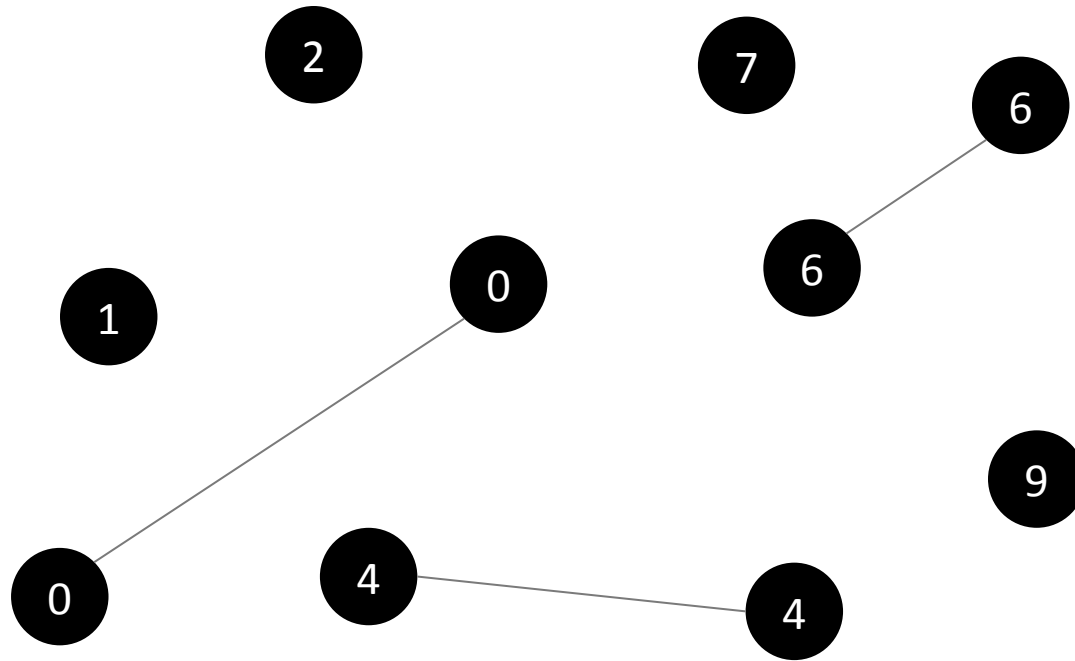
Kruskal's Algorithm



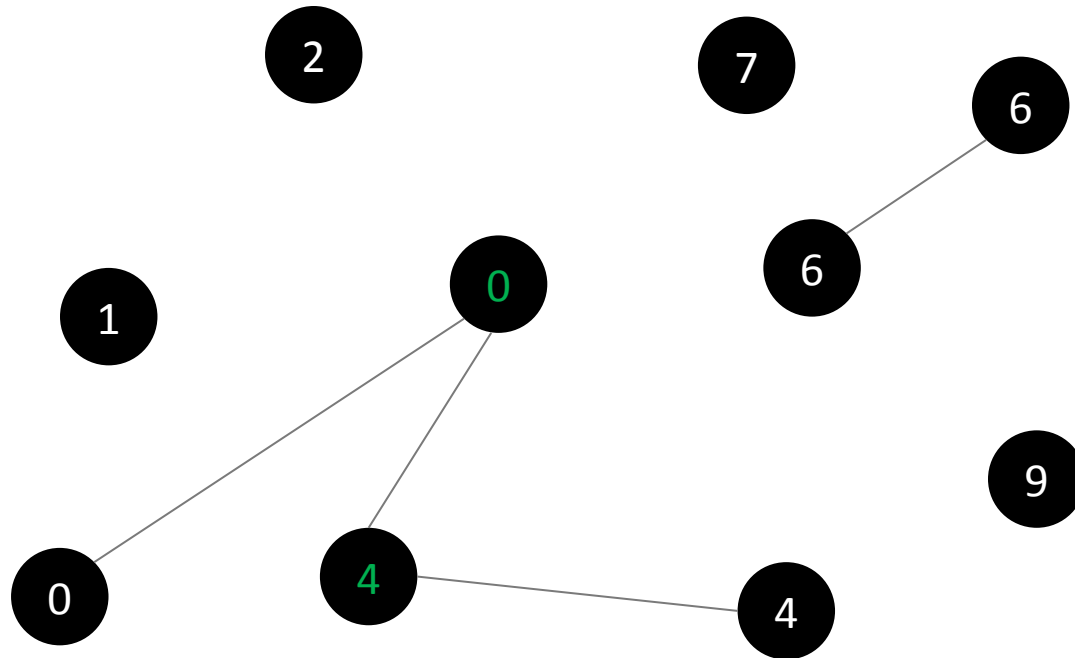
Kruskal's Algorithm



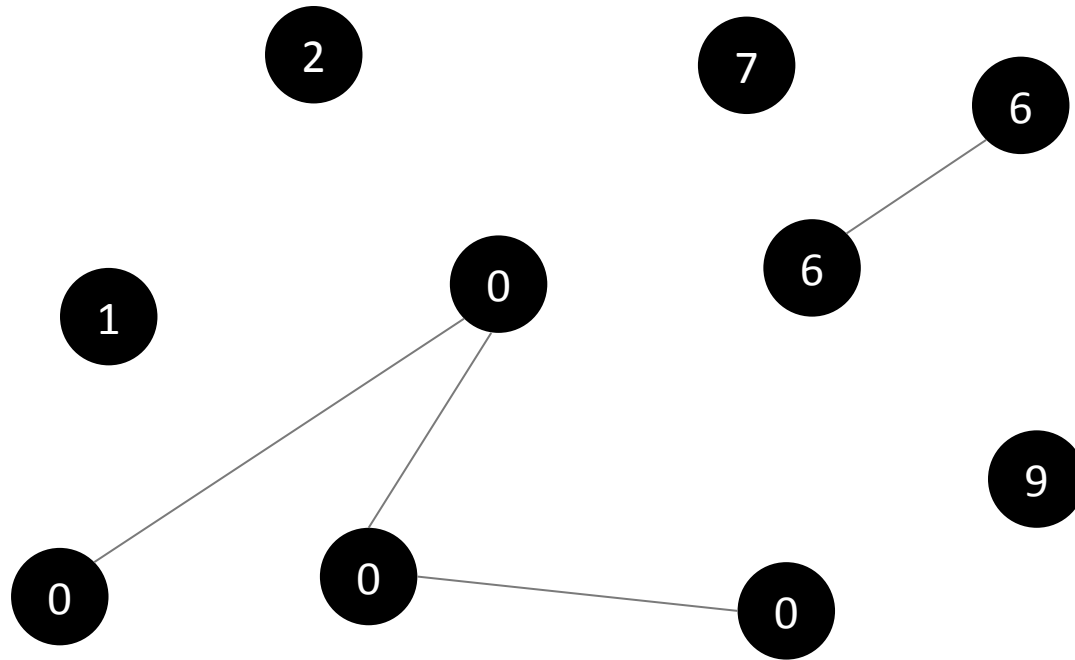
Kruskal's Algorithm



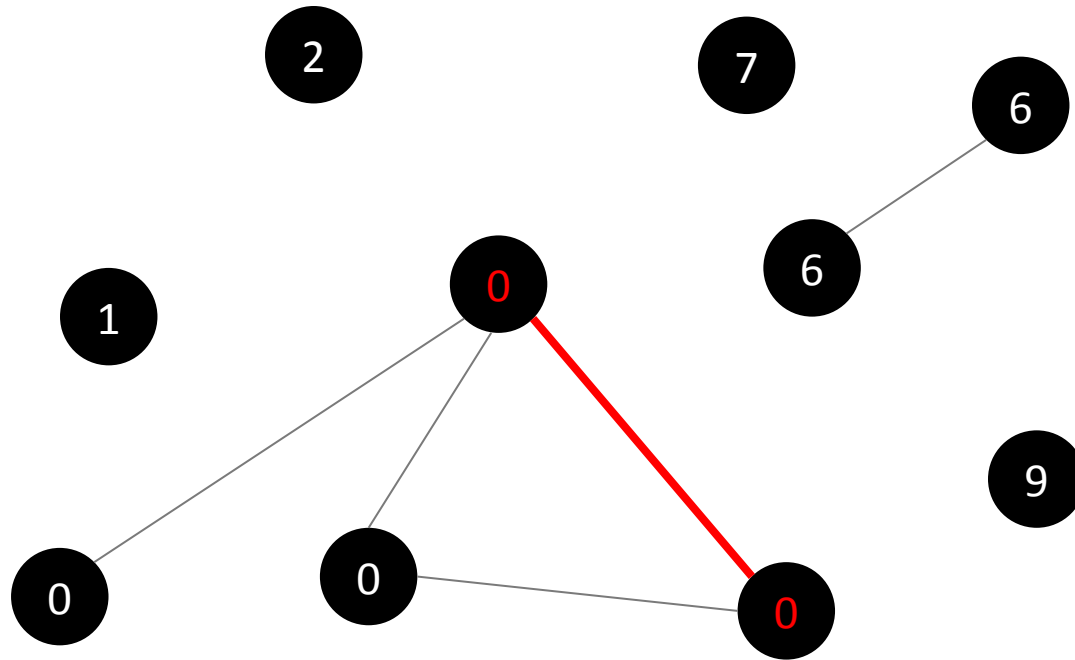
Kruskal's Algorithm



Kruskal's Algorithm



Kruskal's Algorithm



Kruskal's Algorithm

Algorithm 41 Kruskal's algorithm

```
1: function KRUSKAL( $G = (V, E)$ )
2:   sort( $E$ , key( $((u, v)) = w(u, v)$ )           // Sort edges in ascending order of weight
3:    $forest = \text{UnionFind.initialise}(n)$ 
4:    $T = (V, \emptyset)$ 
5:   for each edge  $(u, v)$  in  $E$  do
6:     if  $forest.FIND(u) \neq forest.FIND(v)$  then // Ignore edges that would create a cycle
7:        $forest.UNION(u, v)$ 
8:        $T.add\_edge(u, v)$ 
9:   return  $T$ 
```

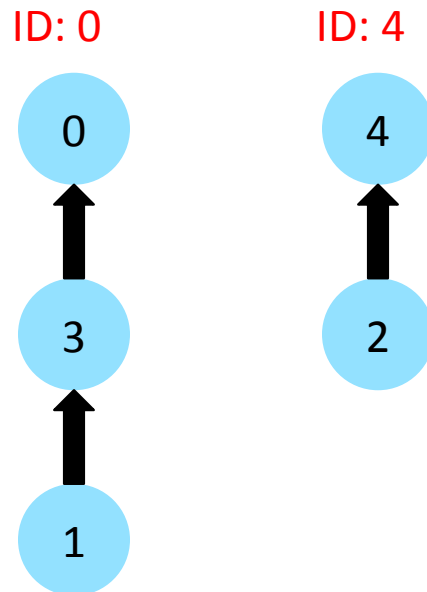
How can we quickly **find** the set of a vertex, and also **union** two sets?

Union-Find Data Structure

- Define two operations: **find(u)** and **union(u,v)**
- **Find(u)**: Given a vertex u , return its set ID
- **Union(u,v)**: Given two vertices u and v , if they have different set IDs, union the two sets they belong to (and update all the set IDs of the vertices in one of the sets)
- We need both **find(u)** and **union(u,v)** to be fast.
 - What would be the time complexity of **find(u)** and **union(u,v)** if we store the set ID of each vertex in an array?
 - **Find** would be $\theta(1)$, but **union** would be $\theta(V)$ in the worst case, thus not good enough...

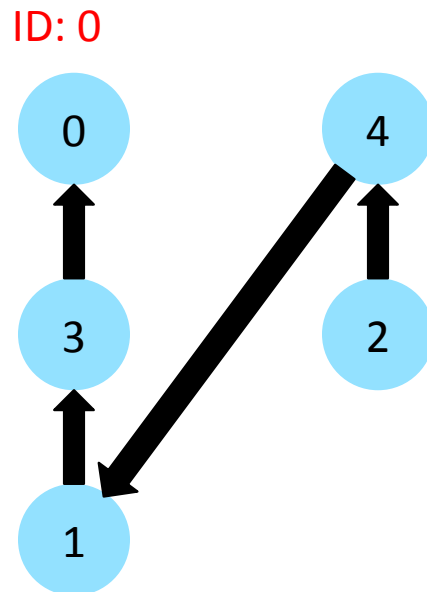
Union-Find Data Structure

- We want to union faster
- **Linked lists** are fast to union (i.e. append one linked list to another)



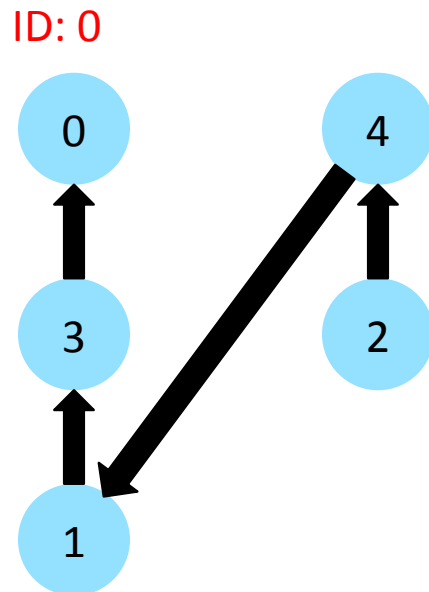
Union-Find Data Structure

- We want to union faster
- Linked lists are fast to union (i.e. append one linked list to another)



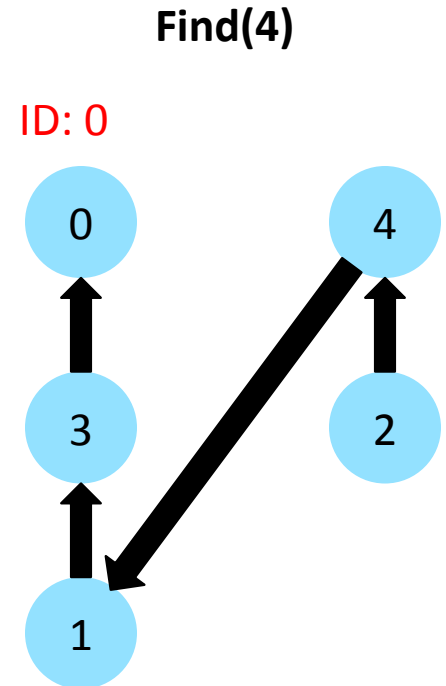
Union-Find Data Structure

- We want to union faster
- Linked lists are fast to union (i.e. append one linked list to another)
- How do we find, with linked lists?



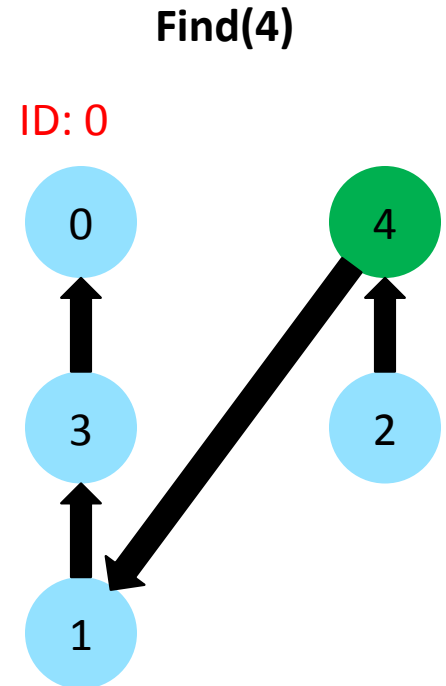
Union-Find Data Structure

- Linked lists are fast to union (i.e. append one linked list to another)
- How do we find, with linked lists?
- Heads know their set ID
- Traverse to the head to find the ID of an element in the list
- This is $O(\text{size of the linked list})$



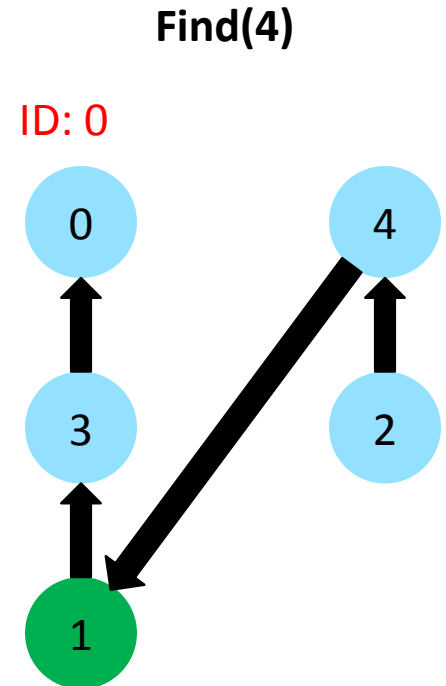
Union-Find Data Structure

- Linked lists are fast to union (i.e. append one linked list to another)
- How do we find, with linked lists?
- Heads know their set ID
- Traverse to the head to find the ID of an element in the list
- This is $O(\text{size of the linked list})$



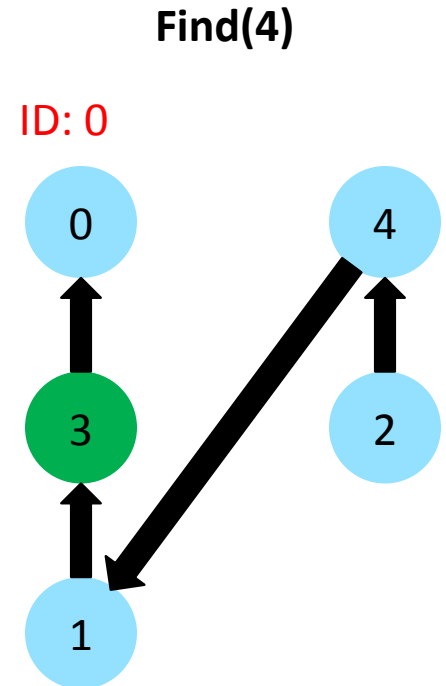
Union-Find Data Structure

- Linked lists are fast to union (i.e. append one linked list to another)
- How do we find, with linked lists?
- Heads know their set ID
- Traverse to the head to find the ID of an element in the list
- This is $O(\text{size of the linked list})$



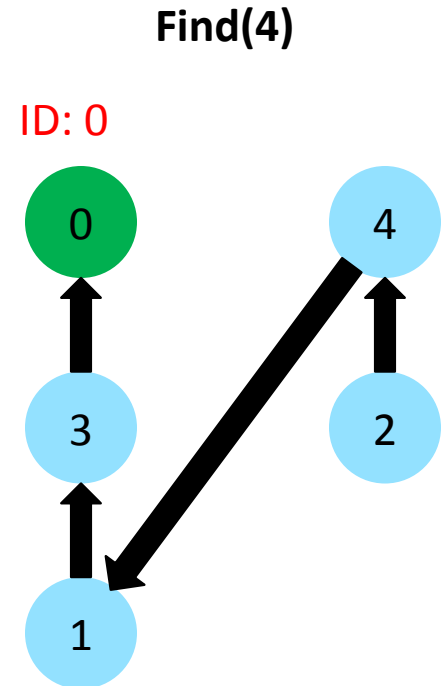
Union-Find Data Structure

- Linked lists are fast to union (i.e. append one linked list to another)
- How do we find, with linked lists?
- Heads know their set ID
- Traverse to the head to find the ID of an element in the list
- This is $O(\text{size of the linked list})$



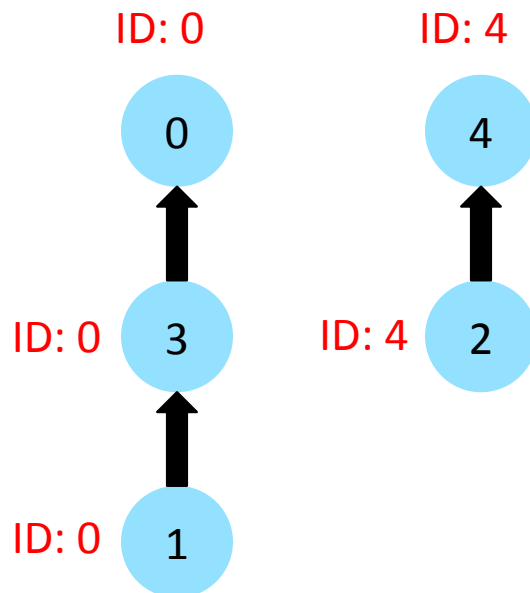
Union-Find Data Structure

- Linked lists are fast to union (i.e. append one linked list to another)
- How do we find, with linked lists?
- Heads know their set ID
- Traverse to the head to find the ID of an element in the list
- This is $O(\text{size of the linked list})$



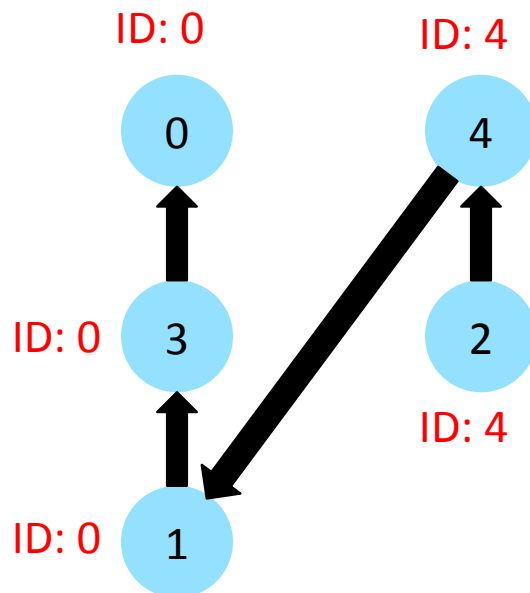
Union-Find Data Structure

- Alternatively, every node could know its ID
- Now find is $O(1)$
- But Union is now slower, we have to update all the IDs



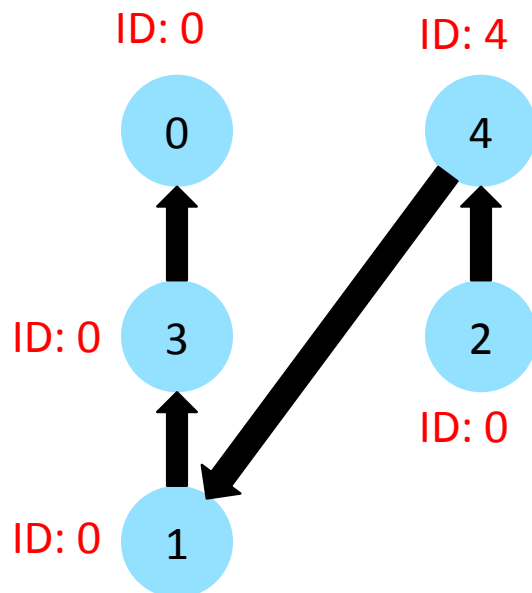
Union-Find Data Structure

- Alternatively, every node could know its ID
- Now find is $O(1)$
- But Union is now slower, we have to update all the IDs



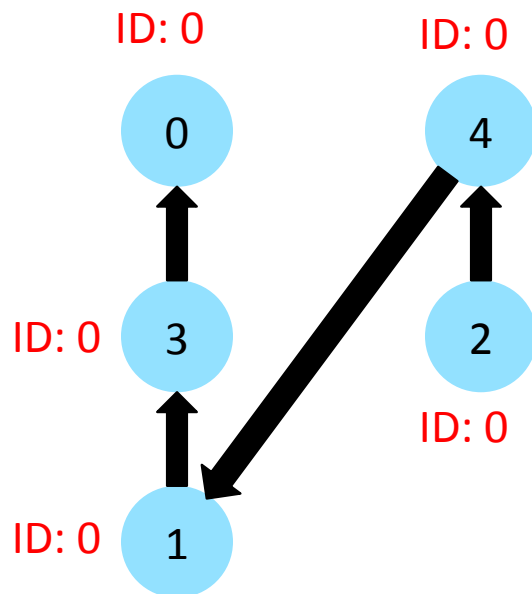
Union-Find Data Structure

- Alternatively, every node could know its ID
- Now find is $O(1)$
- But Union is now slower, we have to update all the IDs



Union-Find Data Structure

- Alternatively, every node could know its ID
- Now find is $O(1)$
- But Union is now slower, we have to update all the IDs



Union-Find Data Structure

- Where are we?
- We want find and union to be fast
- **Linked lists** are an improvement over **arrays**, since they stop us looking at items which are not relevant to the union we are currently doing
- Linked lists allows **$O(1)$ union**
- We can't make **find $O(1)$** because to do that, we have to store the ID at every node which makes union slow (we have to change all the IDs)
- Solution: Change from linked list to **linked tree**

Union-Find Data Structure

Operations:



Vertex ID	0	1	2	3	4	5
Parent	0	1	2	3	4	5

Union-Find Data Structure

Operations:

Union(0,1)

Find(0) = 0

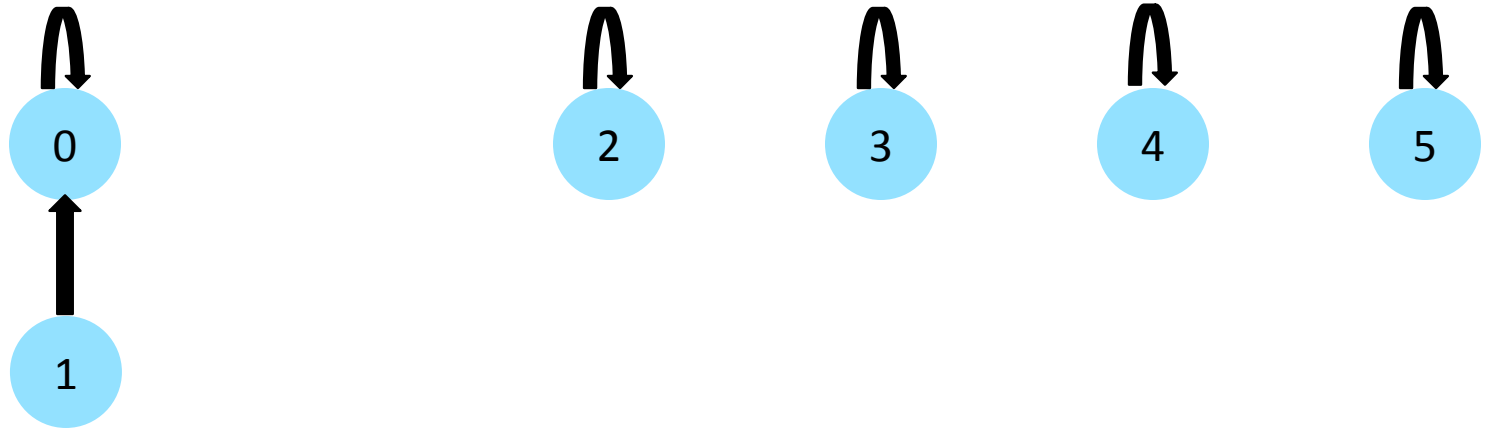
Find(1) = 1



Vertex ID	0	1	2	3	4	5
Parent	0	1	2	3	4	5

Union-Find Data Structure

Operations:
Union(0,1)



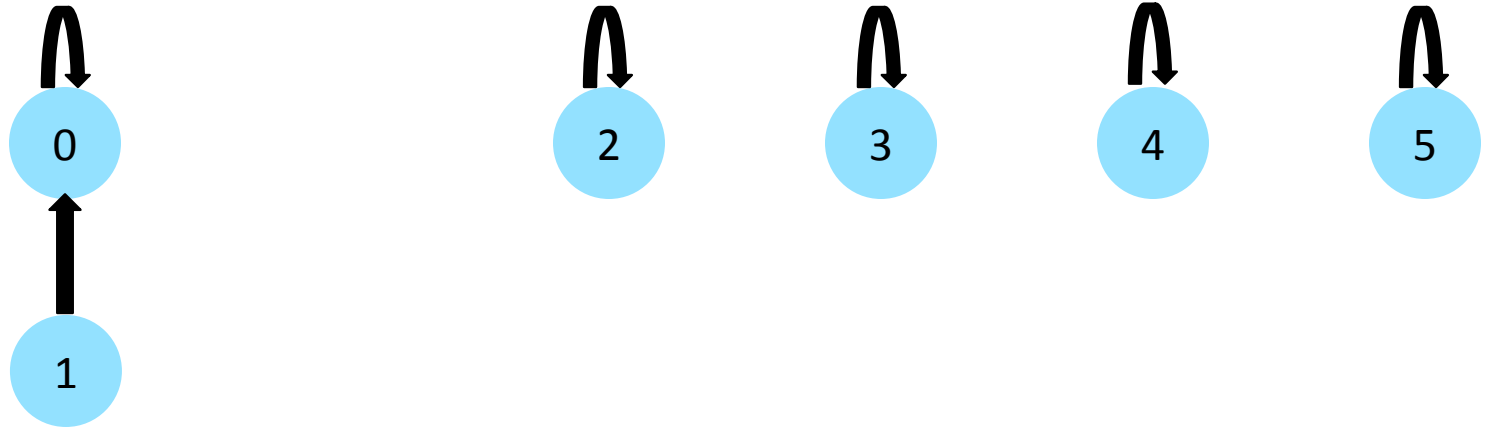
Vertex ID	0	1	2	3	4	5
Parent	0	0	2	3	4	5

Union-Find Data Structure

Operations:

Union(0,1)

Union(2,3)



Vertex ID	0	1	2	3	4	5
Parent	0	0	2	3	4	5

Union-Find Data Structure

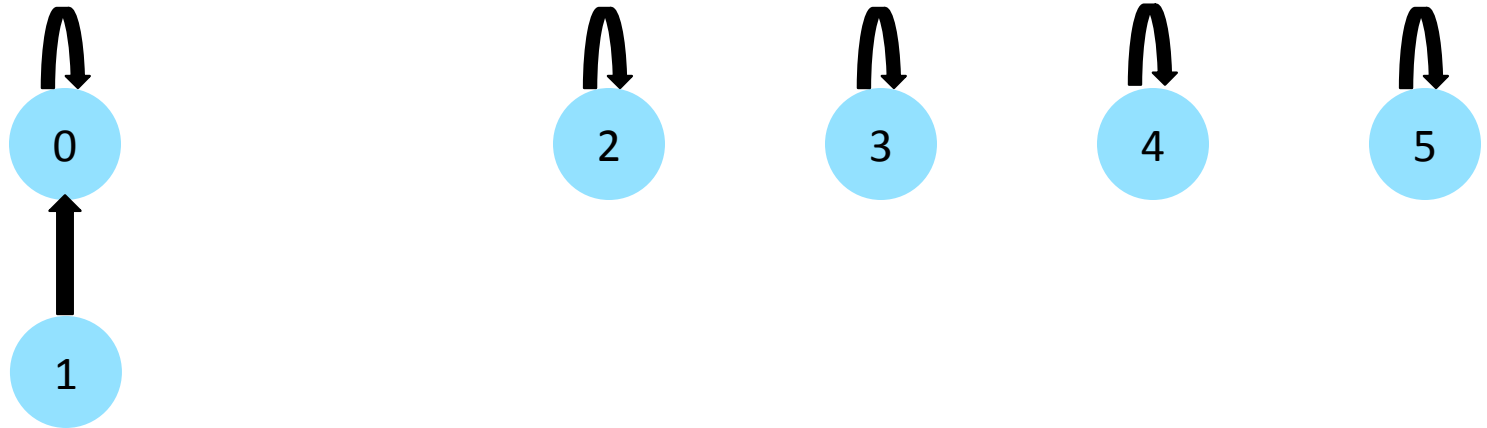
Operations:

Union(0,1)

Union(2,3)

Find(2) = 2

Find(3) = 3



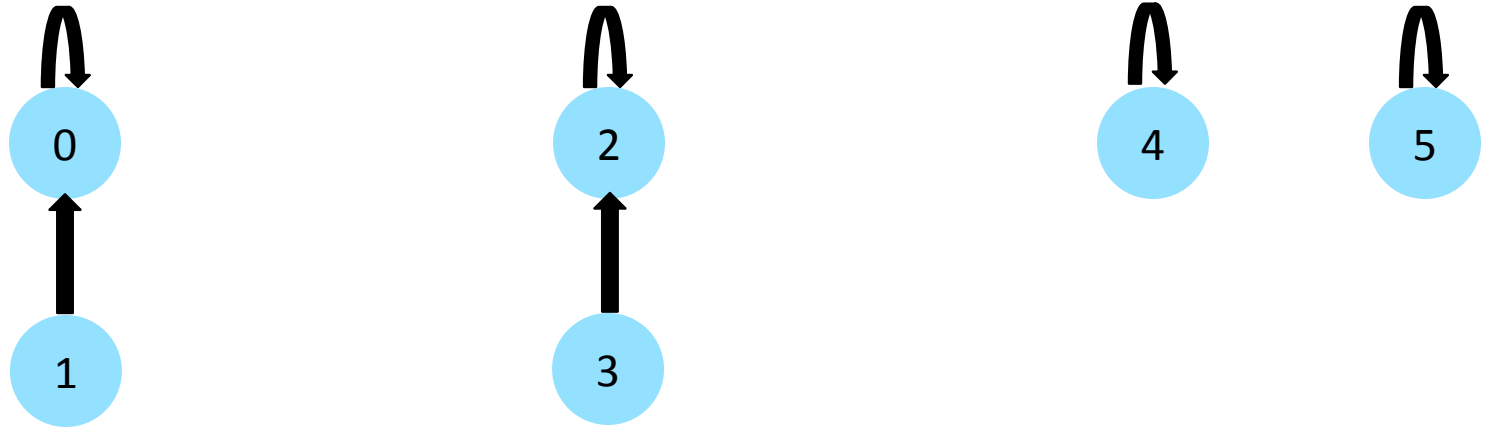
Vertex ID	0	1	2	3	4	5
Parent	0	0	2	3	4	5

Union-Find Data Structure

Operations:

Union(0,1)

Union(2,3)



Vertex ID	0	1	2	3	4	5
Parent	0	0	2	2	4	5

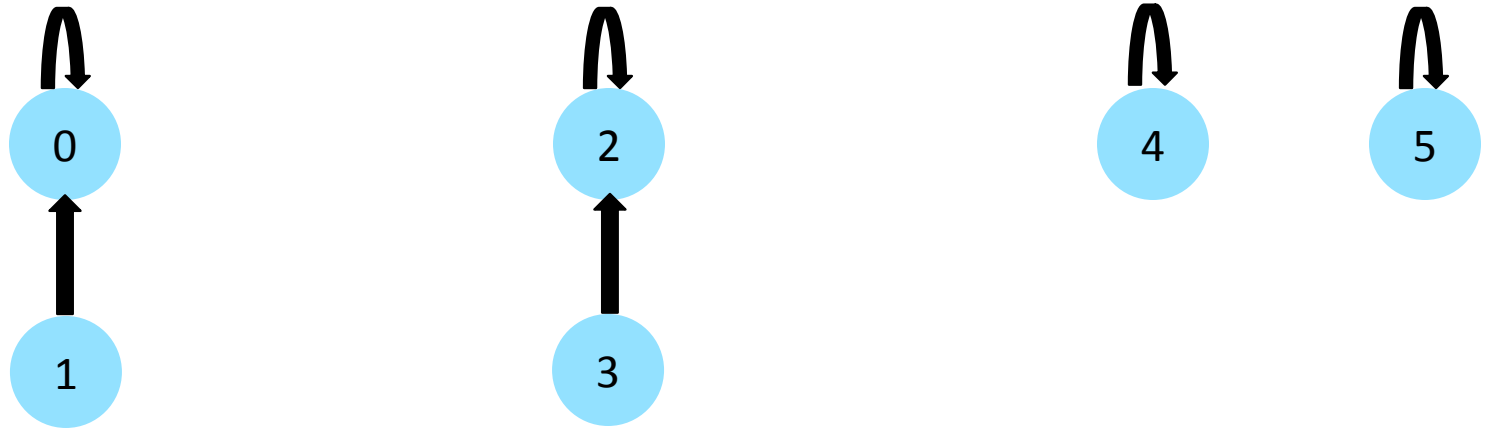
Union-Find Data Structure

Operations:

Union(0,1)

Union(2,3)

Find(3)



Vertex ID	0	1	2	3	4	5
Parent	0	0	2	2	4	5

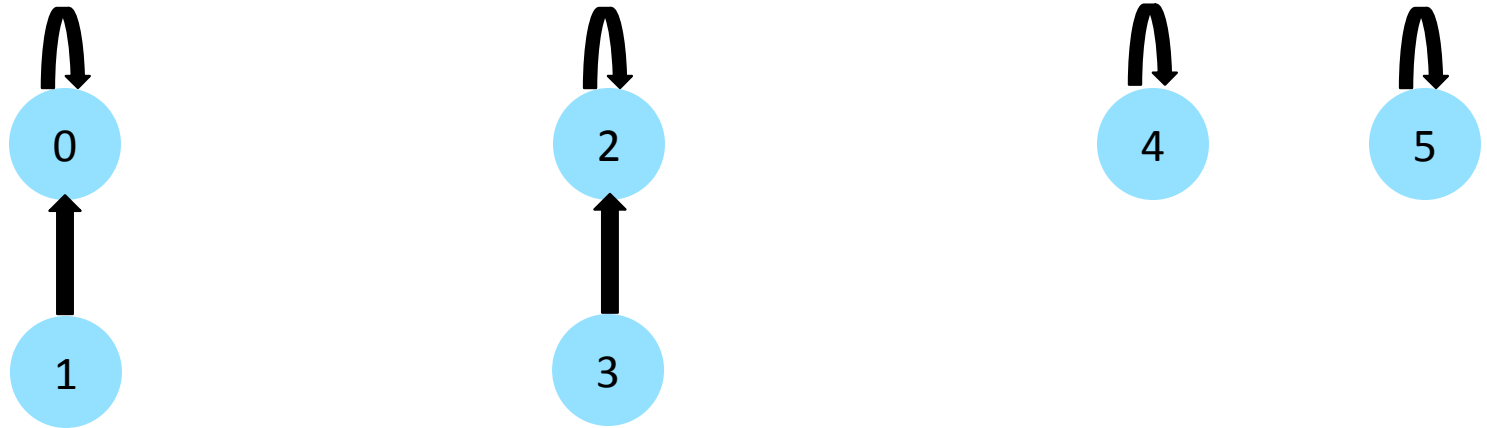
Union-Find Data Structure

Operations:

Union(0,1)

Union(2,3)

Find(3)



Vertex ID	0	1	2	3	4	5
Parent	0	0	2	2	4	5

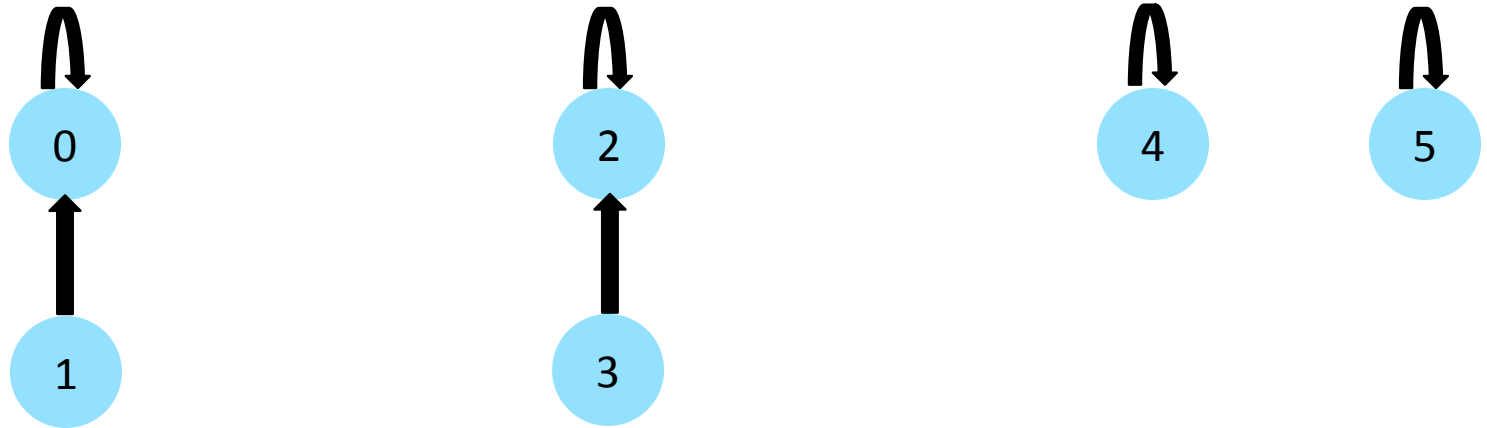
Union-Find Data Structure

Operations:

Union(0,1)

Union(2,3)

Find(3)



Vertex ID	0	1	2	3	4	5
Parent	0	0	2	2	4	5

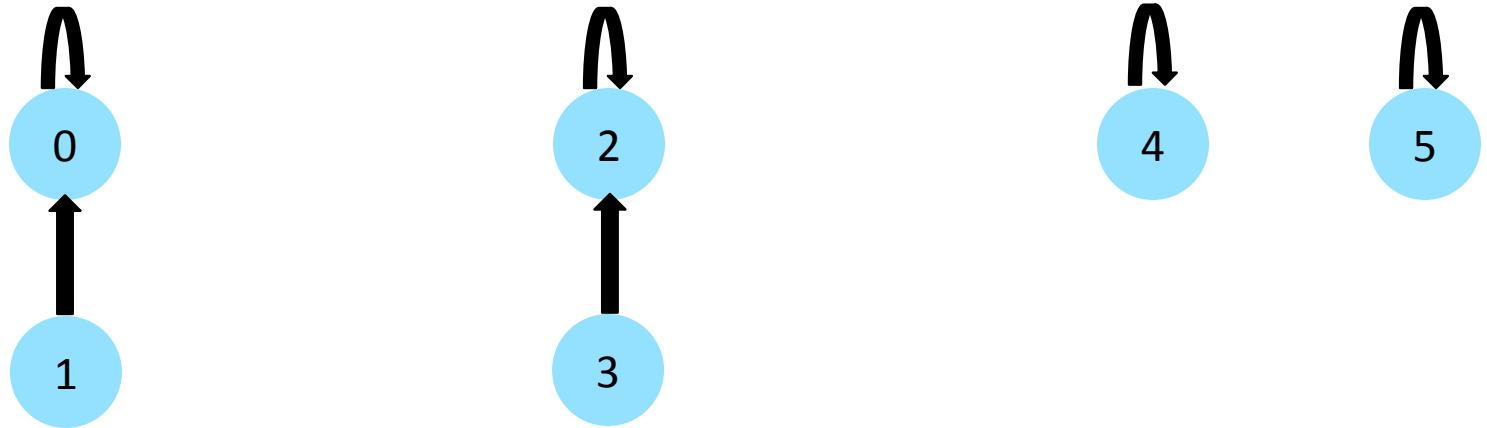
Union-Find Data Structure

Operations:

Union(0,1)

Union(2,3)

Find(3)



Vertex ID	0	1	2	3	4	5
Parent	0	0	2	2	4	5

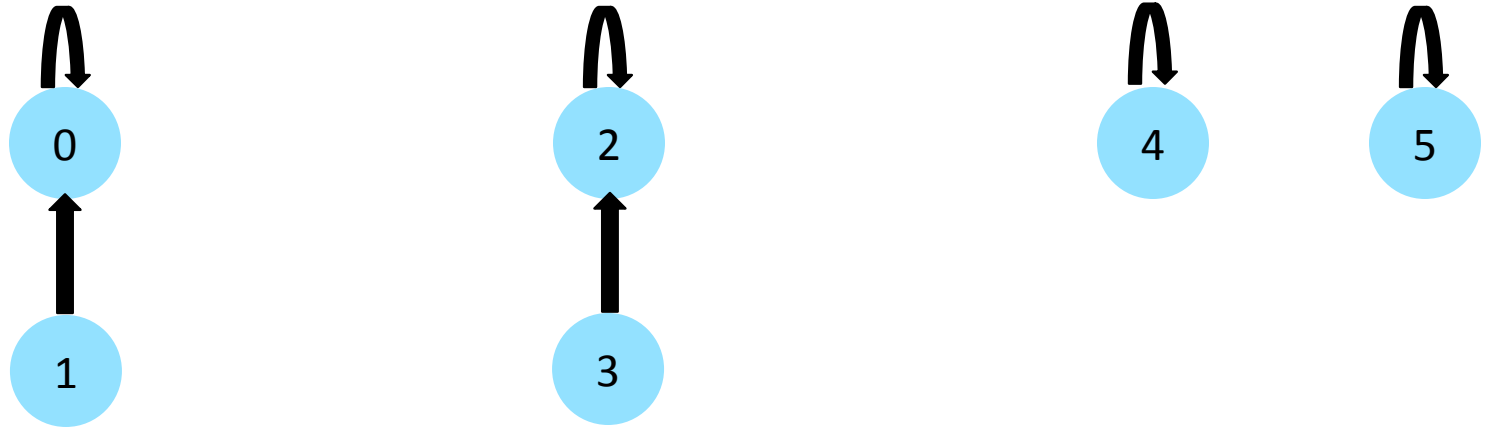
Union-Find Data Structure

Operations:

Union(0,1)

Union(2,3)

Find(3)=2



Vertex ID	0	1	2	3	4	5
Parent	0	0	2	2	4	5

Union-Find Data Structure

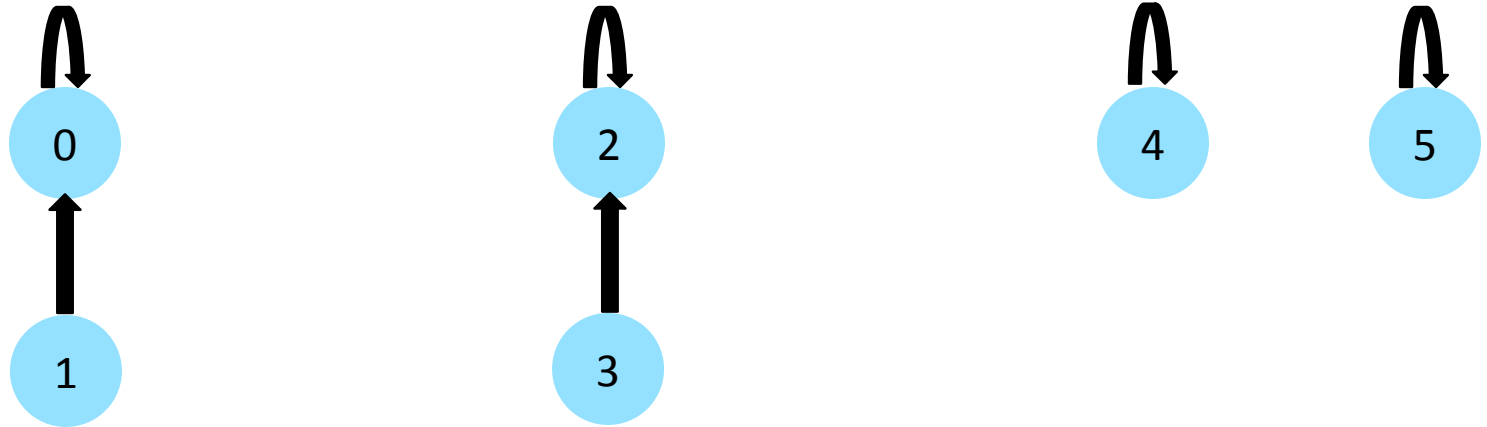
Operations:

Union(0,1)

Union(2,3)

Find(3)=2

Union(0,1)



Vertex ID	0	1	2	3	4	5
Parent	0	0	2	2	4	5

Union-Find Data Structure

Operations:

Union(0,1)

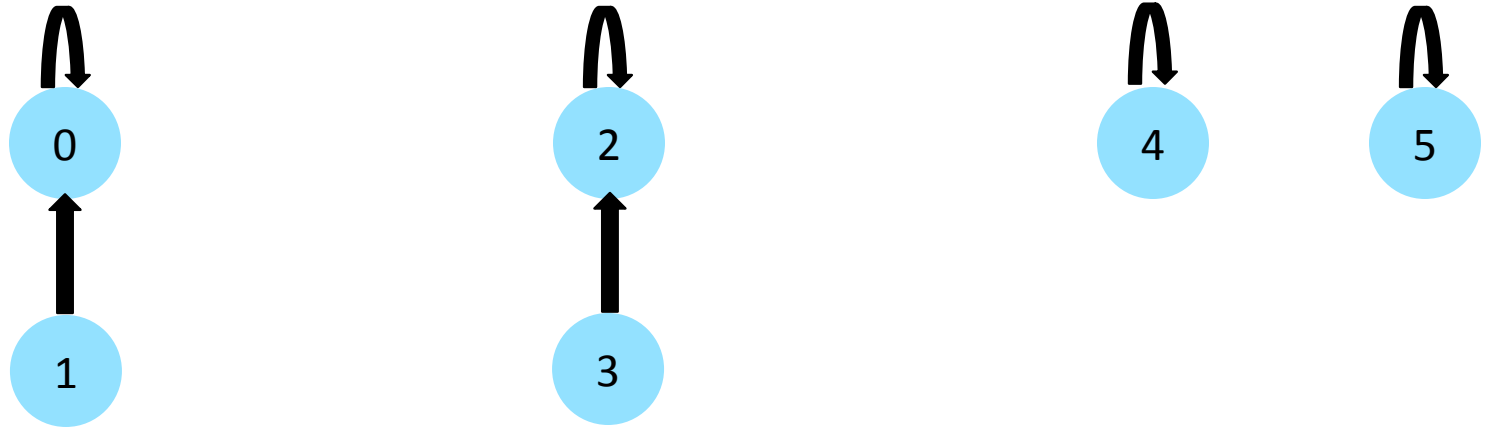
Union(2,3)

Find(3)=2

Union(0,1)

Find(0) = 0

Find(1) = 0



Vertex ID	0	1	2	3	4	5
Parent	0	0	2	2	4	5

Union-Find Data Structure

Operations:

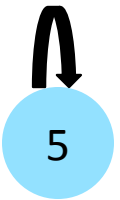
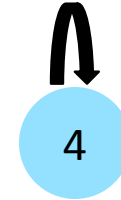
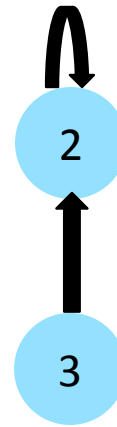
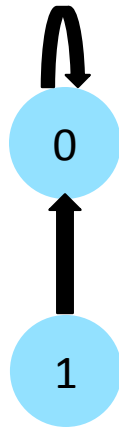
Union(0,1)

Union(2,3)

Find(3)=2

Union(0,1)

Union(1,3)



Vertex ID	0	1	2	3	4	5
Parent	0	0	2	2	4	5

Union-Find Data Structure

Operations:

Union(0,1)

Union(2,3)

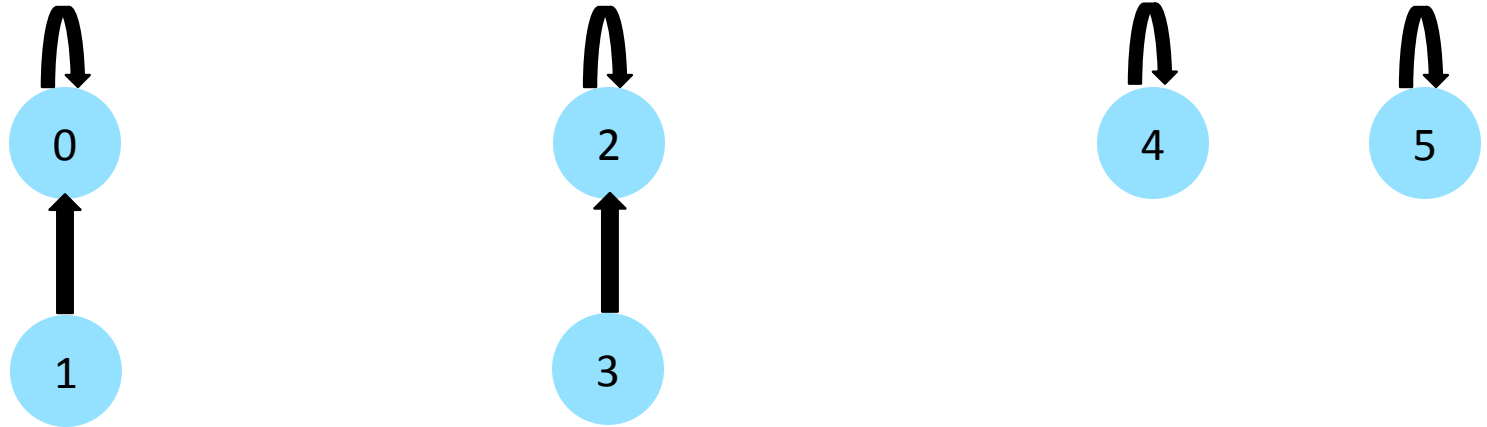
Find(3)=2

Union(0,1)

Union(1,3)

Find(1) = 0

Find(3) = 2



Vertex ID	0	1	2	3	4	5
Parent	0	0	2	2	4	5

Union-Find Data Structure

Operations:

Union(0,1)

Union(2,3)

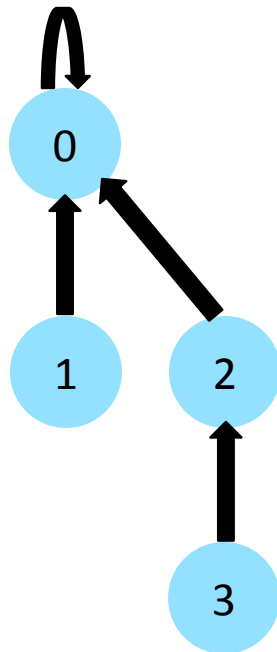
Find(3)=2

Union(0,1)

Union(1,3)

Find(1) = 0

Find(3) = 2



Vertex ID	0	1	2	3	4	5
Parent	0	0	0	2	4	5

Union-Find Data Structure

Operations:

Union(0,1)

Union(2,3)

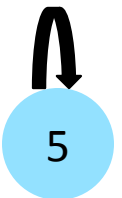
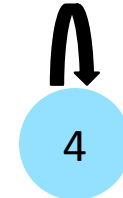
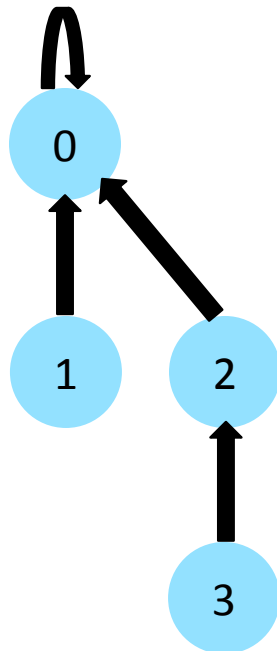
Find(3)=2

Union(0,1)

Union(1,3)

Find(1) = 0

Find(3) = 2



Find: traverse parent pointers until you find a vertex who is its own parent (i.e. a root). That vertex ID is the set ID

Union(u,v): If $\text{find}(u) \neq \text{find}(v)$, set $\text{parent}[\text{find}(u)] = \text{find}(v)$ (or vice versa)

What is the complexity of union and find?

Union is $O(\text{find})$. Find could in theory be $\theta(V)$, but if we can keep the heights of the trees low, then it will be at most $\theta(\text{max height})$

Vertex ID	0	1	2	3	4	5
Parent	0	0	0	2	4	5

Union-Find Data Structure

Operations:

Union(0,1)

Union(2,3)

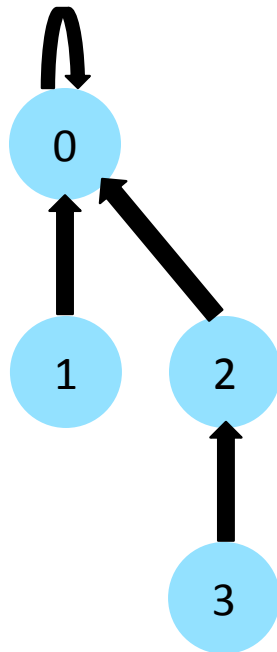
Find(3)=2

Union(0,1)

Union(1,3)

Find(1) = 0

Find(3) = 2



Optimisation: When we union, we have a choice of the new root.

What should we choose?

The set with more nodes!

Vertex ID	0	1	2	3	4	5
Parent	0	0	0	2	4	5

Union-Find Data Structure

Operations:

Union(0,1)

Union(2,3)

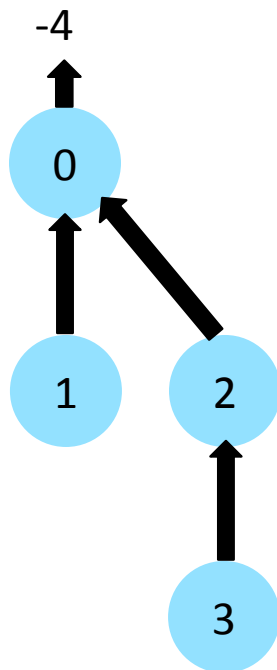
Find(3)=2

Union(0,1)

Union(1,3)

Find(1) = 0

Find(3) = 2



Optimisation: When we union, we have a choice of the new root.

What should we choose?

The set with more nodes!

Union by size: When doing a union, add the sizes and update the parent value of the root appropriately



Vertex ID	0	1	2	3	4	5
Parent	-4	0	0	2	-1	-1

Parent array values are now either parents OR negative sizes

Kruskal's Algorithm: Complexity

Algorithm 41 Kruskal's algorithm

```
1: function KRUSKAL( $G = (V, E)$ )
2:    $\text{sort}(E, \text{key}((u, v)) = w(u, v))$  // Sort edges in ascending order of weight
3:    $\text{forest} = \text{UnionFind.initialise}(n)$ 
4:    $T = (V, \emptyset)$ 
5:   for each edge  $(u, v)$  in  $E$  do
6:     if  $\text{forest.FIND}(u) \neq \text{forest.FIND}(v)$  then // Ignore edges that would create a cycle
7:        $\text{forest.UNION}(u, v)$ 
8:        $T.\text{add\_edge}(u, v)$ 
9:   return  $T$ 
```

But this is not tight. A closer look reveals that the total cost of all UNION_SETS is $\Theta(V \log V)$.

Time Complexity:

- Sorting edges: $\Theta(E \log E)$
 - $E \log E \leq E \log V^2 = 2 E \log V$ so $\Theta(E \log V)$
- Initialization of union find: $\Theta(V)$
- For loop executes $\Theta(E)$ times
 - SET_ID() takes $\Theta(x)$ where x is height of the tree
 - UNION_SET() takes $\Theta(1) + 2$ finds, so it takes $\Theta(x)$ where x is the height of the deeper of the two trees to be unioned (which could be at most V)
- Total cost: $O(EV)$

Complexity of UNION_SETS

- We can show that, when using the **union-by-size** rule, the number of elements N in any tree is at least 2^h , where h is the height of the tree
- In other words, $h \leq \log_2 N$.
- Union takes 2 finds + $O(1)$ effort.
- Find takes effort equal to the height of the tree, which is $\leq \log_2 N$.
- So union is $O(\log_2 N)$, where N is the number of nodes of the taller tree being unioned.
- We need to do $V-1$ unions.
- Each one has a worst case cost of $O(\log V)$ (this is a significant over-estimation, but it makes the maths easy).
- So the **total cost of all unions** is bounded by $O(V \log V)$.

Kruskal's Algorithm: Complexity

Algorithm 41 Kruskal's algorithm

```
1: function KRUSKAL( $G = (V, E)$ )
2:    $\text{sort}(E, \text{key}((u, v)) = w(u, v))$  // Sort edges in ascending order of weight
3:    $\text{forest} = \text{UnionFind.initialise}(n)$ 
4:    $T = (V, \emptyset)$ 
5:   for each edge  $(u, v)$  in  $E$  do
6:     if  $\text{forest.FIND}(u) \neq \text{forest.FIND}(v)$  then // Ignore edges that would create a cycle
7:        $\text{forest.UNION}(u, v)$ 
8:        $T.\text{add\_edge}(u, v)$ 
9:   return  $T$ 
```

Time Complexity:

- Sorting edges: $\Theta(E \log E)$
 - $E \log E = E \log V^2 = 2 E \log V$ so $\Theta(E \log V)$
- Initialization of union find: $\Theta(V)$
- For loop executes $\Theta(E)$ times
 - The two finds take the same effort as the union, $\log(v)$
- UNION takes $\Theta(V \log V)$ in total
- Total cost: $\Theta(E \log V + V \log V) = \Theta(E \log V)$

Complexity of UNION_SETS

- We can improve the disjoint sets data structure significantly with 2 other optimisations:
 - Union by rank
 - Path compression
- These are discussed in the notes, but are not examinable.
- The complexity can be improved from $V \log V$ to $V\alpha(V)$, where α denotes the inverse Ackermann function, an **extremely** slow growing function.
- Note: $\alpha(\text{any number which can be represented using the matter in the universe}) < 5$, so $V\alpha(V)$ is effectively $O(V)$.

Kruskal's Algorithm: Correctness

- The proof of correctness for Kruskal's algorithm can be done following the same approach as Prim's one. Please try to write it down by yourself and check the details on the course notes.

Kruskal's Algorithm: Correctness

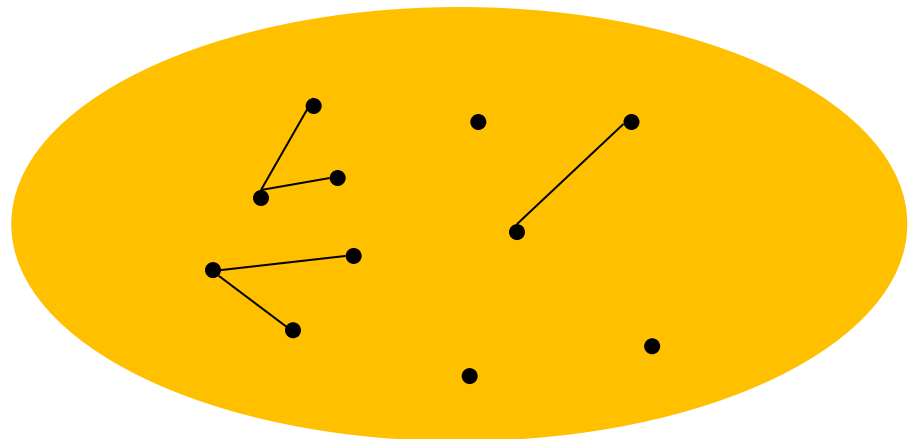
#INV: Every iteration of Kruskal's algorithm, the current set of selected edges in T is a subset of some minimum spanning tree of G .

Base Case:

- The invariant is true initially when T is empty.

Inductive step:

- **We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration.**
- Assume T is a subset of some MST M .



Kruskal's Algorithm: Correctness

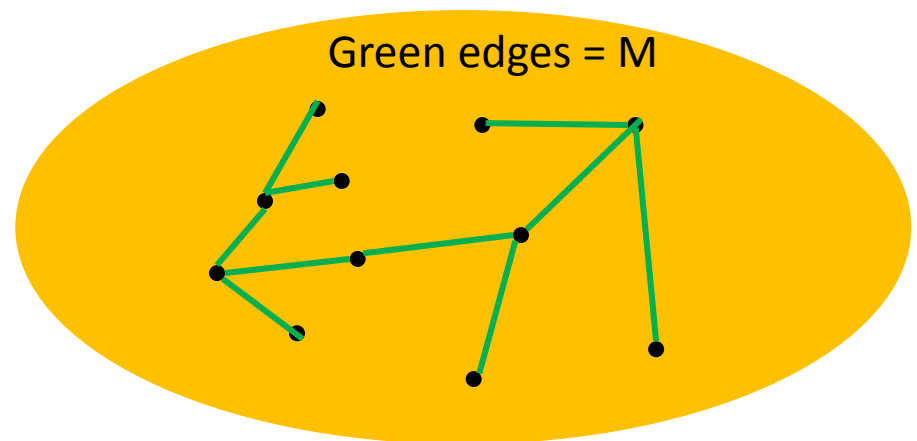
#INV: Every iteration of Kruskal's algorithm, the current set of selected edges in T is a subset of some minimum spanning tree of G .

Base Case:

- The invariant is true initially when T is empty.

Inductive step:

- **We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration.**
- Assume T is a subset of some MST M .



Kruskal's Algorithm: Correctness

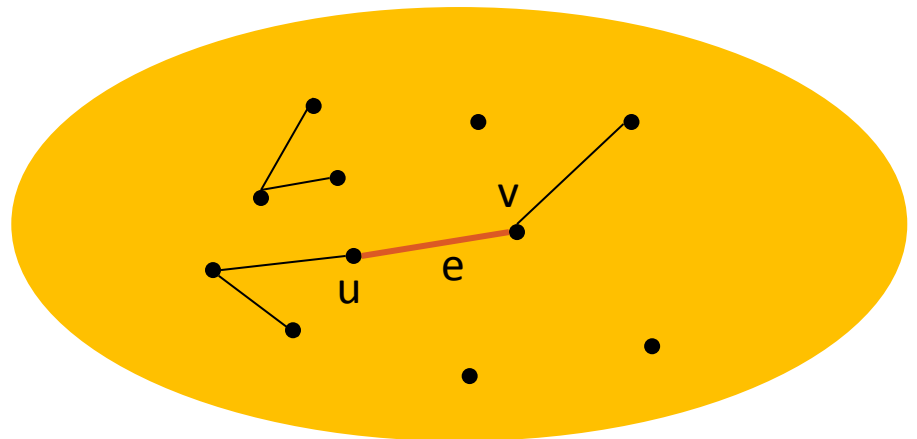
#INV: Every iteration of Kruskal's algorithm, the current set of selected edges in T is a subset of some minimum spanning tree of G .

Base Case:

- The invariant is true initially when T is empty.

Inductive step:

- **We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration.**
- Assume T is a subset of some MST M .
- Let $e = (u, v)$ be the lightest edge that connects two vertices in different connected components of T (i.e this is the edge Kruskal's will choose in this iteration).
- If e is in M , then $T \cup \{e\}$ is a subset of M , which is an MST, so the invariant holds.



Kruskal's Algorithm: Correctness

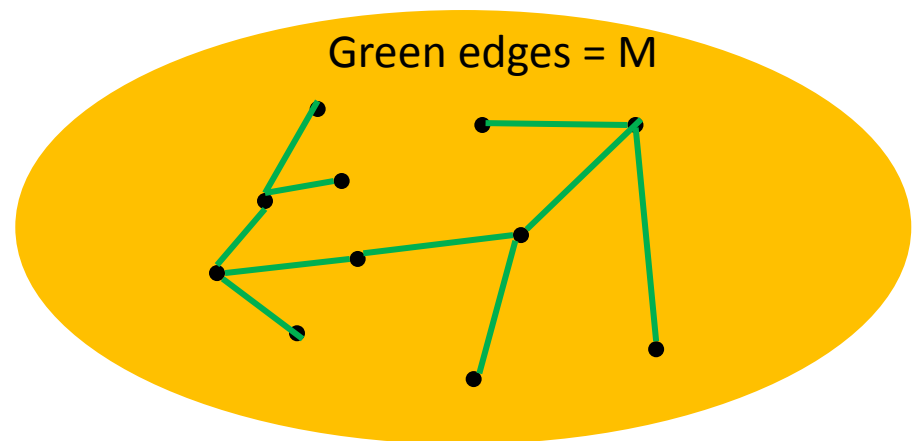
#INV: Every iteration of Kruskal's algorithm, the current set of selected edges in T is a subset of some minimum spanning tree of G .

Base Case:

- The invariant is true initially when T is empty.

Inductive step:

- **We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration.**
- Assume T is a subset of some MST M .
- Let $e = (u,v)$ be the lightest edge that connects two vertices in different connected components of T (i.e this is the edge Kruskal's will choose in this iteration).
- If e is in M , then $T \cup \{e\}$ is a subset of M , which is an MST, so the invariant holds.



Kruskal's Algorithm: Correctness

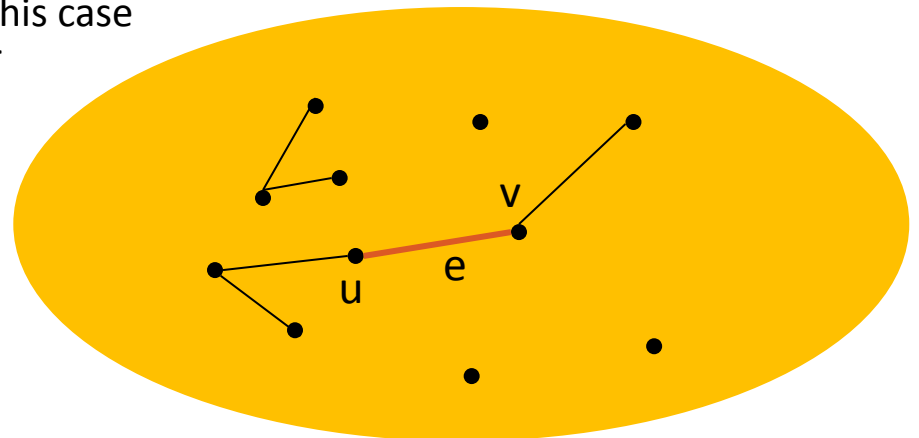
#INV: Every iteration of Kruskal's algorithm, the current set of selected edges in T is a subset of some minimum spanning tree of G .

Base Case:

- The invariant is true initially when T is empty.

Inductive step:

- **We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration.**
- Assume T is a subset of some MST M .
- Let $e = (u, v)$ be the lightest edge that connects two vertices in different connected components of T (i.e this is the edge Kruskal's will choose in this iteration).
- If e is in M , then $T \cup \{e\}$ is a subset of M , which is an MST, so the invariant holds.
- The interesting case is where e is not in M . In this case we have to show that there is some other MST which contains $T \cup \{e\}$.



Kruskal's Algorithm: Correctness

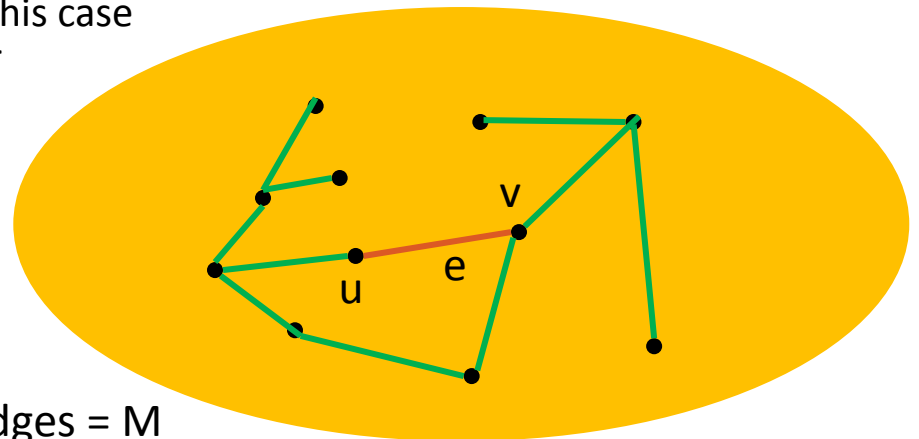
#INV: Every iteration of Kruskal's algorithm, the current set of selected edges in T is a subset of some minimum spanning tree of G

Base Case:

- The invariant is true initially when T is empty

Inductive step:

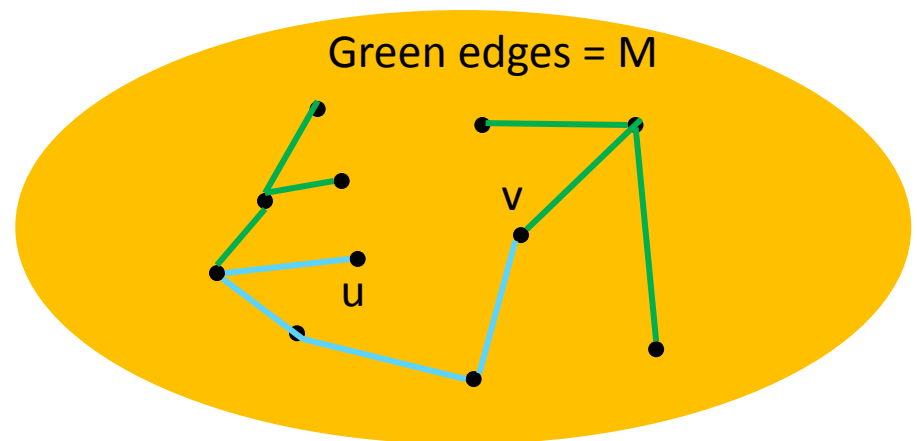
- **We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration**
- Assume T is a subset of some MST M
- Let $e = (u, v)$ be the lightest edge that connects two vertices in different connected components of T (i.e. this is the edge Kruskal's will choose in this iteration).
- If e is in M , then $T \cup \{e\}$ is a subset of M , which is an MST, so the invariant holds
- The interesting case is where e is not in M . In this case we have to show that there is some other MST which contains $T \cup \{e\}$.



Kruskal's Algorithm: Correctness

Inductive step:

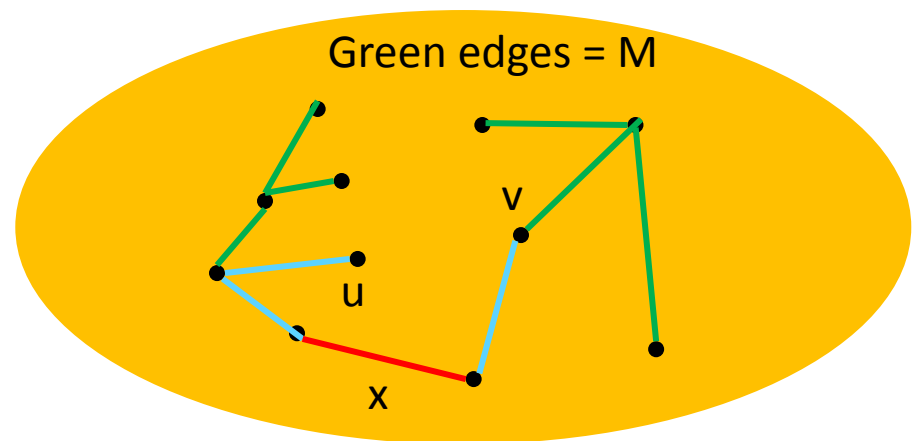
- We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration.
- Since M is a tree, there is exactly one path from u to v in M (shown in blue).
- u and v are not connected in T (since we are adding (u,v) , and we never create a cycle).
- Consider the first edge on the blue path which is **not** contained in T (call this edge x).



Kruskal's Algorithm: Correctness

Inductive step:

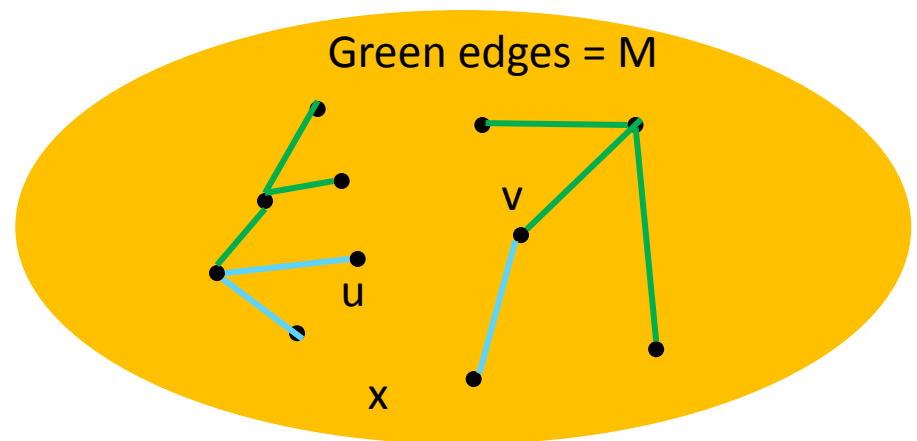
- We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration.
- Since M is a tree, there is exactly one path from u to v in M (shown in blue).
- u and v are not connected in T (since we are adding (u,v) , and we never create a cycle).
- Consider the first edge on the blue path which is **not** contained in T (call this edge x).
- At least one edge on the blue path is not currently in T (otherwise u and v would be connected in T).
- Removing this edge would disconnect M .



Kruskal's Algorithm: Correctness

Inductive step:

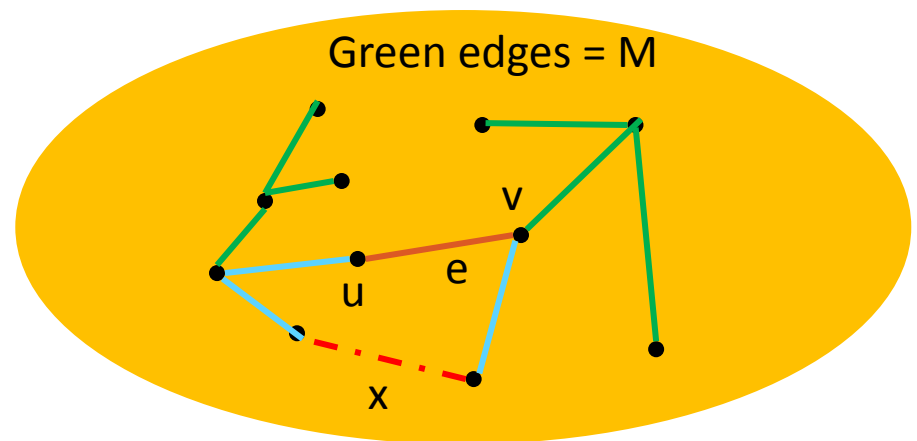
- We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration.
- Since M is a tree, there is exactly one path from u to v in M (shown in blue).
- u and v are not connected in T (since we are adding (u,v) , and we never create a cycle).
- Consider the first edge on the blue path which is **not** contained in T (call this edge x).
- At least one edge on the blue path is not currently in T (otherwise u and v would be connected in T).
- Removing this edge would disconnect M .



Kruskal's Algorithm: Correctness

Inductive step:

- We want to show that, if T is a subset of some MST at the start of some iteration, it is still a subset of some MST at the start of the next iteration.
- Since M is a tree, there is exactly one path from u to v in M (shown in blue).
- u and v are not connected in T (since we are adding (u,v) , and we never create a cycle).
- Consider the first edge on the blue path which is **not** contained in T (call this edge x).
- At least one edge on the blue path is not currently in T (otherwise u and v would be connected in T).
- Removing this edge would disconnect M .
- Adding the edge (u,v) would form a new spanning tree, M' .
- Since the algorithm always selects the lightest edge incident to T , we know that $w(e) \leq w(x)$.
- So the weight of M' is no greater than the weight of M , therefore choosing e is correct.



Reading

- Course Notes: Chapter 6
- You can also check algorithms' textbooks for contents related to this lecture, e.g.:
 - CLRS: Chapter 23, Section 24.3
 - KT: Chapter 4

Concluding Remarks

Take home message

- Dijkstra's algorithm is a greedy algorithm for determining shortest paths on graphs with non-negative weights.
- Prim's and Kruskal's algorithms are both greedy algorithms that correctly determine minimum spanning trees. While Prim's algorithm keeps growing the same connected component at each iteration; Kruskal's algorithm merges the two closest connected components.

Things to do (this list is not exhaustive)

- Make sure you understand:
 - The algorithms, and how to implement Union-Find data structure for Kruskal's algorithm.
 - How proofs of correctness for greedy algorithms work.

Coming Up Next

- Dynamic Programming